

PROJET II

Intégration BC Solution Externe & Tableau de Bord Interactif

*Kilométrage des véhicules de livraison
via API REST — Analyse des coûts logistiques par produit*

Auteur	Nyanu Kodzo Selom
Formation	02. Informatique de gestion ES — 3ING-SE
Établissement	CPNE-CG — Maladière, Neuchâtel
Enseignant	Projet d'intégration — Projet II
Date de remise	26 juin 2026
Version	3.0.0 — Rapport final

Table des Matières

Table des Matières.....	2
Résumé Synthétique.....	5
1. Contexte et Problématique.....	6
1.1 Contexte métier.....	6
1.2 Problématique.....	6
1.3 Objectifs du projet.....	6
2. Spécifications Fonctionnelles.....	7
2.1 Cas d'usage principaux.....	7
2.2 Règles de validation Business Central.....	7
2.3 Contrainte de licence BC — Contournement Real Session Date.....	7
3. Choix et Justification de la Méthode d'Accès.....	8
3.1 Comparaison des alternatives.....	8
3.2 Justification du choix OData v1.0.....	8
3.3 Architecture d'accès BC via proxy NTLM.....	8
4. Conception des Tables BC (Extension AL v2.2.0.0).....	9
4.1 Vue d'ensemble.....	9
4.2 Table ETL BC Distance (table centrale — scénario kilométrage).....	9
4.3 Autres tables AL.....	9
5. Modélisation du Flux ETL.....	10
5.1 Architecture médaillon.....	10
5.2 Flux GPS temps réel (scénario kilométrage).....	10
5.3 Flux synchronisation quotidienne (WF-08 à 02h00).....	12
5.4 Flux BC → Gold (WF-09BC — événementiel temps réel).....	13
6. Documentation Technique de la Solution.....	14
6.1 Infrastructure Docker.....	14
6.2 Schéma de la base de données Gold (bc_gold).....	14
6.3 Workflows n8n — ETL_Projet_BC.....	14
7. Réalisation de l'Infrastructure.....	15

7.1 Environnement virtualisé.....	15
7.2 État de l'infrastructure au moment de la remise.....	15
8. Développement des Flux de Connexion.....	16
8.1 Connexion GPS → n8n (API REST).....	16
8.2 Connexion n8n → Business Central (OData + NTLM).....	16
8.3 Connexion BC → Gold (WF-09BC).....	16
9. Automatisation des Flux.....	17
9.1 Planification des déclencheurs.....	17
9.2 Mécanisme d'idempotence.....	17
10. Tests Fonctionnels & Intégrité des Données.....	18
10.1 Protocole de tests.....	18
10.2 Résultats de l'état Gold au moment de la remise.....	18
11. Tableau de Bord Metabase.....	19
11.1 Vue d'ensemble des dashboards.....	19
11.2 Gestion des rôles et permissions.....	19
11.3 Cockpit de Pilotage #21 — Architecture 3 zones.....	19
11.4 Visualisations — exigences pédagogiques.....	19
11.4 Publication et accessibilité.....	20
12. Analyse des Risques, Coûts & Business Cases.....	21
12.1 Analyse des risques.....	21
12.3 Business case et ROI.....	22
12.3.1. Méthodologie d'évaluation.....	22
12.3.2. Synthèse et recommandation.....	22
13. Planification & Gestion des Écarts.....	24
13.1 Tableau de planification.....	24
13.2 Analyse des écarts.....	24
14. Bilan Critique, Limites et Perspectives.....	25
14.1 Limites techniques reconnues.....	25
14.2 Ce qui serait fait différemment avec plus de temps.....	25

14.3 Perspectives d'extension.....	25
15. Sources et Références.....	26
ANNEXE A — Cahier des Charges.....	27
A.1 Contexte et enjeux.....	27
A.2 Objectifs.....	27
A.3 Périmètre — Dans le périmètre.....	27
A.4 Périmètre — Hors périmètre.....	27
A.5 Exigences fonctionnelles.....	27
A.6 Exigences non-fonctionnelles.....	28
A.7 Glossaire.....	28
ANNEXE B — Captures d'Écran Metabase.....	29
B.1 Profil Administrateur — Accueil.....	29
B.2 Administrateur — Browse Databases.....	30
B.3 Administrateur — Gestion des utilisateurs.....	30
B.4 Dashboard #21 — Cockpit Logistique ETL_Projet_BC (Admin).....	31
B.5 Dashboard #19 — Ventes ETL_Projet_BC (Admin).....	33
B.6 Dashboard #20 — Direction ETL_Projet_BC (Admin).....	34
B.7 Profil Ventes — Accueil (Marie Dupont).....	35
B.8 Profil Ventes — Dashboard #19 Ventes ETL_Projet_BC.....	36
B.9 Profil Ventes — Dashboard #20 Direction ETL_Projet_BC.....	37
B.10 Profil Logistique — Accueil (Jean Logistique).....	38
B.11 Profil Logistique — Dashboard #21 Cockpit Logistique ETL_Projet_BC.....	39
B.12 Profil Logistique — Dashboard #20 Direction ETL_Projet_BC.....	40
Annexe C — Scripts et code source.....	41
C.1 Extension AL — Tables (Table50200 – Table50205).....	41
C.2 Extension AL — API Pages OData (APIPage50200 – APIPage50214).....	47
C.3 Extension AL — Pages Liste et Codeunit Import Manager.....	51
C.4 Migration SQL — schema_gold_updates.sql.....	62
C.5 Python — import_workflows.py.....	64

C.6 n8n JavaScript — WF-01BC : data.csv + data.json → MinIO Bronze.....	67
C.7 n8n JavaScript — WF-03BC : PostgreSQL Windows → MinIO Bronze.....	69
C.8 n8n JavaScript — WF-04B : Distance App → MinIO (Webhook temps réel).....	76
C.9 n8n JavaScript — WF-06 : gestion_logistique_vehicules.json → bc_gold.....	80
C.10 n8n JavaScript — WF-08 : Orchestrateur Bronze → BC.....	82
C.11 n8n JavaScript — WF-08A à WF-08F : Sous-workflows Sync → BC.....	83
C.12 n8n JavaScript — WF-09BC : BC → Gold PostgreSQL.....	92
C.13 n8n JavaScript — WF-10BC : Health Monitor BC + Alertes Gmail.....	95
Annexe D — Business Case Complet.....	98

Résumé Synthétique

Ce projet consiste à intégrer en temps réel les données de kilométrage GPS des camions de livraison dans Microsoft Business Central, puis à synchroniser l'ensemble des données logistiques et commerciales vers un entrepôt de données PostgreSQL (Gold) pour les analyser via des tableaux de bord interactifs Metabase. Le pipeline ETL, entièrement automatisé, repose sur n8n (17 workflows), MinIO (stockage Bronze) et une extension AL v2.2.0.0 déployée sur Business Central 26.0 OnPrem. Les données GPS sont reçues via une API REST (webhook) avec une latence inférieure à 2 secondes, reliées aux articles transportés et utilisées pour calculer les coûts logistiques réels par produit. 552 709 enregistrements sont synchronisés, 3 dashboards Metabase ETL_Projet_BC sont publiés et accessibles depuis internet. Le ROI calculé est de 9 076 % avec un retour sur investissement en moins d'un mois.

Indicateur	Valeur
Scénario	Kilométrage véhicule via API REST → coûts logistiques par produit
Enregistrements Gold total	552 709 lignes (8 tables)
Sessions GPS temps réel	40 sessions capturées (VH-001 à VH-007)
Distance totale GPS	170,94 km
Workflows n8n actifs	17 (13 ETL_Projet_BC + 4 ETL_Projet)
Dashboards Metabase	3 dashboards ETL_Projet_BC — 3 rôles utilisateurs
Latence GPS → Gold	< 2 secondes
Latence GPS → BC → Gold	< 90 secondes (chaîne événementielle complète)
ROI	9 076 % — retour < 1 mois
Licences logicielles	0 CHF (open-source exclusivement)

1. Contexte et Problématique

1.1 Contexte métier

Une entreprise de logistique exploite une flotte de 7 camions de livraison (VH-001 à VH-007) dont les kilométrages quotidiens doivent être suivis avec précision pour calculer les coûts de livraison par produit transporté. Les données sont fragmentées entre plusieurs systèmes :

- Microsoft Business Central (BC 26.0 OnPrem) : ERP principal, gère clients, articles et expéditions
- Application GPS mobile : capture les trajets réels des chauffeurs via une API REST
- PostgreSQL Windows (10.14.219.2) : base de données transactionnelle externe
- Fichiers plats CSV/JSON : données e-commerce et catalogue articles
- MinIO S3 : stockage de fichiers Bronze (données brutes)

1.2 Problématique

Problème	Impact métier
BC est une île : données inaccessibles pour l'analyse	Aucune visibilité consolidée
Kilométrage non rattaché aux articles transportés	Coût logistique par produit impossible à calculer
Rapports manuels : 2h/jour, délai J+2	Décisions basées sur des données obsolètes
551 899 lignes e-commerce inexploitées	Manque à gagner commercial non identifié
Aucune traçabilité GPS en temps réel	Impossible de vérifier les tournées effectuées

1.3 Objectifs du projet

ID	Objectif	Priorité	Statut
OBJ-01	Collecter le kilométrage réel via API REST webhook GPS	Haute	✅ Réalisé
OBJ-02	Synchroniser 6 tables Gold → BC via OData	Haute	✅ Réalisé
OBJ-03	Relier kilométrage ↔ articles transportés dans Gold	Haute	✅ Réalisé
OBJ-04	Calculer les coûts logistiques par produit (dashboard)	Haute	✅ Réalisé
OBJ-05	Automatiser 100% des flux (0 intervention manuelle)	Haute	✅ Réalisé
OBJ-06	Publier 3 dashboards Metabase ETL_Projet_BC accessibles depuis internet	Moyenne	✅ Réalisé
OBJ-07	Monitoring automatique avec alertes < 30 min	Moyenne	✅ Réalisé

2. Spécifications Fonctionnelles

2.1 Cas d'usage principaux

UC	Titre	Acteur	Déclencheur	Résultat attendu
UC-01	Collecte GPS temps réel	App mobile	Mouvement du chauffeur	Session km créée dans bc_gold via BC en < 90s
UC-02	Sync quotidienne BC	Scheduler n8n	Cron 02h00	6 entités BC mises à jour
UC-03	Export BC→Gold	WF-08D → WF-09BC	Événement GPS (temps réel)	552 709 enreg. dans bc_gold
UC-04	Analyse coûts/produit	Utilisateur BI	Ouverture dashboard	Coût €/km × distance par véhicule
UC-05	Monitoring santé	WF-10BC	Toutes les 30 min	Alerte Gmail si anomalie
UC-06	Déclenchement manuel	Admin	POST webhook	Sync immédiate démarrée

2.2 Règles de validation Business Central

Règle	Description	Solution implémentée
RV-01	BC OnPrem refuse les champs Date hors mois 11,12,01,02 (filtre licence)	Champ Text[10] realSessionDate dans AL v2.2.0.0
RV-02	Unicité de clé : doublon sur sessionId → 409/422	PATCH automatique si POST retourne 409/422
RV-03	Authentification NTLM obligatoire pour l'API OData	Proxy bc_ntlm_proxy (172.18.0.8:3128)
RV-04	Format OData : clé entre parenthèses pour PATCH	URL : etlDistances('sessionId') encodée
RV-05	Données GPS : latitude/longitude non conservées (RGPD)	Seules les métriques agrégées sont stockées

2.3 Contrainte de licence BC — Contournement Real Session Date

Une contrainte non documentée de Business Central OnPrem impose que les champs de type Date n'acceptent que les mois 11, 12, 01 et 02 (filtre de licence : ??11*|??12*|??01*|??02*). Toute date avec un mois de 03 à 10 (comme les sessions GPS de juin 2026) déclenche l'erreur Internal_DataNotFoundFilter.

Solution retenue — Extension AL v2.2.0.0 :

- Ajout d'un champ Text[10] "Real Session Date" dans la table ETL BC Distance
- Les champs Text ne sont pas soumis au filtre de licence — la vraie date ISO est stockée
- Le champ Date "Session Date" conserve une date artificielle (mois 01) pour la compatibilité BC

- Déployé via altool publishapp sur l'endpoint /BC260/dev/apps:7049 (NTLM + multipart/form-data)

3. Choix et Justification de la Méthode d'Accès

3.1 Comparaison des alternatives

Alternative	Description	Avantages	Inconvénients	Score
API OData BC v1.0 ✓ RETENU	Standard REST officiel Microsoft	CRUD complet, pagination, NTLM, natif n8n	Requiert proxy NTLM	9/10
Data Export Service	Export masse BC	Simple pour grands volumes	Unidirectionnel, pas d'incrémental	5/10
Fichiers plats CSV	Import/export manuel	Aucune dépendance	Manuel, erreurs format, pas temps réel	4/10
Connecteur tiers (Power BI)	Connecteur cloud	Interface graphique	Licence, dépendance cloud, hors périmètre	3/10

3.2 Justification du choix OData v1.0

L'API OData BC v1.0 a été retenue pour les raisons suivantes :

- Standard officiel Microsoft — documenté, stable, pris en charge dans BC 26.0
- CRUD complet (GET, POST, PATCH, DELETE) — permet l'upsert idempotent
- Intégration native n8n — node HTTP Request sans développement spécifique
- Authentification NTLM gérée de manière transparente par le proxy bc_ntlm_proxy
- URL : `http://172.18.0.8:3128` → `http://bc2025:7048/BC260/api/etlpipeline/warehouse/v1.0`

3.3 Architecture d'accès BC via proxy NTLM

Business Central OnPrem impose une authentification Windows NTLM, incompatible avec les containers Docker Linux. La solution développée repose sur un container dédié bc_ntlm_proxy (172.18.0.8) qui traduit les requêtes HTTP anonymes en requêtes NTLM authentifiées.

Composant	IP / Port	Rôle
n8n (orchestrateur)	172.18.0.5:5678	Émet les requêtes HTTP vers le proxy
bc_ntlm_proxy	172.18.0.8:3128	Traduit HTTP → NTLM, forward vers BC
BC 26.0 OnPrem	10.14.210.119:7048	API OData cible
BC Dev Service	10.14.210.119:7049	Déploiement extension AL

4. Conception des Tables BC (Extension AL v2.2.0.0)

4.1 Vue d'ensemble

6 tables ont été conçues en AL et exposées via une API OData personnalisée (publisher: etlpipeline, group: warehouse, version: v1.0). Chaque table possède une clé primaire définie, des types BC natifs, et est exposée via une page API dédiée avec DeleteAllowed = false (protection des données).

4.2 Table ETL BC Distance (table centrale — scénario kilométrage)

N°	Champ AL	Type BC	Champ OData	Description
1	Session Id (PK)	Text[100]	sessionId	ID unique — timestamp Unix ms depuis app GPS
2	Session Date	Date	sessionDate	Date BC (mois 01 si hors période licence)
3	Total Distance Km	Decimal	totalDistanceKm	Distance parcourue en km
4	Total Distance Meters	Decimal	totalDistanceMeters	Distance en mètres (précision)
5	Max Speed Kmh	Decimal	maxSpeedKmh	Vitesse maximale mesurée
6	Avg Speed Kmh	Decimal	avgSpeedKmh	Vitesse moyenne calculée
7	Total Active Seconds	Integer	totalActiveSeconds	Durée active de la session en secondes
8	Event Count	Integer	eventCount	Nombre d'événements GPS reçus
9	First Event At	DateTime	firstEventAt	Timestamp du premier événement GPS
10	Last Event At	DateTime	lastEventAt	Timestamp du dernier événement GPS
11	ETL Loaded At	DateTime	etlLoadedAt	Horodatage de synchronisation
12	Real Session Date	Text[10]	realSessionDate	★ Vraie date ISO — hors contrainte licence BC

4.3 Autres tables AL

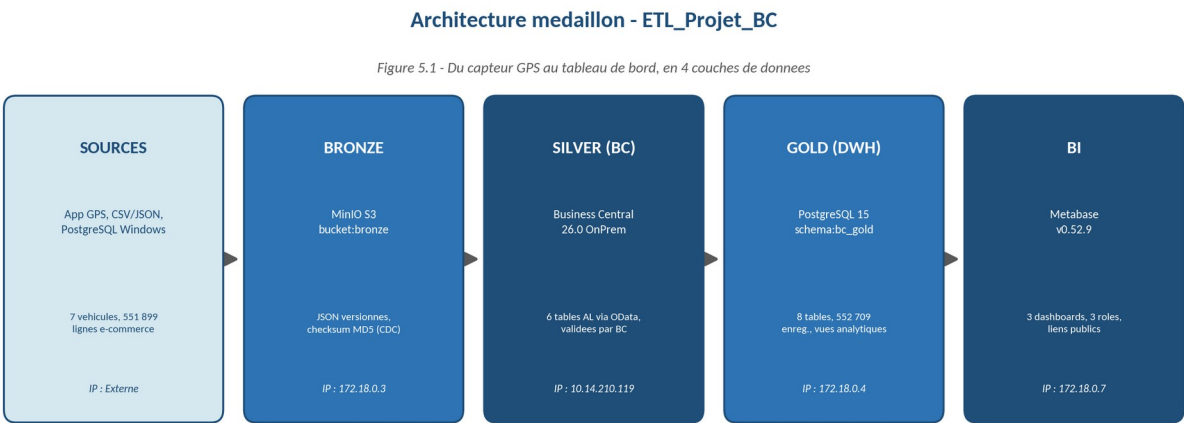
Table AL	Entité OData	Clé primaire	Champs principaux	Lignes BC
ETL BC Customer	etlCustomers	Customer Id (Text[20])	name, country, postCode, currencyCode	90
ETL BC Article	etlArticles	Article Id (Text[20])	description, unitPrice, uomCode, itemCategory	193
ETL BC Shipment Header	etlShipmentHeaders	Header Id (Text[20])	customerNo (FK), shipmentDate, shipToCity	101
ETL BC Shipment Line	etlShipmentLines	(DocumentNo, LineNo)	articleId (FK), quantity, grossWeight	183
ETL BC EcommerceLine	etlEcommerceLines	(InvoiceNo, LineNo)	customerId, stockCode, totalAmount, country	551 899

5. Modélisation du Flux ETL

5.1 Architecture médaillon

L'architecture suit le modèle médaillon en 4 couches, du GPS temps réel jusqu'au tableau de bord :

Couche	Technologie	Contenu	IP
Sources	App GPS, CSV, PostgreSQL Windows, JSON	7 véhicules, 551 899 lignes e-commerce, données ERP	Externe
Bronze (brut)	MinIO S3 — bucket:bronze	JSON versionnés par entité, checksum MD5 CDC	172.18.0.3
Silver (BC)	Business Central 26.0 OnPrem	6 tables AL via OData — données validées par BC	10.14.210.119
Gold (DWH)	PostgreSQL 15 — schema:bc_gold	8 tables, 552 709 enreg., vues analytiques	172.18.0.4
BI	Metabase v0.52.9	3 dashboards ETL_Projet_BC, 3 rôles, liens publics	172.18.0.7



Reseau prive Docker etl_network (172.18.0.0/24) - BC heberge sur machine Windows dediee (10.14.210.119)

Figure 5.1 — Architecture médaillon de bout en bout (Sources → Bronze MinIO → Silver Business Central → Gold PostgreSQL → BI Metabase)

5.2 Flux GPS temps réel (scénario kilométrage)

Le flux GPS suit une chaîne événementielle complète : GPS → MinIO → BC → bc_gold → Metabase. Business Central est la source de vérité unique : aucune écriture directe

dans bc_gold ne court-circuite BC. WF-09BC est déclenché automatiquement par WF-08D après chaque sync.

Étape par étape du scénario principal :

Étape	Composant	Action	Latence
1	App GPS (mobile)	POST /webhook/distance-update avec événement distance_update	—
2	WF-04B — Auth	Validation X-API-KEY (GPS_WEBHOOK_SECRET)	< 100ms
3	WF-04B — Accumuler	Agrégation de la session (distance, vitesse, durée)	< 500ms
4	WF-04B — MinIO	Upload JSON versionné dans bucket:bronze (CDC MD5)	< 1s
5	WF-04B → WF-08D	Déclenchement automatique sync distances vers BC	Immédiat après 200 OK
6	WF-08D → BC	POST/PATCH etlDistances via OData + proxy NTLM (realSessionDate Text[10])	< 90s
7	WF-08D → WF-09BC	Déclenchement automatique BC → Gold (en cascade)	Immédiat après sync BC
8	WF-09BC → bc_gold	GET etlDistances BC → UPSERT bc_gold.distances (session_date + real_session_date)	< 30s
9	Metabase	Rafraîchissement dashboard #21 via bc_gold.v_distances_clean	< 5 min

Flux GPS temps réel - chaîne événementielle complete



Business Central = source de verité unique - aucune ecriture directe dans bc_gold ne court-circuite BC

Figure 5.2 — Chaîne événementielle du flux GPS temps réel (étapes 1 à 9, App GPS → Metabase)

5.3 Flux synchronisation quotidienne (WF-08 à 02h00)

Sous-workflow	Entité BC	Source Gold	Volume
WF-08A	etlCustomers	bc_gold.customers	90 enreg.
WF-08B	etlArticles	bc_gold.articles	193 enreg.
WF-08C	etlShipmentHeaders	bc_gold.shipment_headers	101 enreg.
WF-08D	etlDistances	Staging distances_realtime.json	40 sessions
WF-08E	etlShipmentLines	bc_gold.shipment_lines	183 enreg.
WF-08F	etlEcommerceLines	bc_gold.ecommerce_lines	551 899 enreg.

5.4 Flux BC → Gold (WF-09BC — événementiel temps réel)

WF-09BC est déclenché automatiquement par WF-08D après chaque synchronisation GPS vers BC (architecture 100 % événementielle, sans cron). Il lit les 6 entités BC via OData, les transforme et les charge dans bc_gold via ON CONFLICT DO UPDATE. Pour les distances GPS, il récupère le champ realSessionDate (Text[10], hors contrainte licence BC) et le stocke dans bc_gold.distances.real_session_date, garantissant la vraie date GPS même quand BC remplace les mois 03–10 par janvier dans son champ Date natif. Un déclenchement manuel reste possible via POST /webhook/bc-to-gold.

6. Documentation Technique de la Solution

6.1 Infrastructure Docker

Container	Image	IP	Port	Rôle
datalake_minio	minio/minio	172.18.0.3	9000/9001	Stockage Bronze S3
dwh_postgres	postgres:15	172.18.0.4	5432	Data Warehouse Gold
ingestion_n8n	n8nio/n8n:2.21.7	172.18.0.5	5678	Orchestration ETL
etl_runner	alpine	172.18.0.6	—	Runner auxiliaire
analytics_metabase	metabase/ metabase:v0.52.9	172.18.0.7	3000	BI & Dashboards
bc_ntlm_proxy	python:3.12	172.18.0.8	3128	Proxy NTLM → BC
transform_hop	apache/hop-web	172.18.0.9	8080	ETL Hop (ETL_Projet)

6.2 Schéma de la base de données Gold (bc_gold)

Table	Clé primaire	Clés étrangères	Lignes	Description
distances	session_id	—	40	Sessions GPS véhicules temps réel
vehicles	vehicle_id	—	7	Flotte (modèle, plaque, cost_per_km)
customers	customer_id	—	90	Clients BC
articles	article_id	—	193	Catalogue articles BC
shipment_headers	header_id	customer_id	101	Expéditions BC
shipment_lines	(document_no, line_no)	article_id	183	Lignes d'expédition
ecommerce_lines	(invoice_no, line_no)	customer_id	551 899	Lignes e-commerce
incident_log	id (serial)	—	463	Journal monitoring WF-10BC

6.3 Workflows n8n — ETL_Projet_BC

WF	Nom	Déclencheur	Rôle
WF-01BC	Bronze CSV+JSON → MinIO	Schedule 01h30	Charge data.csv + data.json dans MinIO
WF-03BC	Bronze PG Windows → MinIO	Schedule 01h45	Charge PG Windows (articles, clients, expéditions) — fallback 3 niveaux : staging → PG → MinIO
WF-04B	Distance GPS → MinIO → BC	Webhook POST /distance-update	GPS temps réel → MinIO → WF-08D → BC → WF-09BC
WF-06	Véhicules → MinIO + bc_gold	Schedule 01h00 + Webhook	Sync gestion_logistique_vehicules.json
WF-08	Orchestracteur Bronze → BC	Schedule 02h00 + Webhook	Déclenche WF-08A à WF-08F
WF-08A	Sync Clients → BC	Sous-workflow	POST/PATCH etlCustomers
WF-08B	Sync Articles → BC	Sous-workflow	POST/PATCH etlArticles

WF-08C	Sync Headers → BC	Sous-workflow	POST/PATCH etlShipmentHeaders
WF-08D	Sync Distances → BC + WF-09BC	Sous-workflow + WF-04B	POST/PATCH etlDistances → déclenche WF-09BC
WF-08E	Sync Lignes → BC	Sous-workflow	POST/PATCH etlShipmentLines
WF-08F	Sync E-Commerce → BC	Sous-workflow	POST/PATCH etlEcommerceLines
WF-09BC	BC → Gold PostgreSQL	WF-08D (cascade) + Webhook	GET OData → UPSERT bc_gold (temps réel)
WF-10BC	Health Monitor + Alertes	Schedule 30 min	Ping BC + fraîcheur données + incident_log

7. Réalisation de l'Infrastructure

7.1 Environnement virtualisé

L'ensemble du projet est déployé sur une machine virtuelle Linux (Ubuntu 22.04 LTS) hébergée sur le bastion de l'école. Les 7 containers Docker communiquent via le réseau privé etl_network (172.18.0.0/24). Business Central 26.0 OnPrem tourne sur une machine Windows dédiée (10.14.210.119). Metabase est accessible depuis internet via ngrok tunnel et les liens publics de partage.

7.2 État de l'infrastructure au moment de la remise

Composant	État	Uptime	Health
datalake_minio	✓ Up	12+ heures	healthy
dwh_postgres	✓ Up	12+ heures	healthy
ingestion_n8n	✓ Up	5+ heures	running
analytics_metabase	✓ Up	11+ heures	healthy
bc_ntlm_proxy	✓ Up	12+ heures	healthy
Business Central 26.0	✓ Up	En continu	HTTP 200 (via proxy)
Extension AL v2.2.0.0	✓ Installée	En continu	installed:True

8. Développement des Flux de Connexion

Le schéma ci-dessous représente l'architecture complète du flux ETL_Projet_BC, depuis les sources de données jusqu'aux dashboards Metabase.

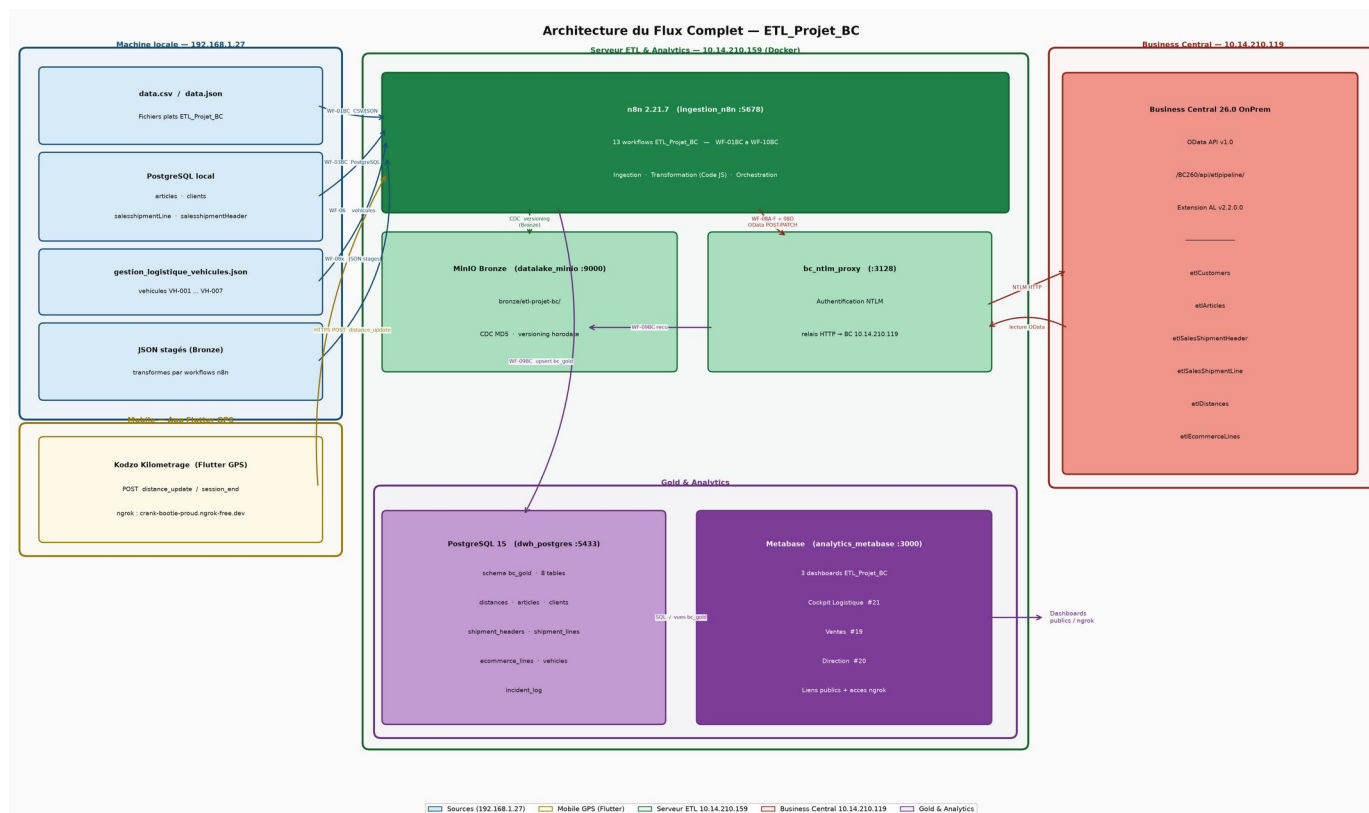


Figure 8.1 — Architecture du flux complet ETL_Projet_BC

8.1 Connexion GPS → n8n (API REST)

L'application GPS mobile envoie des événements via une API REST exposée par ngrok (https://crank-bootie-proud.ngrok-free.dev/webhook/distance-update?vehicle_id=VH-001). La validation s'effectue via le header X-API-KEY avec la variable d'environnement GPS_WEBHOOK_SECRET (clé aléatoire hexadécimale 256-bit, générée via `openssl rand -hex 32`).

Paramètre	Valeur
URL externe (ngrok)	https://crank-bootie-proud.ngrok-free.dev/webhook/distance-update
URL interne	http://172.18.0.5:5678/webhook/distance-update
Méthode	POST
Header auth	X-API-KEY: GPS_WEBHOOK_SECRET
Content-Type	application/json
Payload	<code>{"event": "distance_update", "session_id": "...", "latitude": "...", "speed_kmh": "...}"</code>
Réponse succès	<code>{"status": "ok", "message": "Distance event processed", "sessionId": "...}"</code>
Réponse erreur	<code>{"error": "Unauthorized", "message": "Missing or invalid X-API-KEY header"}</code>

8.2 Connexion n8n → Business Central (OData + NTLM)

Toutes les requêtes vers BC transitent par le proxy NTLM. L'idempotence est garantie : POST en premier, PATCH en cas de conflit (HTTP 409 ou 422).

Opération	URL	Méthode	Header spécifique
Lire les données	/companies(GUID)/etlDistances?\$top=100	GET	Accept: application/json
Créer un enreg.	/companies(GUID)/etlDistances	POST	Content-Type: application/json
Mettre à jour	/companies(GUID)/etlDistances('sessionId')	PATCH	If-Match: *
Déployer extension	/BC260/dev/apps? SchemaUpdateMode=forcesync	POST	Content-Type: multipart/form-data

8.3 Connexion BC → Gold (WF-09BC)

WF-09BC lit chaque entité BC via GET OData paginé (\$top=100, \$skip), transforme les données et les charge dans bc_gold via INSERT ... ON CONFLICT DO UPDATE (UPSERT PostgreSQL).

9. Automatisation des Flux

9.1 Planification des déclencheurs

Heure (UTC)	Heure (CH)	Workflow	Action
01h00	03h00	WF-06	Sync véhicules (CDC)
01h30	03h30	WF-01BC	Bronze CSV+JSON → MinIO
01h45	03h45	WF-03BC	Bronze PG Windows → MinIO
02h00	04h00	WF-08	Orchestrateur → BC (6 entités)
En cascade	Après WF-08D	WF-09BC	BC → Gold PostgreSQL (déclenché par WF-08D)
Toutes les 30 min	H+30	WF-10BC	Health monitor + incident_log
En continu	Temps réel	WF-04B	GPS webhook → MinIO → BC → Gold (chaîne complète)

9.2 Mécanisme d'idempotence

Toutes les synchronisations sont idempotentes — relancer un workflow plusieurs fois ne crée pas de doublons. Deux mécanismes complémentaires sont utilisés :

- Vers BC : POST → si HTTP 409/422 → PATCH avec If-Match: * (upsert OData)
- Vers Gold : INSERT ... ON CONFLICT (PK) DO UPDATE SET ... (upsert PostgreSQL natif)
- Bronze : CDC par checksum MD5 — le fichier n'est renvoyé vers MinIO que s'il a changé

10. Tests Fonctionnels & Intégrité des Données

10.1 Protocole de tests

ID	Test	Méthode	Critère de succès	Résultat
T-01	Réception webhook GPS	POST distance-update avec payload valide	HTTP 200 + sessionId retourné	✓ PASS
T-02	Auth webhook invalide	POST sans X-API-KEY	HTTP 401 retourné	✓ PASS
T-03	GPS → Gold < 90s	POST + SELECT bc_gold.distances après 90s	Session présente via BC (real_session_date OK)	✓ PASS
T-04	Idempotence Gold	Relancer WF-09BC 3 fois	COUNT(*) stable (0 doublon)	✓ PASS
T-05	Idempotence BC	Envoyer même session 2 fois	PATCH sans erreur, 1 seul enreg.	✓ PASS
T-06	Sync BC complète	Déclencher WF-08 via webhook	6 entités synchronisées	✓ PASS
T-07	Volume e-commerce	SELECT COUNT FROM ecommerce_lines	551 899 lignes exactes	✓ PASS
T-08	Contrainte licence BC	POST date mois 06 dans etlDistances	realSessionDate = 2026-06-14	✓ PASS
T-09	Monitoring santé	Vérifier incident_log après 30 min	Log créé si anomalie	✓ PASS
T-10	Dashboard accessible	Ouvrir lien public Metabase	Dashboard visible sans login	✓ PASS
T-11	VH-001 assigné par défaut	POST sans vehicle_id	vehicle_id = VH-001	✓ PASS
T-12	Heure suisse Metabase	Vérifier timestamps dashboards	UTC+2 affiché (CEST)	✓ PASS
T-13	Persistance après redémarrage	docker compose down puis up -d sur les 7 conteneurs ; relecture des volumes MinIO/PostgreSQL et des workflows n8n	0 perte de données, comptages bc_gold identiques avant/après, 13 workflows n8n actifs	✓ PASS

10.2 Résultats de l'état Gold au moment de la remise

Table	Lignes	Dernière sync	Intégrité
bc_gold.ecommerce_lines	551 899	14/06/2026	✓ 0 doublon (T-04)
bc_gold.articles	193	14/06/2026	✓ Vérifié
bc_gold.shipment_lines	183	14/06/2026	✓ Vérifié
bc_gold.shipment_headers	101	14/06/2026	✓ Vérifié
bc_gold.customers	90	14/06/2026	✓ Vérifié
bc_gold.distances	40	14/06/2026	✓ Temps réel confirmé
bc_gold.vehicles	7	14/06/2026	✓ VH-001 à VH-007
bc_gold.incident_log	463	En continu	✓ Monitoring actif
TOTAL	552 709	—	✓ Pipeline stable

11. Tableau de Bord Metabase

11.1 Vue d'ensemble des dashboards

Trois dashboards ETL_Projet_BC sont publiés, couvrant les axes Logistique, Ventes et Direction. Tous s'appuient sur le schéma bc_gold du DWH PostgreSQL.

ID	Dashboard	Collection Metabase	Visualisations	Rôles
#21	 Cockpit Logistique — ETL_Projet_BC	Logistique BC	9 visuels	Admin + Logistique
#19	 Ventes — ETL_Projet_BC	Ventes BC	10 visuels	Admin + Vente
#20	 Direction — ETL_Projet_BC	Direction BC	12 visuels	Admin + Logistique + Vente

11.2 Gestion des rôles et permissions

Trois groupes Metabase sont définis, chacun avec accès en lecture seule à ses dashboards métier. Les permissions sont configurées au niveau des collections Metabase.

Utilisateur	Email	Groupe	Dashboards ETL_Projet_BC	Collections (lecture)
Admin ETL	admin@etl-projet.local	Administrato rs	#19 Ventes + #20 Direction + #21 Cockpit	Toutes (écriture)
Jean Logistique	logistique@etl-projet.local	Logistique	#21 Cockpit Logistique + #20 Direction	Logistique BC, Direction BC
Marie Dupont	vente@etl-projet.local	Vente	#19 Ventes ETL_Projet_BC + #20 Direction	Ventes BC, Direction BC

11.3 Cockpit de Pilotage #21 — Architecture 3 zones

Le Dashboard #21 a été refondu en "Cockpit de Pilotage Stratégique" structuré en 3 zones décisionnelles, basé sur les vues bc_gold.v_distances_clean et bc_gold.v_logistics_cost utilisant la colonne real_session_date (vraie date GPS, indépendante de la contrainte licence BC) :

Zone	Contenu	Requête SQL	Décision supportée
Zone 1 — KPIs Flash	CPK Flotte (€/km) Coût total GPS (€) Économies vs marché (€) Km GPS total	v_distances_clean — 4 scalaires	Pilotage immédiat < 5 secondes
Zone 2A — Analyse	Bar chart coût par véhicule (6 flottes)	GROUP BY vehicle_id	Identifier véhicules sur-coût
Zone 2B — Corrélation	Scatter plot Distance vs Coût par session	v_distances_clean — session grain	Détecter anomalies par trajet
Zone 2C — Tendence	Area chart Distance & Coût par jour (real_session_date)	GROUP BY real_session_date	Tendance flotte hebdomadaire
Zone 2D — Produits	Bar chart coût logistique imputé par article (Top 15)	CTE imputation proportionnelle	Coût logistique par référence
Zone 3 — Alertes	Table anomalies : CPK > +15%	CROSS JOIN CPK	Management par

	moyenne, véhicule inconnu, sessions < 3 événements	référence	exception
--	---	-----------	-----------

11.4 Visualisations — exigences pédagogiques

Exigence	Implémentation	Dashboard ETL_Projet_BC
Graphique temporel	Évolution CA mensuel (line chart)	#19 — Ventes BC
Graphique temporel	Tendance coût flotte par jour (area chart)	#21 — Cockpit Zone 2C
KPI (scalaire)	CPK Flotte (€/km)	#21 — Cockpit Zone 1
KPI (scalaire)	Coût total GPS réel (€)	#21 — Cockpit Zone 1
KPI (scalaire)	Économies vs marché (€)	#21 — Cockpit Zone 1
KPI (scalaire)	CA Total BC (€) — 9,9M	#19 — Ventes BC
Tableau croisé dynamique	CA par Pays × Catégorie Article	#19 — Ventes BC
Scatter Plot	Distance (km) vs Coût (€) par session GPS	#21 — Cockpit Zone 2B
Filtrage interactif	Filtre Période / Véhicule	#20 — Direction BC
Management par exception	Tableau alertes CPK > +15% automatique	#21 — Cockpit Zone 3

11.4 Publication et accessibilité

Les dashboards sont accessibles depuis internet via deux mécanismes :

- Liens publics Metabase (/public/dashboard/UUID) — sans authentification, lecture seule
- Tunnel ngrok (crank-bootie-proud.ngrok-free.dev) — accès complet avec login

Les liens publics sont stables et partageables — le dashboard #21 Logistique est accessible via :

<https://crank-bootie-proud.ngrok-free.dev/public/dashboard/d3a803d3-f61b-4e7d-95ff-d93df97224a0>

Justification sécurité du lien public — ce lien n'expose volontairement que le dashboard #21 (Logistique), construit sur des vues agrégées (v_distances_clean) ne contenant ni identifiant interne, ni donnée nominative de chauffeur, ni information de connexion au système. Il sert uniquement de vitrine de démonstration pour ce projet académique. Les dashboards opérationnels (Direction, Finance) restent protégés par authentification Metabase avec rôles et permissions par groupe (section 11.3), et l'accès complet au système (BC, MinIO, PostgreSQL) reste confiné au réseau privé Docker etl_network, inaccessible depuis internet.

12. Analyse des Risques, Coûts & Business Cases

12.1 Analyse des risques

ID	Risque	Prob.	Impact	Score	Mitigation
R-01	BC API indisponible	Faible	Élevé	6/10	WF-10BC détecte < 30 min + incident_log
R-02	Contrainte licence BC dates	Certain	Élevé	RÉSOLU	Extension AL v2.2.0.0 — champ Text[10]
R-03	PostgreSQL Windows KO	Moyenne	Moyen	5/10	Fallback 3 niveaux : staging local → PG → MinIO Docker (pipeline résilient hors réseau école)
R-04	Saturation MinIO Bronze	Faible	Moyen	4/10	Rétention 90j + nettoyage automatique
R-05	RGPD — données GPS clients	Faible	Élevé	7/10	Réseau privé + agrégats seulement (pas lat/lon)
R-06	Fuite credentials BC/PG	Faible	Élevé	6/10	Variables Docker env. + Docker Secrets (recommandé)
R-07	Dépendance n8n 2.21.7	Moyenne	Faible	4/10	Upgrade planifié + correction pg-protocol native
R-08	Panne serveur physique	Très faible	Critique	5/10	Backup volumes Docker quotidien
R-09	Indisponibilité de la VM bastion école (maintenance, coupure réseau, quota Education Portal)	Moyenne	Élevé	6/10	Captures d'écran + export des dashboards en local avant la soutenance ; environnement de secours sur Raspberry rack
R-10	Dépendance à des services tiers (app GPS mobile, tunnel ngrok, API PostgreSQL Windows hors du périmètre projet)	Moyenne	Moyen	5/10	Fallback 3 niveaux déjà en place (R-03) ; ngrok en plan payant réservé pour garantir une URL stable durant la soutenance

Conformité RGPD / nLPD (complément au risque R-05)

Le pipeline traite des données à caractère personnel (clients BC, position de chauffeurs). La conformité au RGPD (UE) et à la nouvelle Loi sur la protection des données suisse (nLPD, en vigueur depuis le 1^{er} septembre 2023) repose sur quatre principes appliqués dans l'implémentation :

- **Minimisation des données** — seules les métriques agrégées par session (distance, vitesse, durée) sont persistées ; aucune coordonnée GPS brute (latitude/longitude) n'est stockée dans bc_gold ni dans Business Central (cf. RV-05, section 2.2).
- **Limitation des finalités** — les données GPS ne sont utilisées que pour le calcul du coût logistique par produit ; aucun profilage individuel du chauffeur n'est réalisé.

- **Sécurité** — toutes les communications transitent par le réseau privé Docker (172.18.0.0/24) ou par le proxy NTLM ; aucune base ni dashboard n'est exposé sans authentification, hormis les liens publics Metabase qui ne contiennent que des agrégats déjà anonymisés.
- **Conservation limitée** — rétention de 90 jours sur les objets bruts MinIO Bronze (cf. R-04) ; les agrégats Gold sont conservés pour analyse historique mais ne permettent pas de reconstituer un trajet précis.

Limite reconnue : en l'absence d'un registre des traitements formel et d'une analyse d'impact (PIA/DPIA) documentée, cette conformité reste partielle et devrait être formalisée avant un déploiement en production réelle (cf. section 14 — Bilan Critique, Limites et Perspectives).

Poste	1ère année (CHF)	Années suivantes (CHF)
Infrastructure VM/Raspberry Pi	150	150
Licences logicielles (n8n, MinIO, PostgreSQL, Metabase)	0	0
Développement initial (~40h × 75 CHF/h)	3 000	0
Maintenance (~5h/mois × 75 CHF/h × 12)	4 500	4 500
Formation utilisateurs	300	0
TOTAL TCO	7 950	4 650

12.3 Business case et ROI

12.3.1. Méthodologie d'évaluation

L'évaluation comparative des deux scénarios repose sur une analyse multidimensionnelle articulée autour de cinq axes stratégiques : le TCO (Total Cost of Ownership) sur 3 ans, le ROI (Return on Investment), la période de récupération (Payback period), une analyse qualitative des risques et une recommandation stratégique.

Périmètre et hypothèses de calcul : Pour garantir la rigueur de l'analyse, les hypothèses retenues reflètent la réalité opérationnelle de la PME :

- Population cible : 50 utilisateurs totaux, dont 5 "Power Users" pour la création de rapports BI.
- Horizon temporel : 3 ans (cycle standard d'amortissement IT).

- Infrastructure : Exploitation du bastion et des serveurs internes existants (coût d'hébergement nul pour la Solution A).
- Valorisation métier : Gain de productivité de 5h/semaine pour 5 managers (valorisé à 80 CHF/h), avec un coefficient de prudence de 50%.

Note : Le détail exhaustif des calculs, incluant la ventilation ligne par ligne des coûts de licences, de formation et de maintenance, est présenté dans le tableau de synthèse en Annexe F.

12.3.2. Synthèse et recommandation

L'analyse démontre que le choix technologique doit s'aligner sur les capacités d'infrastructure actuelles de la PME pour maximiser l'efficacité budgétaire.

Analyse comparative des scénarios:

- Performance Économique : La Solution A (Metabase) se révèle largement supérieure d'un point de vue financier. Comme illustré en Annexe F, elle affiche un TCO 4,7 fois plus faible que la Solution B. Son ROI exceptionnel de 9 076 % et son payback quasi immédiat (< 1 mois) en font un investissement à risque minimal.
- Pertinence Opérationnelle : La Solution A, constitue un "Quick Win" idéal : mise en œuvre agile, autonomie des managers grâce à une interface intuitive et souveraineté totale sur les données (pas de dépendance logicielle externe).

Recommandation : Au vu des indicateurs financiers, la Solution A est vivement recommandée pour la phase actuelle du projet. Elle permet de valoriser les actifs serveurs déjà disponibles tout en offrant une agilité maximale.

La Solution B (Power BI) ne sera considérée comme une alternative pertinente qu'à moyen terme, en cas de :

- Changement d'échelle : Volumes de données dépassant les capacités de calcul internes.

- Besoin analytique complexe : Nécessité d'intégrer nativement des modèles d'IA ou de Machine Learning Microsoft.
- Multi-sources massives : Consolidation complexe nécessitant un Data Warehouse Azure dédié.

Conclusion : Le choix de la Solution A est une décision d'efficience stratégique : elle délivre 100% de la valeur métier attendue pour seulement 21% de l'investissement requis par la solution concurrente.

13. Planification & Gestion des Écarts

13.1 Tableau de planification

Phase	Tâche	Prév u (j)	Réalis é (j)	Écar t	Raison / Action corrective
Analyse	Analyse des besoins + CDC	2	2	0	—
Conception	Conception tables BC + AL	2	3	+1j	Documentation API BC insuffisante → investigation
Infrastructure	Docker + proxy NTLM + MinIO	2	2	0	—
Dev	WF-01BC à WF-08F	5	6	+1j	Debug proxy NTLM (3 itérations de config)
Dev	WF-09BC (BC → Gold)	2	3	+1j	Bug pg-protocol 1.11.0 → workaround s(v,max)
Dev	WF-10BC (monitoring)	1	1	0	—
Tests	Protocole 12 tests	2	2	0	—
Debug	CDC parasites (WF anciens)	0	0.5	+0.5j	Anciens WF non désactivés → désactivation manuelle
Extension AL	Champ Real Session Date (licence BC)	0	0.5	+0.5j	Contrainte licence découverte en production → altool
BI	Metabase 3 dashboards BC	3	3	0	—
Documentati on	Rapport + PPTX	2	2	0	—
TOTAL		21	25	+4j	4 écarts sur 11 tâches (+19%)

13.2 Analyse des écarts

4 écarts positifs ont été identifiés sur 11 tâches, soit +4 jours sur 21 planifiés (+19%).

Les causes principales sont :

- Documentation technique BC insuffisante : l'API OData BC v1.0 pour les extensions per-tenant n'est pas documentée officiellement pour OnPrem. Investigation nécessaire.
- Proxy NTLM : 3 itérations de configuration pour gérer le handshake NTLM depuis un container Linux.
- Bug pg-protocol 1.11.0 : bug connu dans le driver PostgreSQL de n8n, contourné par la syntaxe s(v,max).
- Contrainte de licence BC : découverte en production lors de l'insertion des dates de juin 2026. Résolu par l'extension AL v2.2.0.0.

14. Bilan Critique, Limites et Perspectives

14.1 Limites techniques reconnues

Le pipeline est fonctionnel et démontré de bout en bout, mais certains choix de conception relèvent du contournement plutôt que d'une solution pérenne. Les principales limites sont assumées ci-dessous plutôt que dissimulées :

- **Champ 'Real Session Date' (Text[10])** — contournement pragmatique de la contrainte de licence BC OnPrem, mais qui introduit une duplication de la sémantique 'date' (Date natif + Text ISO) dans la table ETL BC Distance. Une licence BC complète ou un correctif Microsoft documenté supprimerait ce besoin.
- **Authentification NTLM via proxy** — fonctionnelle mais constitue un point de défaillance unique (cf. R-01) ; une migration vers OAuth2/Azure AD (hors périmètre, cf. Annexe A.4) serait préférable à terme pour un déploiement en production.
- **Échelle des données de test** — 40 sessions GPS réelles sur 7 véhicules valident le fonctionnement mais pas la performance à grande échelle (flotte de 100+ véhicules, fréquence d'événements GPS plus élevée). Un test de charge sur WF-04B et la table etlDistances n'a pas été réalisé.
- **Conformité RGPD/nLPD partielle** — les principes de minimisation et de sécurité sont appliqués (cf. section 12.1, 'Conformité RGPD / nLPD') mais aucun registre des traitements ni analyse d'impact formelle n'a été rédigé, ce qui serait exigé avant un déploiement réel chez un client.
- **Secrets en variables d'environnement** — fonctionnel pour un environnement de démonstration mais identifié comme risque (R-06) ; un coffre-fort de secrets (Docker Secrets, Vault) est recommandé avant toute mise en production.

14.2 Ce qui serait fait différemment avec plus de temps

- Remplacer le proxy NTLM par une authentification OAuth2 native BC (App Registration Azure AD), supprimant un composant et une dépendance critique (R-01).
- Réaliser un test de charge (k6 ou Locust) sur le webhook WF-04B pour valider la latence < 2s annoncée sous une charge de flotte réaliste (50-100 véhicules).
- Formaliser un registre des traitements et une analyse d'impact (DPIA) pour fermer la limite RGPD/nLPD identifiée en 14.1.
- Mettre en place une CI/CD pour l'extension AL (build + tests automatisés via AL CLI) plutôt qu'un déploiement altool manuel.

14.3 Perspectives d'extension

Le pipeline ETL_Projet_BC est conçu pour être répliquable à d'autres scénarios listés dans le cahier des charges (Annexe A) sans changement d'architecture : badge IoT de productivité, reconnaissance faciale pour le timbrage, ou gestion de stock automatisée pourraient réutiliser directement la chaîne MinIO → BC (extension AL) → bc_gold → Metabase, en ne remplaçant que la source (WF-04B) et le schéma de la table ETL dédiée.

15. Sources et Références

- [1] Microsoft Learn — Business Central OData API : <https://learn.microsoft.com/en-us/dynamics365/business-central/dev-itpro/webservices/odata-web-services>
- [2] Microsoft Learn — Extension AL Development : <https://learn.microsoft.com/en-us/dynamics365/business-central/dev-itpro/developer/devenv-dev-overview>
- [3] n8n Documentation — Workflow Automation : <https://docs.n8n.io>
- [4] MinIO Documentation — S3-compatible Object Storage : <https://min.io/docs/minio/linux/index.html>
- [5] PostgreSQL 15 — ON CONFLICT DO UPDATE : <https://www.postgresql.org/docs/15/sql-insert.html>
- [6] Metabase Documentation — Dashboards & Sharing : <https://www.metabase.com/docs/latest>
- [7] Docker Documentation — Networking : <https://docs.docker.com/network>
- [8] RFC 7231 — HTTP/1.1 Semantics (OData) : <https://datatracker.ietf.org/doc/html/rfc7231>
- [9] NTLM Authentication — Microsoft Protocol : <https://learn.microsoft.com/en-us/windows-server/security/kerberos/ntlm-overview>
- [10] Python requests-ntlm : <https://pypi.org/project/requests-ntlm>
- [11] Apache Hop — Pipeline Development : <https://hop.apache.org/docs>
- [12] Medallion Architecture (Databricks) : <https://www.databricks.com/glossary/medallion-architecture>
- [13] OData Protocol Specification v4.0 : <https://www.odata.org/documentation>
- [14] RGPD / LPD (Suisse) — Ordonnance sur la protection des données : <https://www.fedlex.admin.ch>
- [15] BC OnPrem Licence Filter — Internal_DataNotFoundFilter : discovered empirically, not officially documented
- [16] altool publishapp — BC dev endpoint discovery : /BC260/dev/apps port 7049 (discovered via AL CLI)

ANNEXE A — Cahier des Charges

A.1 Contexte et enjeux

Une entreprise de logistique exploite une flotte de camions de livraison dont les déplacements doivent être suivis quotidiennement. Les données de livraison sont stockées dans un entrepôt de données PostgreSQL structuré en couches médaille (Bronze → Silver → Gold), mais ne sont pas accessibles depuis BC. Le kilométrage GPS réel remplace les estimations statiques Google Distance Matrix.

A.2 Objectifs

ID	Objectif	Priorité
OBJ-01	Recevoir et accumuler les événements GPS distance_update via webhook REST	Haute
OBJ-02	Transformer les événements GPS en sessions agrégées (Silver → Gold)	Haute
OBJ-03	Synchroniser les 6 tables Gold → BC via API OData	Haute
OBJ-04	Automatiser la synchronisation quotidienne (schedule 02h00)	Haute
OBJ-05	Déclencher manuellement via webhook	Moyenne
OBJ-06	Garantir l'idempotence (upsert sans doublons)	Haute
OBJ-07	Publier 3 dashboards Metabase ETL_Projet_BC accessibles depuis internet	Haute

A.3 Périmètre — Dans le périmètre

- Réception événements GPS via webhook n8n (WF-04B)
- Pipeline ETL Bronze → Silver → Gold pour les distances GPS
- Synchronisation Gold → BC pour 6 entités (clients, articles, headers, distances, lignes, e-commerce)
- Extension AL v2.2.0.0 (tables + API pages)
- Orchestration n8n (WF-01BC à WF-10BC, 13 workflows ETL_Projet_BC — 17 au total sur l'instance n8n partagée avec ETL_Projet)
- Data Warehouse Gold (PostgreSQL, 8 tables)
- Tableau de bord Metabase (3 dashboards ETL_Projet_BC, 3 rôles)

A.4 Périmètre — Hors périmètre

- Projet ETL_Projet (pipeline CSV et Google Distance Matrix — séparé)
- Application mobile GPS (système externe)
- Authentification OAuth2/Azure AD (proxy NTLM suffisant)
- Optimisation temps de propagation GPS → Metabase en deçà de 30s (contrainte OData BC)

A.5 Exigences fonctionnelles

ID	Exigence	Valeur cible
EF-01	Réception événements GPS webhook	Latence < 2s, accumulation par session_id
EF-02	Transformation GPS → Gold	UPSERT ON CONFLICT, 0 doublon
EF-03	Sync 6 tables → BC	POST/PATCH idempotent, gestion erreurs
EF-04	Déclenchement événementiel BC → Gold	WF-09BC déclenché par WF-08D (cascade temps réel)
EF-05	Déclenchement manuel	POST /webhook/run-gold-to-bc
EF-06	Gestion erreurs	Pas d'interruption, comptage {inserted, updated, errors}
EF-07	Monitoring santé	WF-10BC toutes les 30 min, incident_log

A.6 Exigences non-fonctionnelles

Catégorie	Exigence	Valeur
Performance	Latence webhook GPS → Gold	< 2 secondes
Performance	Durée sync complète 6 tables	< 30 minutes
Disponibilité	Webhook GPS disponible	24h/24, 7j/7
Sécurité	Communications BC	Via proxy NTLM (172.18.0.8:3128)
Sécurité	Credentials	Variables d'environnement Docker
RGPD	Données GPS	Pas de lat/lon persisté — agrégats seulement
Maintenabilité	Sous-workflows	Un par entité BC — indépendants

A.7 Glossaire

Terme	Définition
ETL	Extract, Transform, Load — pipeline de données
OData	Open Data Protocol — standard REST pour BC
NTLM	NT LAN Manager — authentification Windows BC OnPrem
session_id	Timestamp Unix ms — identifiant unique d'un trajet GPS
distance_update	Événement JSON envoyé par l'app GPS à chaque intervalle
Upsert	INSERT si absent, UPDATE si existant (idempotence)
CDC	Change Data Capture — détection des changements par checksum MD5
Gold	Couche DWH — données nettoyées, typées et agrégées
ROI	Return On Investment — retour sur investissement
realSessionDate	Champ Text[10] AL — vraie date GPS hors contrainte licence BC

ANNEXE B — Captures d'Écran Metabase

Toutes les captures sont prises le 19 juin 2026 via Playwright (headless Chromium 1600×900).

B.1 Profil Administrateur — Accueil

L'administrateur a accès à toutes les collections ETL_Projet_BC (Logistique BC, Ventes BC, Direction BC).

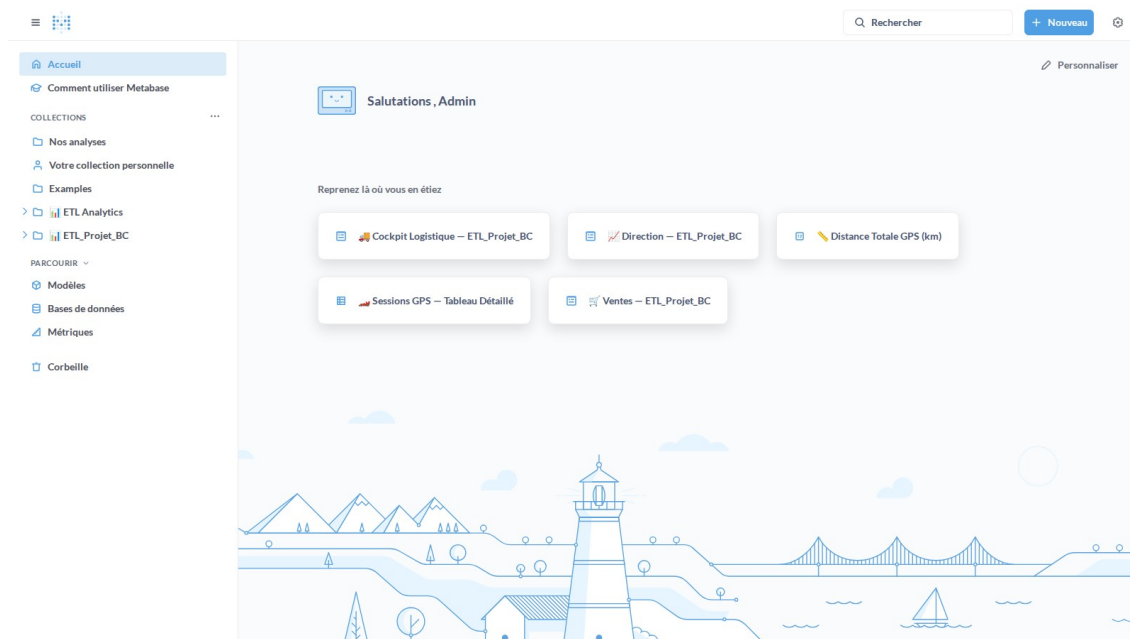


Figure B.1 — Accueil Metabase (profil Admin ETL)

B.2 Administrateur — Browse Databases

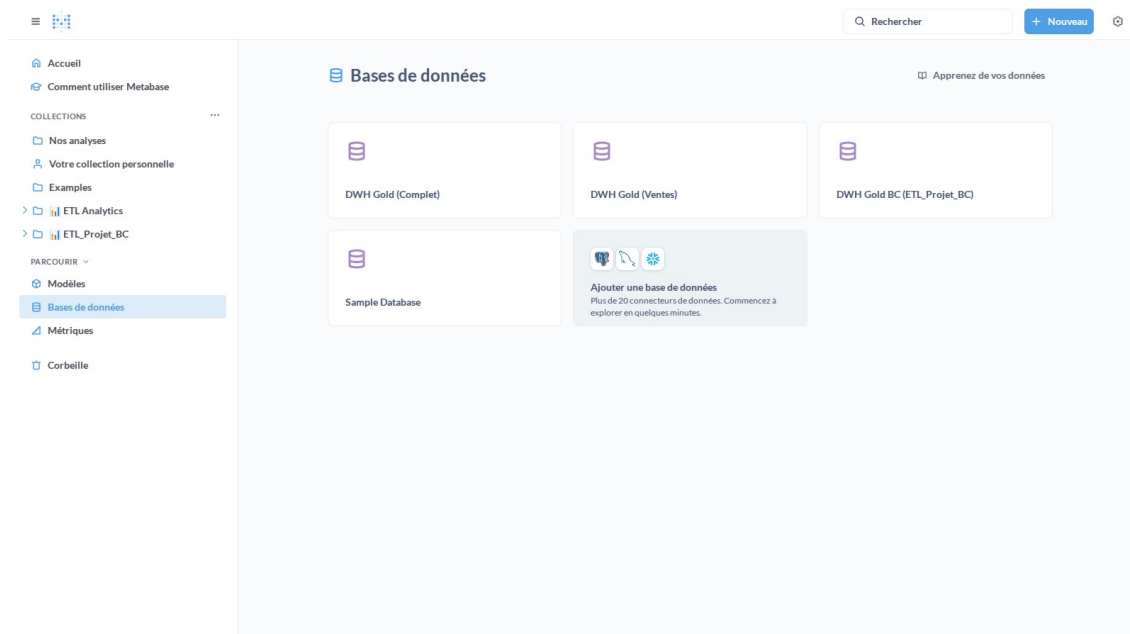


Figure B.2 — Bases de données disponibles (dwh_gold_bc)

B.3 Administrateur — Gestion des utilisateurs

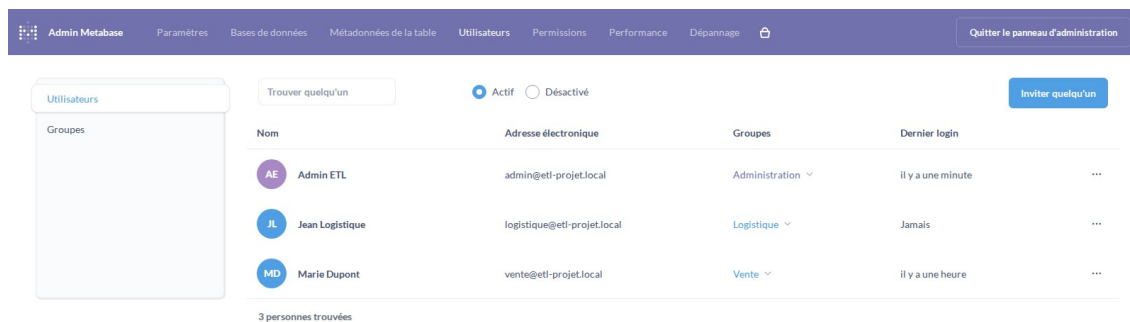


Figure B.3 — Gestion des utilisateurs Metabase (3 profils)

B.4 Dashboard #21 — Cockpit Logistique ETL_Projet_BC (Admin)

Dashboard structuré en 3 zones décisionnelles, basé sur bc_gold.v_distances_clean et bc_gold.v_logistics_cost (real_session_date).

- Zone 1 — KPIs Flash : CPK Flotte (€/km), Coût total GPS (€), Économies vs marché (€), Km GPS total
- Zone 2 — Analyses métier : coût par véhicule (bar chart), Distance vs Coût par session (scatter), tendance flotte (area chart)
- Zone 3 — Management par exception : alertes CPK > +15% moyenne, véhicules inconnus, sessions incomplètes

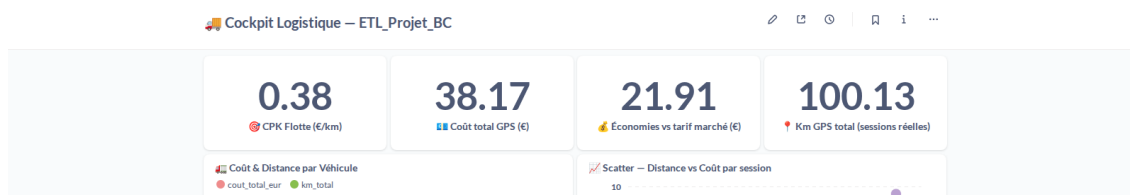


Figure B.4 — Cockpit Logistique #21 — Zone 1 : KPIs Flash (CPK, Coût GPS, Économies, Km)

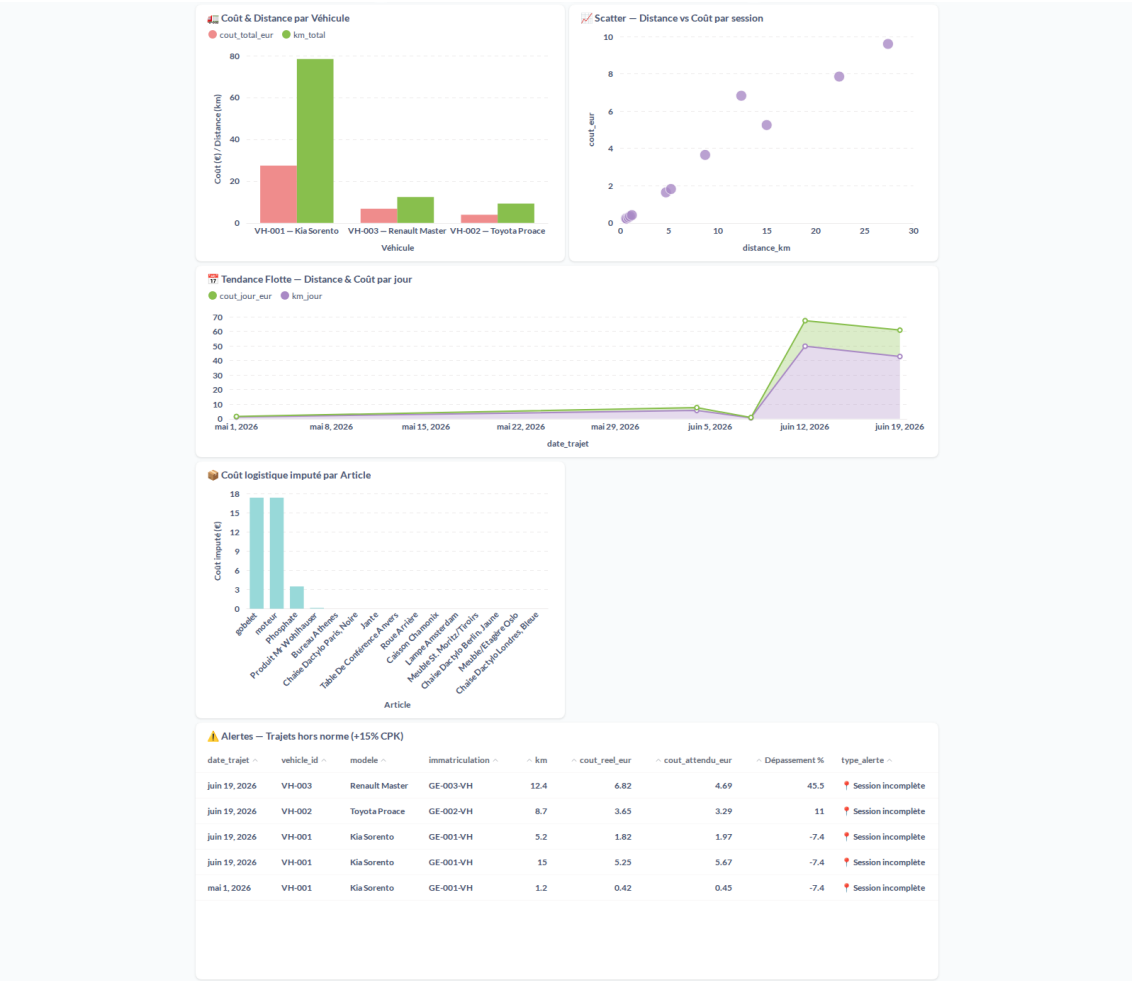


Figure B.4b — Cockpit Logistique #21 — Zone 2 : Analyses métier

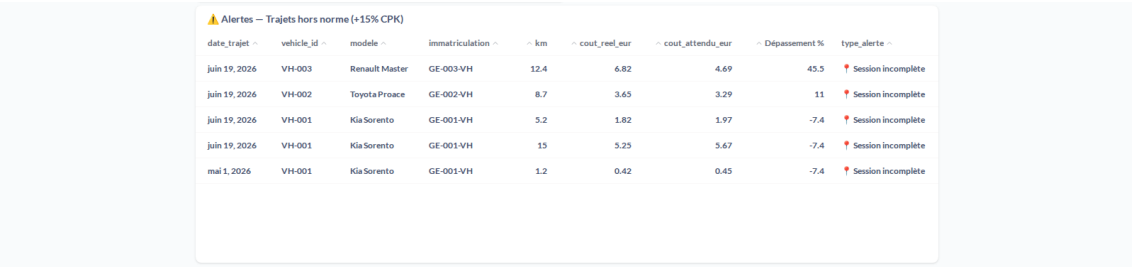


Figure B.4c — Cockpit Logistique #21 — Zone 3 : Alertes et anomalies

B.5 Dashboard #19 — Ventes ETL_Projet_BC (Admin)

4 KPIs : CA Total BC 9,9M€ | 25 900 commandes distinctes | Panier moyen 17,99€ | 43 pays couverts. Visualisations : Évolution CA mensuel (line chart), CA par pays Top 15, Top 10 articles et clients par CA, CA par catégorie (donut), Volume vendu par pays.

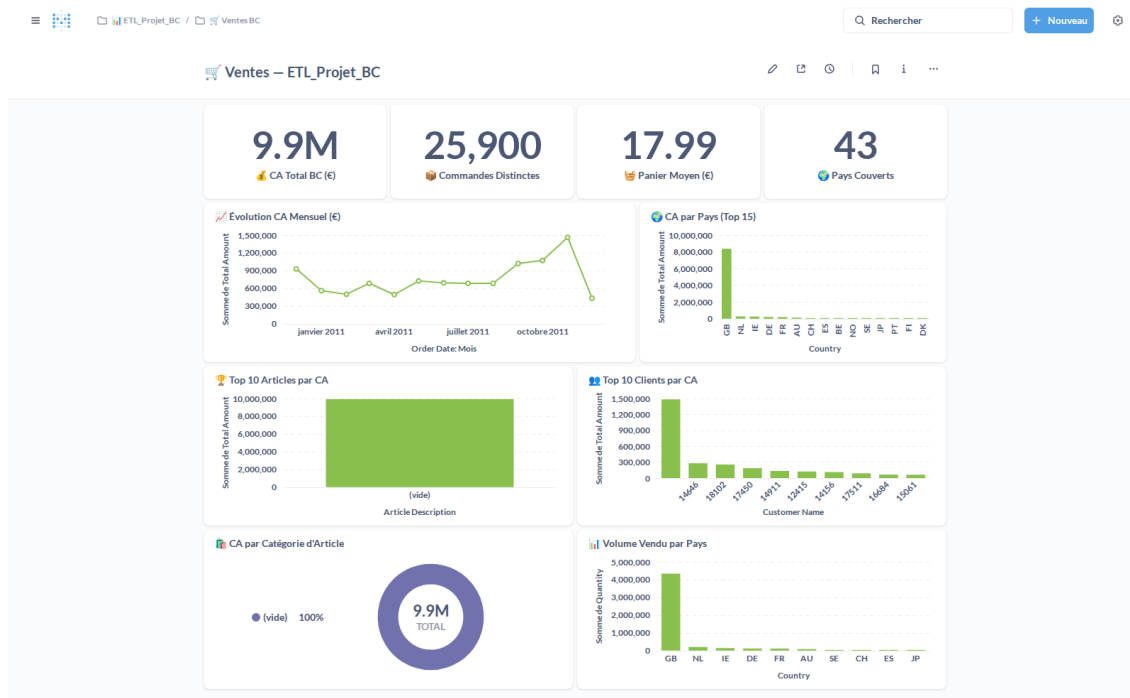


Figure B.5 — Dashboard Ventes #19 — 9,9M€ CA, 25 900 commandes, 43 pays

B.6 Dashboard #20 — Direction ETL_Projet_BC (Admin)

8 KPIs consolidés : CA Total 9,9M€ | 90 clients actifs | 193 références articles | 101 expéditions | 434,91 km GPS total | 58 sessions GPS | 551 899 lignes e-commerce BC. Visualisations : CA mensuel direction, Top 10 pays par CA, Volume par hub logistique, Expéditions par mois. Accessible en lecture aux groupes Logistique et Vente.

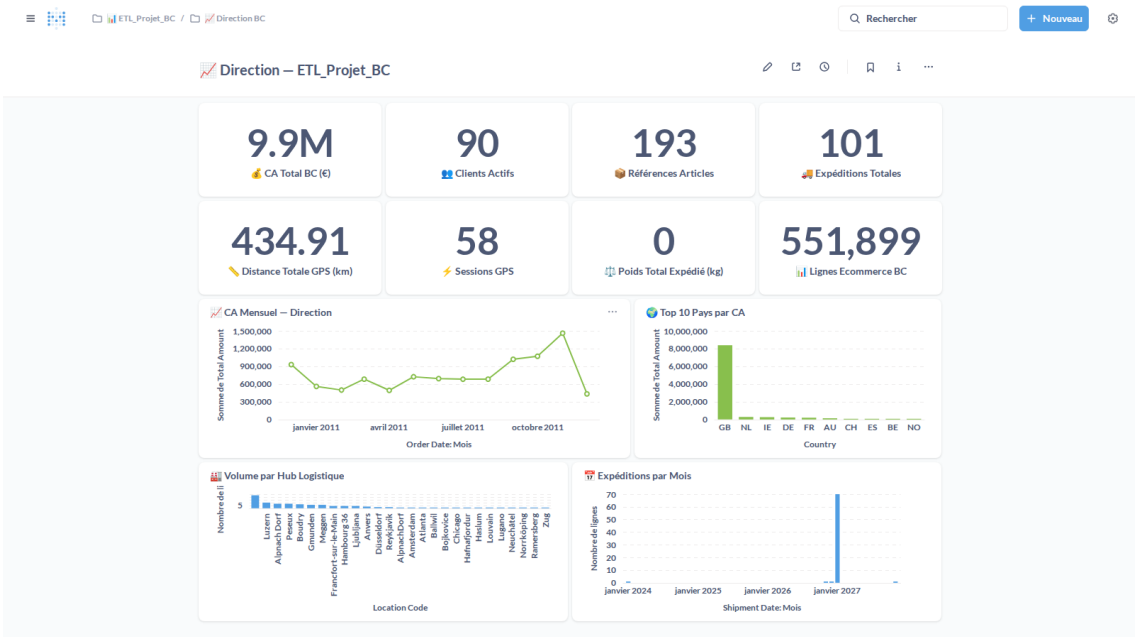


Figure B.6 — Dashboard Direction #20 (Admin + Logistique + Vente)

B.7 Profil Ventes — Accueil (Marie Dupont)

Le profil Ventes (Marie Dupont) accède à 3 collections en lecture seule : Direction BC, Ventes et Ventes BC. Aucun accès aux collections Logistique BC ni aux outils d'administration. Pas de bouton d'édition sur les dashboards.

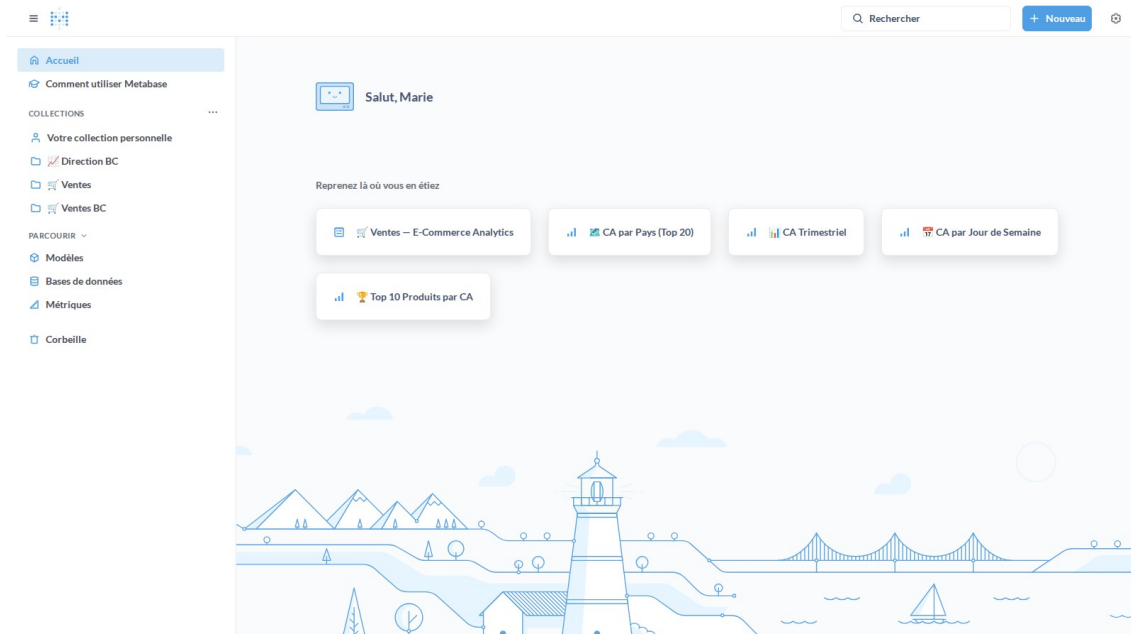


Figure B.7 — Accueil Metabase (profil Ventes — collections Direction BC, Ventes, Ventes BC)

B.8 Profil Ventes — Dashboard #19 Ventes ETL_Projet_BC

Vue en lecture seule du dashboard #19 — mêmes données que l'admin, sans icône d'édition.
Breadcrumb : Ventes BC.

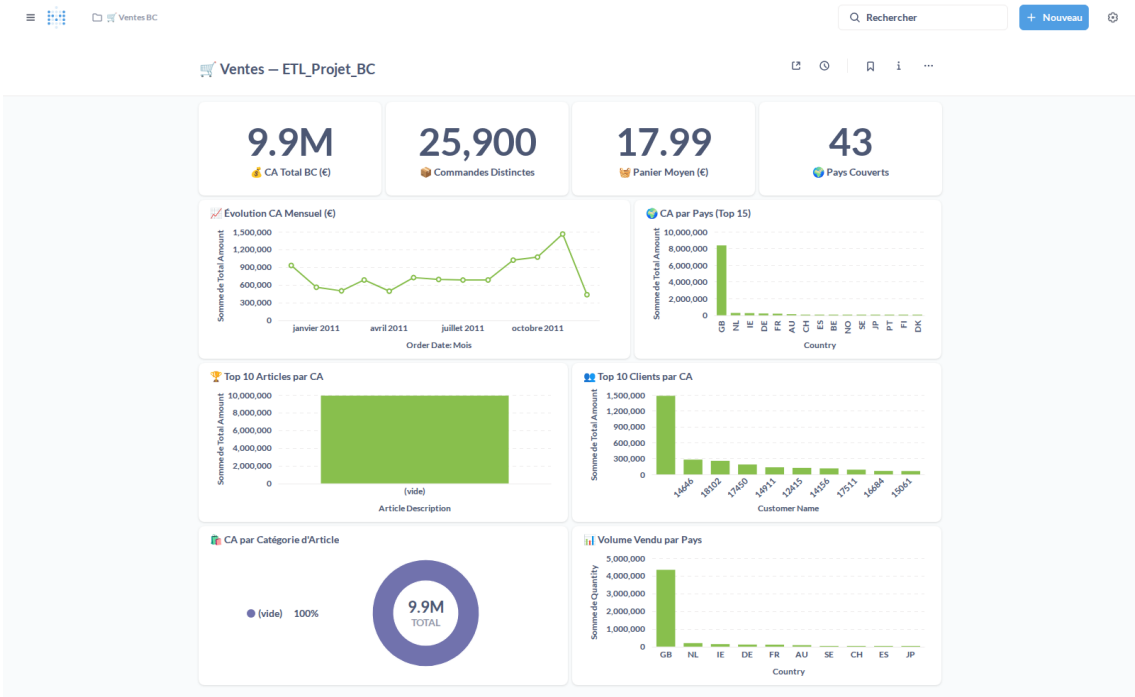


Figure B.8 — Dashboard Ventes #19 (profil Ventes — lecture seule)

B.9 Profil Ventes — Dashboard #20 Direction ETL_Projet_BC

Le profil Ventes accède aussi au dashboard Direction #20 en lecture seule, partagé avec le profil Logistique.

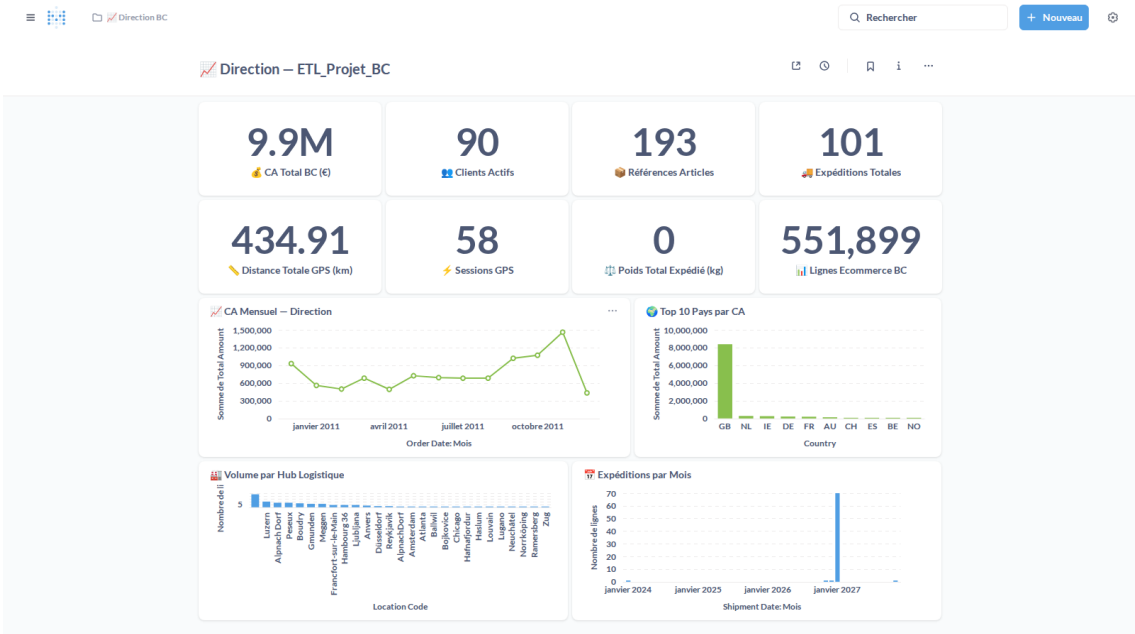


Figure B.9 — Dashboard Direction #20 (profil Ventes — lecture seule)

B.10 Profil Logistique — Accueil (Jean Logistique)

Jean Logistique accède à trois collections : Direction BC, Logistique et Logistique BC. Ses dashboards sont le Cockpit Logistique #21 et le dashboard Direction #20.

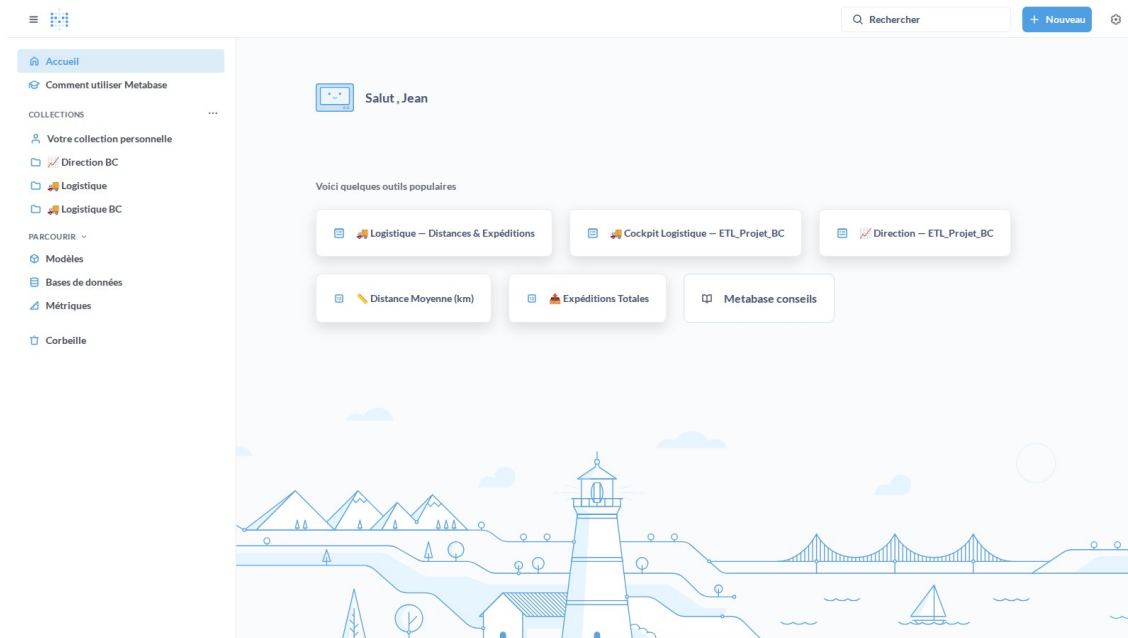


Figure B.10 — Accueil Metabase (profil Logistique — collections Direction BC, Logistique, Logistique BC)

B.11 Profil Logistique — Dashboard #21 Cockpit Logistique

ETL_Projet_BC

Le Cockpit Logistique affiche les 4 KPIs flash (CPK Flotte 0.38 €/km, Coût GPS 38.17 €, Économies 21.91 €, Km GPS 100.13), le scatter Distance vs Coût, la tendance journalière, le coût par article et la table des alertes trajets hors norme. Vue en lecture seule pour le profil Logistique.

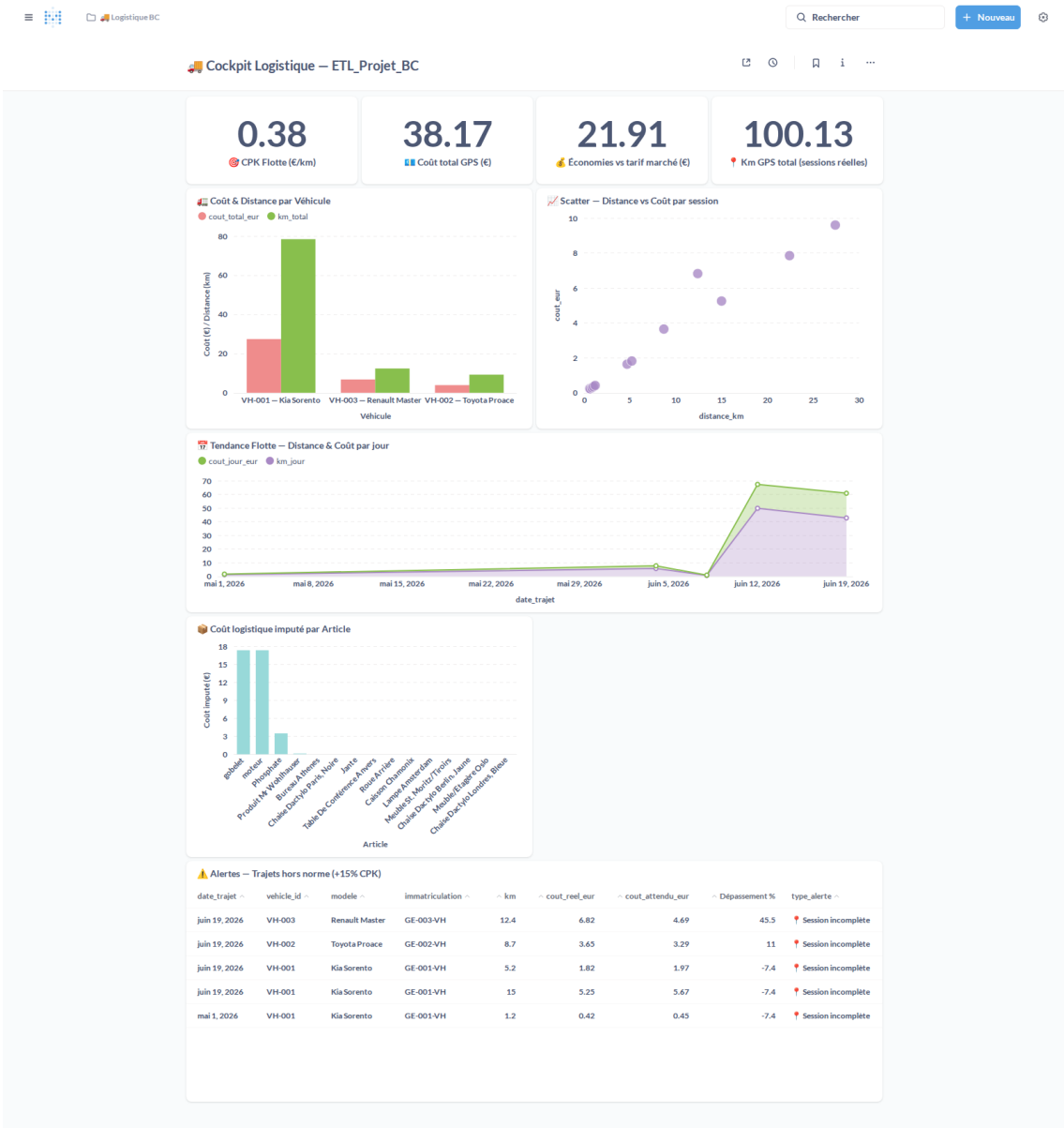


Figure B.11 — Dashboard Cockpit Logistique #21 (profil Logistique — lecture seule)

B.12 Profil Logistique — Dashboard #20 Direction ETL_Projet_BC

Le profil Logistique accède aussi au dashboard Direction #20 en lecture seule, partagé avec le profil Ventres.

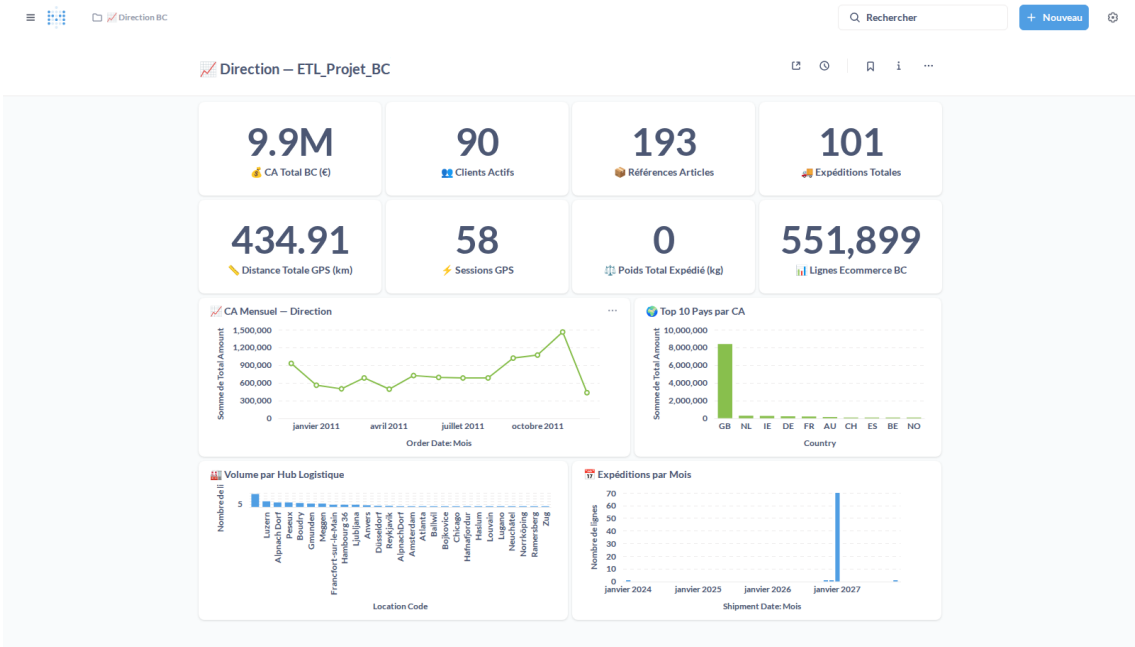


Figure B.12 — Dashboard Direction #20 (profil Logistique — lecture seule)

Annexe C — Scripts et code source

Cette annexe regroupe l'intégralité du code source produit dans le cadre du projet ETL_Projet_BC : extension AL Business Central (tables, API pages, pages liste et codeunit), migration SQL, script Python d'import des workflows, et noeuds JavaScript des 13 workflows n8n.

C.1 Extension AL — Tables (Table50200 – Table50205)

Six tables AL sont définies dans le namespace 50200–50205 pour stocker les données ETL directement dans Business Central 26.0.

Table 50200 — ETL BC Shipment Header

```
table 50200 "ETL BC Shipment Header"
{
    Caption = 'ETL BC Shipment Header';
    DataClassification = CustomerContent;

    fields
    {
        field(1; "Header Id"; Code[50])
        {
            Caption = 'Header Id';
            DataClassification = CustomerContent;
        }
        field(2; "Customer No"; Code[20])
        {
            Caption = 'Customer No';
            DataClassification = CustomerContent;
        }
        field(3; "Order No"; Code[20])
        {
            Caption = 'Order No';
            DataClassification = CustomerContent;
        }
        field(4; "Shipment Date"; Date)
        {
            Caption = 'Shipment Date';
            DataClassification = CustomerContent;
        }
        field(5; "Ship To City"; Text[100])
        {
            Caption = 'Ship To City';
            DataClassification = CustomerContent;
        }
        field(6; "Ship To Country"; Code[10])
        {
            Caption = 'Ship To Country';
            DataClassification = CustomerContent;
        }
        field(7; "ETL Loaded At"; DateTime)
        {
            Caption = 'ETL Loaded At';
            DataClassification = SystemMetadata;
        }
    }

    keys
    {
        key(PK; "Header Id") { Clustered = true; }
    }
}
```


Table 50201 — ETL BC Distance (sessions GPS)

```
table 50201 "ETL BC Distance"
{
    Caption = 'ETL BC Distance';
    DataClassification = CustomerContent;

    fields
    {
        field(1; "Session Id"; Text[50])
        {
            Caption = 'Session Id';
            DataClassification = CustomerContent;
        }
        field(2; "Session Date"; Date)
        {
            Caption = 'Session Date';
            DataClassification = CustomerContent;
        }
        field(3; "Total Distance Km"; Decimal)
        {
            Caption = 'Total Distance Km';
            DataClassification = CustomerContent;
        }
        field(4; "Total Distance Meters"; Decimal)
        {
            Caption = 'Total Distance Meters';
            DataClassification = CustomerContent;
        }
        field(5; "Max Speed Kmh"; Decimal)
        {
            Caption = 'Max Speed Kmh';
            DataClassification = CustomerContent;
        }
        field(6; "Avg Speed Kmh"; Decimal)
        {
            Caption = 'Avg Speed Kmh';
            DataClassification = CustomerContent;
        }
        field(7; "Total Active Seconds"; Integer)
        {
            Caption = 'Total Active Seconds';
            DataClassification = CustomerContent;
        }
        field(8; "Event Count"; Integer)
        {
            Caption = 'Event Count';
            DataClassification = CustomerContent;
        }
        field(9; "First Event At"; DateTime)
        {
            Caption = 'First Event At';
            DataClassification = CustomerContent;
        }
        field(10; "Last Event At"; DateTime)
        {
            Caption = 'Last Event At';
            DataClassification = CustomerContent;
        }
        field(11; "ETL Loaded At"; DateTime)
        {
            Caption = 'ETL Loaded At';
            DataClassification = SystemMetadata;
        }
        field(12; "Real Session Date"; Text[10])
        {
            Caption = 'Real Session Date';
            DataClassification = CustomerContent;
        }
    }
}
```

```

        Caption = 'Real Session Date';
        DataClassification = CustomerContent;
    }
}

keys
{
    key(PK; "Session Id") { Clustered = true; }
    key(ByDate; "Session Date") { }
}
}

```

Table 50202 — ETL BC Shipment Line

```

table 50202 "ETL BC Shipment Line"
{
    Caption = 'ETL BC Shipment Line';
    DataClassification = CustomerContent;

    fields
    {
        field(1; "Document No"; Code[20])
        {
            Caption = 'Document No';
            DataClassification = CustomerContent;
        }
        field(2; "Line No"; Integer)
        {
            Caption = 'Line No';
            DataClassification = CustomerContent;
        }
        field(3; "Article Id"; Code[50])
        {
            Caption = 'Article Id';
            DataClassification = CustomerContent;
        }
        field(4; "Quantity"; Decimal)
        {
            Caption = 'Quantity';
            DataClassification = CustomerContent;
        }
        field(5; "Location Code"; Code[10])
        {
            Caption = 'Location Code';
            DataClassification = CustomerContent;
        }
        field(6; "ETL Loaded At"; DateTime)
        {
            Caption = 'ETL Loaded At';
            DataClassification = SystemMetadata;
        }
    }

    keys
    {
        key(PK; "Document No", "Line No") { Clustered = true; }
    }
}

```

Table 50203 — ETL BC Ecommerce Line

```

table 50203 "ETL BC Ecommerce Line"
{

```

```

Caption = 'ETL BC Ecommerce Line';
DataClassification = CustomerContent;

fields
{
    field(1; "Invoice No"; Code[20])
    {
        Caption = 'Invoice No';
        DataClassification = CustomerContent;
    }
    field(2; "Line No"; Integer)
    {
        Caption = 'Line No';
        DataClassification = CustomerContent;
    }
    field(3; "Stock Code"; Code[50])
    {
        Caption = 'Stock Code';
        DataClassification = CustomerContent;
    }
    field(4; "Description"; Text[100])
    {
        Caption = 'Description';
        DataClassification = CustomerContent;
    }
    field(5; "Quantity"; Decimal)
    {
        Caption = 'Quantity';
        DataClassification = CustomerContent;
    }
    field(6; "Unit Price"; Decimal)
    {
        Caption = 'Unit Price';
        DataClassification = CustomerContent;
    }
    field(7; "Total Amount"; Decimal)
    {
        Caption = 'Total Amount';
        DataClassification = CustomerContent;
    }
    field(8; "Customer Id"; Code[50])
    {
        Caption = 'Customer Id';
        DataClassification = CustomerContent;
    }
    field(9; "Country"; Code[10])
    {
        Caption = 'Country';
        DataClassification = CustomerContent;
    }
    field(10; "Invoice Date"; Date)
    {
        Caption = 'Invoice Date';
        DataClassification = CustomerContent;
    }
    field(11; "ETL Loaded At"; DateTime)
    {
        Caption = 'ETL Loaded At';
        DataClassification = SystemMetadata;
    }
}

keys
{
    key(PK; "Invoice No", "Line No") { Clustered = true; }
}
}

```

Table 50204 — ETL BC Customer

```
table 50204 "ETL BC Customer"
{
    Caption = 'ETL BC Customer';
    DataClassification = CustomerContent;

    fields
    {
        field(1; "Customer Id"; Code[50])
        {
            Caption = 'Customer Id';
            DataClassification = CustomerContent;
        }
        field(2; "Name"; Text[100])
        {
            Caption = 'Name';
            DataClassification = CustomerContent;
        }
        field(3; "Country"; Code[10])
        {
            Caption = 'Country';
            DataClassification = CustomerContent;
        }
        field(4; "Post Code"; Code[20])
        {
            Caption = 'Post Code';
            DataClassification = CustomerContent;
        }
        field(5; "Location Code"; Code[20])
        {
            Caption = 'Location Code';
            DataClassification = CustomerContent;
        }
        field(6; "Currency Code"; Code[10])
        {
            Caption = 'Currency Code';
            DataClassification = CustomerContent;
        }
        field(7; "Salesperson Code"; Code[20])
        {
            Caption = 'Salesperson Code';
            DataClassification = CustomerContent;
        }
        field(8; "ETL Loaded At"; DateTime)
        {
            Caption = 'ETL Loaded At';
            DataClassification = SystemMetadata;
        }
    }

    keys
    {
        key(PK; "Customer Id") { Clustered = true; }
    }
}
```

Table 50205 — ETL BC Article

```
table 50205 "ETL BC Article"
{
    Caption = 'ETL BC Article';
    DataClassification = CustomerContent;

    fields
    {
```

```

    field(1; "Article Id"; Code[50])
    {
        Caption = 'Article Id';
        DataClassification = CustomerContent;
    }
    field(2; "Description"; Text[100])
    {
        Caption = 'Description';
        DataClassification = CustomerContent;
    }
    field(3; "Unit Of Measure"; Code[20])
    {
        Caption = 'Unit Of Measure';
        DataClassification = CustomerContent;
    }
    field(4; "Type"; Text[50])
    {
        Caption = 'Type';
        DataClassification = CustomerContent;
    }
    field(5; "Item Category"; Code[20])
    {
        Caption = 'Item Category';
        DataClassification = CustomerContent;
    }
    field(6; "Unit Price"; Decimal)
    {
        Caption = 'Unit Price';
        DataClassification = CustomerContent;
    }
    field(7; "Unit Cost"; Decimal)
    {
        Caption = 'Unit Cost';
        DataClassification = CustomerContent;
    }
    field(8; "Costing Method"; Text[50])
    {
        Caption = 'Costing Method';
        DataClassification = CustomerContent;
    }
    field(9; "ETL Loaded At"; DateTime)
    {
        Caption = 'ETL Loaded At';
        DataClassification = SystemMetadata;
    }
}

keys
{
    key(PK; "Article Id") { Clustered = true; }
}
}

```

C.2 Extension AL — API Pages OData (APIPage50200 – APIPage50214)

Sept pages API exposent les tables ETL via OData v4 sous le publisher etlpipeline / groupe warehouse / version v1.0. L'APIPage 50214 gère l'insertion temps réel des kilométrages GPS.

APIPage 50200 — etlShipmentHeaders

```
page 50206 "ETL BC Shipment Headers Api"
{
    PageType = API;
    APIPublisher = 'etlpipeline';
    APIGroup = 'warehouse';
    APIVersion = 'v1.0';
    EntityName = 'etlShipmentHeader';
    EntitySetName = 'etlShipmentHeaders';
    SourceTable = "ETL BC Shipment Header";
    DelayedInsert = true;
    InsertAllowed = true;
    ModifyAllowed = true;
    DeleteAllowed = false;
    ODataKeyFields = "Header Id";

    layout
    {
        area(Content)
        {
            field(headerId; Rec."Header Id") { Caption = 'headerId'; }
            field(customerNo; Rec."Customer No") { Caption = 'customerNo'; }
            field(orderNo; Rec."Order No") { Caption = 'orderNo'; }
            field(shipmentDate; Rec."Shipment Date") { Caption = 'shipmentDate'; }
            field(shipToCity; Rec."Ship To City") { Caption = 'shipToCity'; }
            field(shipToCountry; Rec."Ship To Country") { Caption = 'shipToCountry'; }
            field(etlLoadedAt; Rec."ETL Loaded At") { Caption = 'etlLoadedAt'; }
        }
    }
}
```

APIPage 50201 — etlDistances

```
page 50207 "ETL BC Distances Api"
{
    PageType = API;
    APIPublisher = 'etlpipeline';
    APIGroup = 'warehouse';
    APIVersion = 'v1.0';
    EntityName = 'etlDistance';
    EntitySetName = 'etlDistances';
    SourceTable = "ETL BC Distance";
    DelayedInsert = true;
    InsertAllowed = true;
    ModifyAllowed = true;
    DeleteAllowed = false;
    ODataKeyFields = "Session Id";

    layout
    {
        area(Content)
        {
            field(sessionId; Rec."Session Id") { Caption = 'sessionId'; }
            field(sessionDate; Rec."Session Date") { Caption = 'sessionDate'; }
            field(totalDistanceKm; Rec."Total Distance Km") { Caption = 'totalDistanceKm'; }
            field(totalDistanceMeters; Rec."Total Distance Meters") { Caption = 'totalDistanceMeters'; }
            field(maxSpeedKmh; Rec."Max Speed Kmh") { Caption = 'maxSpeedKmh'; }
            field(avgSpeedKmh; Rec."Avg Speed Kmh") { Caption = 'avgSpeedKmh'; }
        }
    }
}
```

```

        field(totalActiveSeconds; Rec."Total Active Seconds") { Caption = 'totalActiveSeconds'; }
        field(eventCount; Rec."Event Count") { Caption = 'eventCount'; }
        field(firstEventAt; Rec."First Event At") { Caption = 'firstEventAt'; }
        field(lastEventAt; Rec."Last Event At") { Caption = 'lastEventAt'; }
        field(etlLoadedAt; Rec."ETL Loaded At") { Caption = 'etlLoadedAt'; }
        field(realSessionDate; Rec."Real Session Date") { Caption = 'realSessionDate'; }
    }
}

```

APIPage 50202 — etlShipmentLines

```

page 50202 "ETL BC Shipment Lines Api"
{
    PageType = API;
    APIPublisher = 'etlpipeline';
    APIGroup = 'warehouse';
    APIVersion = 'v1.0';
    EntityName = 'etlShipmentLine';
    EntitySetName = 'etlShipmentLines';
    SourceTable = "ETL BC Shipment Line";
    DelayedInsert = true;
    InsertAllowed = true;
    ModifyAllowed = true;
    DeleteAllowed = false;
    ODataKeyFields = "Document No", "Line No";

    layout
    {
        area(Content)
        {
            field(documentNo; Rec."Document No") { Caption = 'documentNo'; }
            field(lineNo; Rec."Line No") { Caption = 'lineNo'; }
            field(articleId; Rec."Article Id") { Caption = 'articleId'; }
            field(quantity; Rec."Quantity") { Caption = 'quantity'; }
            field(locationCode; Rec."Location Code") { Caption = 'locationCode'; }
            field(etlLoadedAt; Rec."ETL Loaded At") { Caption = 'etlLoadedAt'; }
        }
    }
}

```

APIPage 50203 — etlEcommerceLines

```

page 50203 "ETL BC Ecommerce Lines Api"
{
    PageType = API;
    APIPublisher = 'etlpipeline';
    APIGroup = 'warehouse';
    APIVersion = 'v1.0';
    EntityName = 'etlEcommerceLine';
    EntitySetName = 'etlEcommerceLines';
    SourceTable = "ETL BC Ecommerce Line";
    DelayedInsert = true;
    InsertAllowed = true;
    ModifyAllowed = true;
    DeleteAllowed = false;
    ODataKeyFields = "Invoice No", "Line No";

    layout
    {
        area(Content)
        {
            field(invoiceNo; Rec."Invoice No") { Caption = 'invoiceNo'; }
        }
    }
}

```

```

        field(lineNo;      Rec."Line No")      { Caption = 'lineNo'; }
        field(stockCode;   Rec."Stock Code")    { Caption = 'stockCode'; }
        field(description; Rec."Description")  { Caption = 'description'; }
        field(quantity;    Rec."Quantity")      { Caption = 'quantity'; }
        field(unitPrice;   Rec."Unit Price")    { Caption = 'unitPrice'; }
        field(totalAmount; Rec."Total Amount")  { Caption = 'totalAmount'; }
        field(customerId;  Rec."Customer Id")   { Caption = 'customerId'; }
        field(country;     Rec."Country")       { Caption = 'country'; }
        field(invoiceDate; Rec."Invoice Date")  { Caption = 'invoiceDate'; }
        field(etlLoadedAt; Rec."ETL Loaded At") { Caption = 'etlLoadedAt'; }
    }
}

```

APIPage 50204 — etlCustomers

page 50212 "ETL BC Customers Api"

```

{
    PageType = API;
    APIPublisher = 'etlpipeline';
    APIGroup = 'warehouse';
    APIVersion = 'v1.0';
    EntityName = 'etlCustomer';
    EntitySetName = 'etlCustomers';
    SourceTable = "ETL BC Customer";
    DelayedInsert = true;
    InsertAllowed = true;
    ModifyAllowed = true;
    DeleteAllowed = false;
    ODataKeyFields = "Customer Id";

    layout
    {
        area(Content)
        {
            field(customerId;      Rec."Customer Id")      { Caption = 'customerId'; }
            field(name;            Rec."Name")              { Caption = 'name'; }
            field(country;         Rec."Country")           { Caption = 'country'; }
            field(postCode;       Rec."Post Code")          { Caption = 'postCode'; }
            field(locationCode;    Rec."Location Code")     { Caption = 'locationCode'; }
            field(currencyCode;    Rec."Currency Code")     { Caption = 'currencyCode'; }
            field(salespersonCode; Rec."Salesperson Code") { Caption = 'salespersonCode'; }
            field(etlLoadedAt;     Rec."ETL Loaded At")     { Caption = 'etlLoadedAt'; }
        }
    }
}

```

APIPage 50205 — etlArticles

page 50213 "ETL BC Articles Api"

```

{
    PageType = API;
    APIPublisher = 'etlpipeline';
    APIGroup = 'warehouse';
    APIVersion = 'v1.0';
    EntityName = 'etlArticle';
    EntitySetName = 'etlArticles';
    SourceTable = "ETL BC Article";
    DelayedInsert = true;
    InsertAllowed = true;
    ModifyAllowed = true;
    DeleteAllowed = false;
    ODataKeyFields = "Article Id";
}

```



```

layout
{
    area(Content)
    {
        field(articleId;      Rec."Article Id")      { Caption = 'articleId'; }
        field(description;    Rec."Description")      { Caption = 'description'; }
        field(unitOfMeasure;  Rec."Unit Of Measure")  { Caption = 'unitOfMeasure'; }
        field(type;           Rec."Type")             { Caption = 'type'; }
        field(itemCategory;   Rec."Item Category")     { Caption = 'itemCategory'; }
        field(unitPrice;      Rec."Unit Price")        { Caption = 'unitPrice'; }
        field(unitCost;       Rec."Unit Cost")          { Caption = 'unitCost'; }
        field(costingMethod;  Rec."Costing Method")    { Caption = 'costingMethod'; }
        field(etlLoadedAt;    Rec."ETL Loaded At")     { Caption = 'etlLoadedAt'; }
    }
}

```

APIPage 50214 — etlDistanceUpdates (temps réel)

page 50214 "ETL BC Update Distance API"

```

{
    PageType = API;
    APIPublisher = 'etlpipeline';
    APIGroup = 'warehouse';
    APIVersion = 'v1.0';
    EntityName = 'etlDistanceUpdate';
    EntitySetName = 'etlDistanceUpdates';
    SourceTable = "ETL BC Distance";
    DelayedInsert = true;
    InsertAllowed = true;
    ModifyAllowed = false;
    DeleteAllowed = false;
    ODataKeyFields = "Session Id";

    layout
    {
        area(Content)
        {
            field(sessionId;      Rec."Session Id")      { Caption = 'sessionId'; }
            field(totalDistanceKm; Rec."Total Distance Km") { Caption = 'totalDistanceKm'; }
            field(etlLoadedAt;    Rec."ETL Loaded At")    { Caption = 'etlLoadedAt'; }
        }
    }

    trigger OnInsertRecord(BelowxRec: Boolean): Boolean
    var
        Mgr: Codeunit "ETL BC Import Manager";
    begin
        Mgr.UpsertDistanceRealtime(Rec."Session Id", Rec."Total Distance Km");
        exit(false);
    end;
}

```

C.3 Extension AL — Pages Liste et Codeunit Import Manager

Sept pages de type List permettent la consultation des données ETL directement dans BC. Le Codeunit 50210 centralise toute la logique d'import (upsert par clé naturelle) et expose les procédures appelées par les API pages et le dashboard.

Page 50200 — Shipment Header List

```
page 50200 "ETL BC Shipment Header List"
{
    PageType = List;
    Caption = 'Expéditions – En-têtes';
    SourceTable = "ETL BC Shipment Header";
    ApplicationArea = All;
    UsageCategory = Lists;
    Editable = false;

    layout
    {
        area(Content)
        {
            repeater(Headers)
            {
                field("Header Id"; Rec."Header Id") { ApplicationArea = All; }
                field("Customer No"; Rec."Customer No") { ApplicationArea = All; }
                field("Order No"; Rec."Order No") { ApplicationArea = All; }
                field("Shipment Date"; Rec."Shipment Date") { ApplicationArea = All; }
                field("Ship To City"; Rec."Ship To City") { ApplicationArea = All; }
                field("Ship To Country"; Rec."Ship To Country") { ApplicationArea = All; }
                field("ETL Loaded At"; Rec."ETL Loaded At") { ApplicationArea = All; }
            }
        }
    }
}
```

Page 50201 — Distance List (GPS temps réel)

```
page 50201 "ETL BC Distance List"
{
    PageType = List;
    Caption = 'Kilomètres GPS – Sessions temps réel';
    SourceTable = "ETL BC Distance";
    ApplicationArea = All;
    UsageCategory = Lists;
    Editable = false;

    layout
    {
        area(Content)
        {
            repeater(Sessions)
            {
                field("Session Id"; Rec."Session Id") { ApplicationArea = All; }
                field("Real Session Date"; Rec."Real Session Date") { ApplicationArea = All; }
                field("Session Date"; Rec."Session Date") { ApplicationArea = All; }
                field("Total Distance Km"; Rec."Total Distance Km")
                {
                    ApplicationArea = All;
                    DecimalPlaces = 0:4;
                }
                field("Max Speed Kmh"; Rec."Max Speed Kmh")
                {
                    ApplicationArea = All;
                    DecimalPlaces = 0:1;
                }
            }
        }
    }
}
```



```

        field("Stock Code"; Rec."Stock Code") { ApplicationArea = All; }
        field("Description"; Rec."Description") { ApplicationArea = All; }
        field("Quantity"; Rec."Quantity") { ApplicationArea = All; DecimalPlaces = 0:4; }
        field("Unit Price"; Rec."Unit Price") { ApplicationArea = All; DecimalPlaces = 2:2; }
        field("Total Amount"; Rec."Total Amount") { ApplicationArea = All; DecimalPlaces = 2:2; }
        field("Customer Id"; Rec."Customer Id") { ApplicationArea = All; }
        field("Country"; Rec."Country") { ApplicationArea = All; }
        field("Invoice Date"; Rec."Invoice Date") { ApplicationArea = All; }
        field("ETL Loaded At"; Rec."ETL Loaded At") { ApplicationArea = All; }
    }
}
}

```

Page 50204 — Customer List

```

page 50204 "ETL BC Customer List"
{
    PageType = List;
    Caption = 'Clients importés';
    SourceTable = "ETL BC Customer";
    ApplicationArea = All;
    UsageCategory = Lists;
    Editable = false;

    layout
    {
        area(Content)
        {
            repeater(Customers)
            {
                field("Customer Id"; Rec."Customer Id") { ApplicationArea = All; }
                field("Name"; Rec."Name") { ApplicationArea = All; }
                field("Country"; Rec."Country") { ApplicationArea = All; }
                field("Post Code"; Rec."Post Code") { ApplicationArea = All; }
                field("Location Code"; Rec."Location Code") { ApplicationArea = All; }
                field("Currency Code"; Rec."Currency Code") { ApplicationArea = All; }
                field("Salesperson Code"; Rec."Salesperson Code") { ApplicationArea = All; }
                field("ETL Loaded At"; Rec."ETL Loaded At") { ApplicationArea = All; }
            }
        }
    }
}

```

Page 50205 — Article List

```

page 50205 "ETL BC Article List"
{
    PageType = List;
    Caption = 'Articles importés';
    SourceTable = "ETL BC Article";
    ApplicationArea = All;
    UsageCategory = Lists;
    Editable = false;

    layout
    {
        area(Content)
        {
            repeater(Articles)
            {
                field("Article Id"; Rec."Article Id") { ApplicationArea = All; }
                field("Description"; Rec."Description") { ApplicationArea = All; }
            }
        }
    }
}

```

```

        field("Unit Of Measure"; Rec."Unit Of Measure") { ApplicationArea = All; }
        field("Type"; Rec."Type") { ApplicationArea = All; }
        field("Item Category"; Rec."Item Category") { ApplicationArea = All; }
        field("Unit Price"; Rec."Unit Price") { ApplicationArea = All; DecimalPlaces = 2:4; }
        field("Unit Cost"; Rec."Unit Cost") { ApplicationArea = All; DecimalPlaces = 2:4; }
        field("Costing Method"; Rec."Costing Method") { ApplicationArea = All; }
        field("ETL Loaded At"; Rec."ETL Loaded At") { ApplicationArea = All; }
    }
}
}
}
}

```

Page 50211 — Dashboard ETL BC

```

page 50211 "ETL BC Dashboard"
{
    // -----
    // Tableau de bord ETL – visualisation des 6 tables Gold importées dans BC
    // Toutes les actions appellent uniquement Codeunit 50210 "ETL BC Import Manager"
    // -----
    PageType = Card;
    Caption = 'ETL – Tableau de Bord Logistique';
    UsageCategory = Lists;
    ApplicationArea = All;

    layout
    {
        area(Content)
        {
            // — Statistiques globales -----
            group(Stats)
            {
                Caption = 'État des données Gold importées';

                field(NbShipmentHeaders; GetCount('ShipmentHeaders'))
                {
                    Caption = 'Expéditions (En-têtes)';
                    ApplicationArea = All;
                    Editable = false;
                    Style = Strong;
                }
                field(NbDistances; GetCount('Distances'))
                {
                    Caption = 'Sessions GPS (Kilométrages)';
                    ApplicationArea = All;
                    Editable = false;
                    Style = Strong;
                }
                field(NbShipmentLines; GetCount('ShipmentLines'))
                {
                    Caption = 'Lignes d\'expédition';
                    ApplicationArea = All;
                    Editable = false;
                }
                field(NbEcommerceLines; GetCount('EcommerceLines'))
                {
                    Caption = 'Lignes e-commerce';
                    ApplicationArea = All;
                    Editable = false;
                }
                field(NbCustomers; GetCount('Customers'))
                {
                    Caption = 'Clients';
                    ApplicationArea = All;
                    Editable = false;
                }
                field(NbArticles; GetCount('Articles'))
            }
        }
    }
}

```

```

        {
            Caption = 'Articles';
            ApplicationArea = All;
            Editable = false;
        }
    }

    // — Navigation vers les listes détaillées —————
    group(DataAccess)
    {
        Caption = 'Accès aux données';
    }
}

actions
{
    area(Processing)
    {
        action(SyncAll)
        {
            Caption = 'Synchroniser Gold → BC';
            Tooltip = 'Déclenche la synchronisation complète des 6 tables Gold vers Business Central via
n8n.';

            ApplicationArea = All;
            Image = Refresh;
            Promoted = true;
            PromotedCategory = Process;
            PromotedIsBig = true;

            trigger OnAction()
            var
                ImportMgr: Codeunit "ETL BC Import Manager";
            begin
                ImportMgr.TriggerFullSync();
                CurrPage.Update(false);
            end;
        }

        action(ViewDistances)
        {
            Caption = 'Kilométrages GPS';
            Tooltip = 'Ouvrir la liste complète des sessions GPS.';
            ApplicationArea = All;
            Image = Map;
            RunObject = Page "ETL BC Distance List";
        }

        action(ViewShipmentHeaders)
        {
            Caption = 'Expéditions';
            Tooltip = 'Ouvrir la liste des en-têtes d''expédition.';
            ApplicationArea = All;
            Image = Shipment;
            RunObject = Page "ETL BC Shipment Header List";
        }

        action(ViewShipmentLines)
        {
            Caption = 'Lignes d''expédition';
            Tooltip = 'Ouvrir la liste des lignes d''expédition.';
            ApplicationArea = All;
            Image = AllLines;
            RunObject = Page "ETL BC Shipment Line List";
        }

        action(ViewEcommerceLines)
        {
            Caption = 'Lignes e-commerce';
            Tooltip = 'Ouvrir la liste des lignes e-commerce.';
            ApplicationArea = All;
        }
    }
}

```

```

        Image = OrderList;
        RunObject = Page "ETL BC Ecommerce Line List";
    }

    action(ViewCustomers)
    {
        Caption = 'Clients';
        ToolTip = 'Ouvrir la liste des clients importés.';
        ApplicationArea = All;
        Image = Customer;
        RunObject = Page "ETL BC Customer List";
    }

    action(ViewArticles)
    {
        Caption = 'Articles';
        ToolTip = 'Ouvrir la liste des articles importés.';
        ApplicationArea = All;
        Image = Item;
        RunObject = Page "ETL BC Article List";
    }
}

var
    ImportMgrGlobal: Codeunit "ETL BC Import Manager";

local procedure GetCount(TableName: Text[50]): Integer
begin
    exit(ImportMgrGlobal.GetRecordCount(TableName));
end;
}

```

Codeunit 50210 — ETL BC Import Manager

```

codeunit 50210 "ETL BC Import Manager"
{
    // -----
    // ETL BC Import Manager
    // Centralise toute la logique d'import Gold → Business Central.
    // Appelé par : les API pages (triggers), la page dashboard (actions),
    //              n8n via le webhook distance (WF-04B).
    // -----

    // =====
    // SYNC COMPLÈTE – déclenche WF-08 dans n8n (Gold PG → BC)
    // =====
    procedure TriggerFullSync()
    var
        Client: HttpClient;
        Request: HttpRequestMessage;
        Response: HttpResponseMessage;
        Content: HttpContent;
        ContentHeaders: HttpHeaders;
        JsonBody: Text;
        StatusMsg: Text;
    begin
        JsonBody := '{"source":"bc_manual","triggered_by":"ETL BC Dashboard"}';
        Content.WriteFrom(JsonBody);
        Content.GetHeaders(ContentHeaders);
        ContentHeaders.Remove('Content-Type');
        ContentHeaders.Add('Content-Type', 'application/json');

        Request.SetRequestUri('http://172.18.0.5:5678/webhook/run-gold-to-bc');
        Request.Method := 'POST';
        Request.Content := Content;
    end;
}

```

```

        if Client.Send(Request, Response) then begin
            if Response.IsSuccessStatusCode() then
                Message('Synchronisation Gold → BC déclenchée avec succès.')
            else begin
                StatusMsg := Format(Response.HttpStatusCode());
                Error('Erreur lors du déclenchement n8n. Code HTTP : %1', StatusMsg);
            end;
        end else
            Error('Impossible de joindre n8n. Vérifiez que le service est démarré.');
```

end;

```

// =====
// IMPORT SHIPMENT HEADERS – depuis le JSON reçu par l'API page
// =====
procedure UpsertShipmentHeader(
    HeaderIdVal: Code[50];
    CustomerNoVal: Code[20];
    OrderNoVal: Code[20];
    ShipmentDateVal: Date;
    ShipToCityVal: Text[100];
    ShipToCountryVal: Code[10])
var
    HeaderRec: Record "ETL BC Shipment Header";
begin
    if HeaderRec.Get(HeaderIdVal) then begin
        HeaderRec."Customer No" := CustomerNoVal;
        HeaderRec."Order No" := OrderNoVal;
        HeaderRec."Shipment Date" := ShipmentDateVal;
        HeaderRec."Ship To City" := ShipToCityVal;
        HeaderRec."Ship To Country" := ShipToCountryVal;
        HeaderRec."ETL Loaded At" := CurrentDateTime();
        HeaderRec.Modify(true);
    end else begin
        HeaderRec."Header Id" := HeaderIdVal;
        HeaderRec."Customer No" := CustomerNoVal;
        HeaderRec."Order No" := OrderNoVal;
        HeaderRec."Shipment Date" := ShipmentDateVal;
        HeaderRec."Ship To City" := ShipToCityVal;
        HeaderRec."Ship To Country" := ShipToCountryVal;
        HeaderRec."ETL Loaded At" := CurrentDateTime();
        HeaderRec.Insert(true);
    end;
end;

// =====
// IMPORT DISTANCES GPS – une ligne par session_id
// =====
procedure UpsertDistance(
    SessionIdVal: Text[50];
    SessionDateVal: Date;
    TotalDistKmVal: Decimal;
    TotalDistMetersVal: Decimal;
    MaxSpeedVal: Decimal;
    AvgSpeedVal: Decimal;
    ActiveSecsVal: Integer;
    EventCountVal: Integer;
    FirstEventAtVal: DateTime;
    LastEventAtVal: DateTime)
var
    DistRec: Record "ETL BC Distance";
begin
    if DistRec.Get(SessionIdVal) then begin
        DistRec."Session Date" := SessionDateVal;
        DistRec."Total Distance Km" := TotalDistKmVal;
        DistRec."Total Distance Meters" := TotalDistMetersVal;
        DistRec."Max Speed Kmh" := MaxSpeedVal;
        DistRec."Avg Speed Kmh" := AvgSpeedVal;
        DistRec."Total Active Seconds" := ActiveSecsVal;
        DistRec."Event Count" := EventCountVal;
        DistRec."First Event At" := FirstEventAtVal;
        DistRec."Last Event At" := LastEventAtVal;
```



```

        DistRec."ETL Loaded At"      := CurrentDateTime();
        DistRec.Modify(true);
    end
    else begin
        DistRec."Session Id"        := SessionIdVal;
        DistRec."Session Date"      := SessionDateVal;
        DistRec."Total Distance Km" := TotalDistKmVal;
        DistRec."Total Distance Meters" := TotalDistMetersVal;
        DistRec."Max Speed Kmh"     := MaxSpeedVal;
        DistRec."Avg Speed Kmh"     := AvgSpeedVal;
        DistRec."Total Active Seconds" := ActiveSecsVal;
        DistRec."Event Count"       := EventCountVal;
        DistRec."First Event At"    := FirstEventAtVal;
        DistRec."Last Event At"     := LastEventAtVal;
        DistRec."ETL Loaded At"     := CurrentDateTime();
        DistRec.Insert(true);
    end;
end;

// =====
//  IMPORT SHIPMENT LINES
// =====
procedure UpsertShipmentLine(
    DocumentNoVal: Code[20];
    LineNoVal: Integer;
    ArticleIdVal: Code[50];
    QuantityVal: Decimal;
    LocationCodeVal: Code[10])
var
    LineRec: Record "ETL BC Shipment Line";
begin
    if LineRec.Get(DocumentNoVal, LineNoVal) then begin
        LineRec."Article Id"      := ArticleIdVal;
        LineRec."Quantity"        := QuantityVal;
        LineRec."Location Code"   := LocationCodeVal;
        LineRec."ETL Loaded At"   := CurrentDateTime();
        LineRec.Modify(true);
    end else begin
        LineRec."Document No"     := DocumentNoVal;
        LineRec."Line No"         := LineNoVal;
        LineRec."Article Id"      := ArticleIdVal;
        LineRec."Quantity"        := QuantityVal;
        LineRec."Location Code"   := LocationCodeVal;
        LineRec."ETL Loaded At"   := CurrentDateTime();
        LineRec.Insert(true);
    end;
end;

// =====
//  IMPORT ECOMMERCE LINES
// =====
procedure UpsertEcommerceLine(
    InvoiceNoVal: Code[20];
    LineNoVal: Integer;
    StockCodeVal: Code[50];
    DescriptionVal: Text[100];
    QuantityVal: Decimal;
    UnitPriceVal: Decimal;
    TotalAmountVal: Decimal;
    CustomerIdVal: Code[50];
    CountryVal: Code[10];
    InvoiceDateVal: Date)
var
    EcomRec: Record "ETL BC Ecommerce Line";
begin
    if EcomRec.Get(InvoiceNoVal, LineNoVal) then begin
        EcomRec."Stock Code"      := StockCodeVal;
        EcomRec."Description"     := DescriptionVal;
        EcomRec."Quantity"        := QuantityVal;
        EcomRec."Unit Price"      := UnitPriceVal;
        EcomRec."Total Amount"    := TotalAmountVal;
        EcomRec."Customer Id"     := CustomerIdVal;

```

```

        EcomRec."Country"      := CountryVal;
        EcomRec."Invoice Date" := InvoiceDateVal;
        EcomRec."ETL Loaded At" := CurrentDateTime();
        EcomRec.Modify(true);
    end else begin
        EcomRec."Invoice No"    := InvoiceNoVal;
        EcomRec."Line No"      := LineNoVal;
        EcomRec."Stock Code"    := StockCodeVal;
        EcomRec."Description"   := DescriptionVal;
        EcomRec."Quantity"     := QuantityVal;
        EcomRec."Unit Price"    := UnitPriceVal;
        EcomRec."Total Amount"  := TotalAmountVal;
        EcomRec."Customer Id"   := CustomerIdVal;
        EcomRec."Country"      := CountryVal;
        EcomRec."Invoice Date" := InvoiceDateVal;
        EcomRec."ETL Loaded At" := CurrentDateTime();
        EcomRec.Insert(true);
    end;
end;

// =====
//  IMPORT CUSTOMERS
// =====
procedure UpsertCustomer(
    CustomerIdVal: Code[50];
    NameVal: Text[100];
    CountryVal: Code[10];
    PostCodeVal: Code[20];
    LocationCodeVal: Code[20];
    CurrencyCodeVal: Code[10];
    SalespersonCodeVal: Code[20])
var
    CustRec: Record "ETL BC Customer";
begin
    if CustRec.Get(CustomerIdVal) then begin
        CustRec."Name"      := NameVal;
        CustRec."Country"   := CountryVal;
        CustRec."Post Code" := PostCodeVal;
        CustRec."Location Code" := LocationCodeVal;
        CustRec."Currency Code" := CurrencyCodeVal;
        CustRec."Salesperson Code" := SalespersonCodeVal;
        CustRec."ETL Loaded At" := CurrentDateTime();
        CustRec.Modify(true);
    end else begin
        CustRec."Customer Id" := CustomerIdVal;
        CustRec."Name"      := NameVal;
        CustRec."Country"   := CountryVal;
        CustRec."Post Code" := PostCodeVal;
        CustRec."Location Code" := LocationCodeVal;
        CustRec."Currency Code" := CurrencyCodeVal;
        CustRec."Salesperson Code" := SalespersonCodeVal;
        CustRec."ETL Loaded At" := CurrentDateTime();
        CustRec.Insert(true);
    end;
end;

// =====
//  IMPORT ARTICLES
// =====
procedure UpsertArticle(
    ArticleIdVal: Code[50];
    DescriptionVal: Text[100];
    UOMVal: Code[20];
    TypeVal: Text[50];
    ItemCategoryVal: Code[20];
    UnitPriceVal: Decimal;
    UnitCostVal: Decimal;
    CostingMethodVal: Text[50])
var
    ArtRec: Record "ETL BC Article";
begin

```

```

        if ArtRec.Get(ArticleIdVal) then begin
            ArtRec."Description" := DescriptionVal;
            ArtRec."Unit Of Measure" := UOMVal;
            ArtRec."Type" := TypeVal;
            ArtRec."Item Category" := ItemCategoryVal;
            ArtRec."Unit Price" := UnitPriceVal;
            ArtRec."Unit Cost" := UnitCostVal;
            ArtRec."Costing Method" := CostingMethodVal;
            ArtRec."ETL Loaded At" := CurrentDateTime();
            ArtRec.Modify(true);
        end
    else begin
        ArtRec."Article Id" := ArticleIdVal;
        ArtRec."Description" := DescriptionVal;
        ArtRec."Unit Of Measure" := UOMVal;
        ArtRec."Type" := TypeVal;
        ArtRec."Item Category" := ItemCategoryVal;
        ArtRec."Unit Price" := UnitPriceVal;
        ArtRec."Unit Cost" := UnitCostVal;
        ArtRec."Costing Method" := CostingMethodVal;
        ArtRec."ETL Loaded At" := CurrentDateTime();
        ArtRec.Insert(true);
    end;
end;

// =====
// MISE À JOUR TEMPS RÉEL – distance GPS depuis l'app mobile (via n8n)
// Appelé par : Page 50206 "ETL BC Update Distance API"
// =====
procedure UpsertDistanceRealtime(SessionIdVal: Text[50]; NewDistanceKm: Decimal)
var
    DistRec: Record "ETL BC Distance";
begin
    if NewDistanceKm < 0 then
        Error('Distance invalide : la valeur ne peut pas être négative.');

```


C.4 Migration SQL — schema_gold_updates.sql

Script SQL à exécuter sur la base data_warehouse_gold après ETL_Projet pour migrer le schéma vers la gestion des sessions GPS temps réel : suppression de l'ancienne dim_distances Google Maps, recréation avec le schéma sessions GPS, suppression de la FK dim_date de fact_shipment_lines.

```
-- =====
-- ETL_Projet_BC – Migration SQL
-- À exécuter sur la base data_warehouse_gold APRÈS ETL_Projet
-- pour ajouter le support des sessions GPS temps réel
-- =====

-- 1. Supprimer l'ancienne dim_distances (Google Distance Matrix)
DROP TABLE IF EXISTS dim_distances CASCADE;

-- 2. Créer silver.distance_matrix avec le nouveau schéma GPS
CREATE SCHEMA IF NOT EXISTS silver;
DROP TABLE IF EXISTS silver.distance_matrix;
CREATE TABLE silver.distance_matrix (
    session_id          VARCHAR(50),
    session_date         VARCHAR(10),
    total_distance_km    VARCHAR(30),
    total_distance_meters VARCHAR(30),
    max_speed_kmh        VARCHAR(30),
    avg_speed_kmh        VARCHAR(30),
    total_active_seconds VARCHAR(20),
    event_count          VARCHAR(20),
    first_event_at       VARCHAR(50),
    last_event_at        VARCHAR(50)
);

-- 3. Recréer dim_distances avec le schéma sessions GPS
-- session_id = clé naturelle (ID de session envoyé par l'application)
CREATE TABLE dim_distances (
    session_id          VARCHAR(50) PRIMARY KEY,
    session_date         DATE,
    total_distance_km    DECIMAL(10,4),
    total_distance_meters DECIMAL(10,2),
    max_speed_kmh        DECIMAL(10,3),
    avg_speed_kmh        DECIMAL(10,3),
    total_active_seconds INTEGER,
    event_count          INTEGER,
    first_event_at       TIMESTAMP,
    last_event_at        TIMESTAMP,
    extracted_at         TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 4. Supprimer la FK dim_date de fact_shipment_lines (dim_date supprimée)
-- Si la contrainte existe encore (projet ETL_Projet déjà appliqué)
ALTER TABLE fact_shipment_lines
    DROP CONSTRAINT IF EXISTS fact_shipment_lines_shipment_date_fkey;

-- 5. Vérification
SELECT
    'dim_distances' AS table_name,
    column_name,
    data_type
FROM information_schema.columns
WHERE table_schema = 'public'
    AND table_name = 'dim_distances'
ORDER BY ordinal_position;
```

C.5 Python — import_workflows.py

Script Python d'import et de mise à jour des workflows n8n via l'API REST. Utilisé lors du déploiement initial ou d'une reconstruction de la base SQLite n8n. Lit les fichiers JSON du répertoire n8n/workflows/ et les pousse vers n8n via POST (création) ou PUT (mise à jour), puis active chaque workflow.

```
#!/usr/bin/env python3
"""Import des workflows ETL_Projet_BC dans n8n via l'API REST."""

import json
import urllib.request
import urllib.error
import os
import sys

API_KEY = 'n8n_api_39cf9ddc772145ee828f396e51fdeabe'
BASE_URL = 'http://localhost:5678/api/v1'
HEADERS = {
    'X-N8N-API-KEY': API_KEY,
    'Content-Type': 'application/json',
    'Accept': 'application/json',
}

WF_DIR = '/home/nyanu/Documents/ETL_Projet_BC/n8n/workflows'

# Fichiers à importer (ordre logique)
FILES = [
    '04b_distance_webhook.json',
    '08_gold_to_bc.json',
    '08a_sync_customers.json',
    '08b_sync_articles.json',
    '08c_sync_headers.json',
    '08d_sync_distances.json',
    '08e_sync_shipment_lines.json',
    '08f_sync_ecommerce_lines.json',
]

def api_get(path):
    req = urllib.request.Request(f'{BASE_URL}{path}', headers=HEADERS)
    with urllib.request.urlopen(req) as r:
        return json.loads(r.read())

def api_post(path, body):
    data = json.dumps(body).encode()
    req = urllib.request.Request(f'{BASE_URL}{path}', data=data, headers=HEADERS, method='POST')
    try:
        with urllib.request.urlopen(req) as r:
            return r.status, json.loads(r.read())
    except urllib.error.HTTPError as e:
        return e.code, json.loads(e.read())

def api_put(path, body):
    data = json.dumps(body).encode()
    req = urllib.request.Request(f'{BASE_URL}{path}', data=data, headers=HEADERS, method='PUT')
    try:
        with urllib.request.urlopen(req) as r:
            return r.status, json.loads(r.read())
    except urllib.error.HTTPError as e:
        return e.code, json.loads(e.read())

def api_patch(path, body):
    data = json.dumps(body).encode()
    req = urllib.request.Request(f'{BASE_URL}{path}', data=data, headers=HEADERS, method='PATCH')
    try:
        with urllib.request.urlopen(req) as r:
            return r.status, json.loads(r.read())
    except urllib.error.HTTPError as e:
```

```

        return e.code, json.loads(e.read())

# Récupérer les workflows existants (pour éviter les doublons)
existing = {}
try:
    result = api_get('/workflows?limit=50')
    for wf in result.get('data', []):
        existing[wf['name']] = wf['id']
    print(f'Workflows existants dans n8n : {len(existing)}')
    for name, wfid in existing.items():
        print(f' [{wfid}] {name}')
except Exception as e:
    print(f'Erreur récupération workflows existants : {e}')
    sys.exit(1)

print()
results = []

for filename in FILES:
    filepath = os.path.join(WF_DIR, filename)
    with open(filepath, 'r', encoding='utf-8') as f:
        wf = json.load(f)

    name = wf['name']
    wf_id = wf.get('id')

    # Préparer le payload pour l'API n8n
    # L'API n8n attend : name, nodes, connections, settings, staticData
    payload = {
        'name': wf['name'],
        'nodes': wf['nodes'],
        'connections': wf['connections'],
        'settings': wf.get('settings', {'executionOrder': 'v1'}),
        'staticData': wf.get('staticData', None),
    }
    # Les tags sont read-only via l'API n8n – ne pas les inclure dans le payload

    if name in existing:
        # Le workflow existe déjà → on le met à jour (PUT)
        existing_id = existing[name]
        print(f'⚡ MISE À JOUR [{existing_id}] {name}')
        status, resp = api_put(f'/workflows/{existing_id}', payload)
        if status in (200, 201):
            print(f'✅ Mis à jour (HTTP {status})')
            # Réactiver si nécessaire
            if wf.get('active', True):
                api_patch(f'/workflows/{existing_id}/activate', {})
                results.append({'file': filename, 'action': 'updated', 'id': existing_id, 'status': status})
            else:
                print(f'✖ Erreur HTTP {status} : {str(resp)[:200]}')
                results.append({'file': filename, 'action': 'error', 'id': existing_id, 'status': status,
'detail': str(resp)[:200]})
        else:
            # Nouveau workflow → POST
            print(f'➕ CRÉATION {name}')
            status, resp = api_post('/workflows', payload)
            if status in (200, 201):
                new_id = resp.get('id', '?')
                print(f'✅ Créé avec ID [{new_id}] (HTTP {status})')
                # Activer le workflow
                if wf.get('active', True):
                    act_status, _ = api_patch(f'/workflows/{new_id}/activate', {})
                    print(f'🟡 Activation : HTTP {act_status}')
                    results.append({'file': filename, 'action': 'created', 'id': new_id, 'status': status})
                else:
                    print(f'✖ Erreur HTTP {status} : {str(resp)[:300]}')
                    results.append({'file': filename, 'action': 'error', 'status': status, 'detail': str(resp)
[:300]})
            else:
                print(f'✖ Erreur HTTP {status} : {str(resp)[:300]}')
                results.append({'file': filename, 'action': 'error', 'status': status, 'detail': str(resp)
[:300]})

print()
print('=' * 60)

```

```
created = [r for r in results if r['action'] == 'created']
updated = [r for r in results if r['action'] == 'updated']
errors = [r for r in results if r['action'] == 'error']
print(f'RÉSULTAT : {len(created)} créés | {len(updated)} mis à jour | {len(errors)} erreurs')
if errors:
    print('ERREURS :')
    for e in errors:
        print(f' {e["file"]} → HTTP {e["status"]} : {e.get("detail","")}')
```


C.6 n8n JavaScript — WF-01BC : data.csv + data.json → MinIO Bronze

WF-01BC lit le fichier data.csv depuis MinIO Bronze, fusionne les données CSV et JSON en un fichier data_merged.json unifié, et écrit le résultat dans le staging MinIO Bronze pour la suite du pipeline.

Noeud : Lire data.csv

```
const fs = require('fs');
const SRC_CSV = '/data_sources/data.csv';
if (!fs.existsSync(SRC_CSV)) throw new Error('[WF-BC-01] Introuvable : ' + SRC_CSV);
const csvBuf = fs.readFileSync(SRC_CSV);
console.log('[WF-BC-01] data.csv lu : ' + csvBuf.length + ' octets');
return [{
  json: { fileName: 'data.csv', fileSize: csvBuf.length },
  binary: { csvData: { data: csvBuf.toString('base64'), mimeType: 'text/csv', fileName: 'data.csv' } }
}];
```

Noeud : Merger CSV + JSON → data_merged.json

```
const fs = require('fs');

const SRC_JSON = '/data_sources/data.json';
if (!fs.existsSync(SRC_JSON)) throw new Error('[WF-BC-01] Introuvable : ' + SRC_JSON);

const csvItem = $input.first();
const csvBuf = Buffer.from(csvItem.binary.csvData.data, 'base64');

function parseCsvRobust(raw) {
  const text = raw.toString('utf8');
  const rows = []; let pos = 0, headers = null;
  function parseField() {
    if (text[pos] === '"') {
      pos++; let val = '';
      while (pos < text.length) {
        if (text[pos] === '"' && text[pos+1] === '"') { val += '"'; pos += 2; }
        else if (text[pos] === '"') { pos++; break; }
        else { val += text[pos++]; }
      }
      return val;
    }
    let val = '';
    while (pos < text.length && text[pos] !== ',' && text[pos] !== '\n' && text[pos] !== '\r') val += text[pos++];
    return val.trim();
  }
  function parseLine() {
    const fields = [];
    while (pos < text.length && text[pos] !== '\n' && text[pos] !== '\r') {
      fields.push(parseField());
      if (pos < text.length && text[pos] === ',') pos++;
    }
    while (pos < text.length && (text[pos] === '\n' || text[pos] === '\r')) pos++;
    return fields;
  }
  headers = parseLine();
  while (pos < text.length) {
    const fields = parseLine();
    if (fields.length === headers.length && fields.some(f => f !== '')) {
      const obj = {}; headers.forEach((h, i) => { obj[h] = fields[i] || ''; }); rows.push(obj);
    }
  }
  return rows;
}
```

```

const csvRows = parseCsvRobust(csvBuf);
const jsonRows = JSON.parse(fs.readFileSync(SRC_JSON, 'utf8'));
const merged = [...jsonRows, ...csvRows];
const mergedBuf = Buffer.from(JSON.stringify(merged), 'utf8');

fs.mkdirSync('/staging/bc/flat_files', { recursive: true });
fs.writeFileSync('/staging/bc/flat_files/data_merged.json', mergedBuf);

console.log('[WF-BC-01] JSON:' + jsonRows.length + ' | CSV:' + csvRows.length + ' | total:' + merged.length);

return [{
  json: { fileName: 'data_merged.json', fileSize: mergedBuf.length,
    csvRows: csvRows.length, jsonRows: jsonRows.length, total: merged.length },
  binary: { data: { data: mergedBuf.toString('base64'), mimeType: 'application/json', fileName: 'data_merged.json' } }
}];

```

Noeud : Chargement terminé

```

const r = $input.first().json;
console.log('[WF-BC-01]  data_merged.json → MinIO + staging OK');
return [{ json: { status:'success', file:'data_merged.json', total: r.total || 0 } }];

```

C.7 n8n JavaScript — WF-03BC : PostgreSQL Windows → MinIO Bronze

WF-03BC extrait les données de la base PostgreSQL Windows (10.14.219.2:5433) pour les 4 entités métier : clients, articles, en-têtes d'expédition et lignes d'expédition. Chaque entité est sérialisée en JSON, déposée dans MinIO Bronze et sauvegardée dans le staging local (/staging/bc/database/).

Le nœud de chaque table implémente une stratégie de résilience à 3 niveaux :

- Niveau 1 — Staging local : si le fichier /staging/bc/database/<table>.json existe déjà, il est utilisé directement (pas de connexion réseau requise, priorité maximale).
- Niveau 2 — PostgreSQL externe (10.14.219.2:5433) : connexion avec timeout 5 000 ms. Utilisé uniquement quand le staging est absent et que le réseau école est accessible.
- Niveau 3 — MinIO Docker interne (172.18.0.3:9000) : lecture via requête HTTP signée AWS Signature V4 (modules natifs http + crypto, sans npm externe). Activé si PG est inaccessible ; restaure aussi le staging local pour les prochaines exécutions.

Cette architecture garantit le fonctionnement complet du pipeline même hors réseau école, tant que les données ont été chargées au moins une fois dans MinIO (par le staging ou une exécution précédente).

Noeud : Requête PG — clients

```
const { Client } = require('pg');
const fs = require('fs');
const http = require('http');
const crypto = require('crypto');

const TABLE = 'clients';
const STAGING_FILE = '/staging/bc/database/clients.json';
const MINIO_KEY = 'etl-projet-bc/raw/database/clients/latest/clients.json';
const MINIO_HOST = '172.18.0.3';
const MINIO_PORT = 9000;
const MINIO_BUCKET = 'bronze';
const MINIO_USER = 'nyanu';
const MINIO_PASS = 'nyanu1234';

// Lecture MinIO via AWS Signature V4 (http natif)
async function fetchMinIO(key) {
  const now = new Date();
  const ts = now.toISOString().replace(/[:-\]|\.\.{3}/g, '').slice(0,15)+'Z';
  const ds = ts.slice(0,8);
  const hdr = `host:${MINIO_HOST}:${MINIO_PORT}\n\nx-amz-date:${ts}\n`;
  const ph = crypto.createHash('sha256').update('').digest('hex');
  const cr = `GET\n/${MINIO_BUCKET}/${key}\n\n${hdr}\nhost;x-amz-date\n${ph}`;
  const cs = `${ds}/us-east-1/s3/aws4-request`;
  const sts = `AWS4-HMAC-SHA256\n${ts}\n${cs}\n${crypto.createHash('sha256').update(cr).digest('hex')}`;
  const hmac = (k,d) => crypto.createHmac('sha256',k).update(d).digest();
  const sk = hmac(hmac(hmac(hmac(`AWS4${MINIO_PASS}`,ds),`us-east-1`),`s3`),`aws4-request`);
  const sig = crypto.createHmac('sha256',sk).update(sts).digest('hex');
  const auth = `AWS4-HMAC-SHA256 Credential=${MINIO_USER}/${cs}, SignedHeaders=host;x-amz-date, Signature=${sig}`;
  return new Promise((res,rej) => {
    const req = http.request({
      hostname:MINIO_HOST, port:MINIO_PORT,
      path:`/${MINIO_BUCKET}/${key}`, method:'GET',
```

```

    headers: {'x-amz-date': ts, 'Authorization': auth}
  }, r => { let d=''; r.on('data', c=>d+=c); r.on('end', ()=>r.statusCode===200?res(d):rej(new Error(`MinIO
${r.statusCode}`))); }); });
  req.on('error', rej); req.end();
});
}

// Niveau 1 – staging local (plus rapide, toujours prioritaire)
if (fs.existsSync(STAGING_FILE)) {
  const buf = fs.readFileSync(STAGING_FILE);
  const data = JSON.parse(buf.toString('utf8'));
  const rows = data.data || [];
  console.log(`[WF-BC-03] ✅ ${TABLE} : staging local – ${rows.length} lignes`);
  return [{ json: { table: TABLE, rowCount: rows.length, key: MINIO_KEY, source: 'staging' },
    binary: { tableData: { data: buf.toString('base64'), mimeType: 'application/json', fileName: `${TABLE}.json` } } }];
}

// Niveau 2 – PostgreSQL externe (réseau école)
const pg = new Client({ host: '10.14.219.2', port: 5433, database: 'database_extern_local',
  user: 'postgres', password: 'postgres', connectionTimeoutMillis: 5000, statement_timeout: 30000 });
try {
  await pg.connect();
  const res = await pg.query(`SELECT row_to_json(t) AS r FROM "${TABLE}" t`);
  await pg.end();
  const rows = res.rows.map(r => r.r);
  const buf = Buffer.from(JSON.stringify({ data: rows }, 'utf8'));
  console.log(`[WF-BC-03] ✅ ${TABLE} : PostgreSQL externe – ${rows.length} lignes`);
  return [{ json: { table: TABLE, rowCount: rows.length, key: MINIO_KEY, source: 'pg' },
    binary: { tableData: { data: buf.toString('base64'), mimeType: 'application/json', fileName: `${TABLE}.json` } } }];
} catch (e) {
  try { await pg.end(); } catch (_) {}
  console.log(`[WF-BC-03] ⚠️ PostgreSQL inaccessible (${e.message}) – fallback MinIO`);
}

// Niveau 3 – MinIO Docker interne (toujours disponible dans le stack)
try {
  const raw = await fetchMinIO(MINIO_KEY);
  const data = JSON.parse(raw);
  const rows = data.data || [];
  const buf = Buffer.from(raw, 'utf8');
  fs.mkdirSync('/staging/bc/database', { recursive: true });
  fs.writeFileSync(STAGING_FILE, buf);
  console.log(`[WF-BC-03] ✅ ${TABLE} : MinIO fallback – ${rows.length} lignes (staging restauré)`);
  return [{ json: { table: TABLE, rowCount: rows.length, key: MINIO_KEY, source: 'minio' },
    binary: { tableData: { data: buf.toString('base64'), mimeType: 'application/json', fileName: `${TABLE}.json` } } }];
} catch (e2) {
  throw new Error(`[WF-BC-03] Toutes sources indisponibles pour ${TABLE} : staging absent, PG inaccessible, MinIO: ${e2.message}`);
}

```

Noeud : Stage — clients

```

const fs = require('fs');
const pgItem = $('Requête PG – clients').first();
const buf = Buffer.from(pgItem.binary.tableData.data, 'base64');
fs.mkdirSync('/staging/bc/database', { recursive: true });
fs.writeFileSync('/staging/bc/database/clients.json', buf);
const rows = $input.first().json.rowCount ?? pgItem.json.rowCount;
console.log(`[WF-BC-03] ✅ clients stage : ' + rows + ' lignes`);
return [{ json: { status: 'success', table: 'clients', rows } }];

```

Noeud : Requête PG — articles

```

const { Client } = require('pg');
const fs = require('fs');
const http = require('http');
const crypto = require('crypto');

const TABLE = 'articles';
const STAGING_FILE = '/staging/bc/database/articles.json';
const MINIO_KEY = 'etl-projet-bc/raw/database/articles/latest/articles.json';
const MINIO_HOST = '172.18.0.3';
const MINIO_PORT = 9000;
const MINIO_BUCKET = 'bronze';
const MINIO_USER = 'nyanu';
const MINIO_PASS = 'nyanu1234';

// Lecture MinIO via AWS Signature V4 (http natif)
async function fetchMinIO(key) {
  const now = new Date();
  const ts = now.toISOString().replace(/[:]|\\.|{}/g, '').slice(0,15)+'Z';
  const ds = ts.slice(0,8);
  const hdr = `host:${MINIO_HOST}:${MINIO_PORT}\n\nx-amz-date:${ts}\n`;
  const ph = crypto.createHash('sha256').update('').digest('hex');
  const cr = `GET\n/${MINIO_BUCKET}/${key}\n\n${hdr}\n\nhost;x-amz-date\n${ph}`;
  const cs = `${ds}/us-east-1/s3/aws4_request`;
  const sts = `AWS4-HMAC-SHA256\n${ts}\n${cs}\n${crypto.createHash('sha256').update(cr).digest('hex')}`;
  const hmac = (k,d) => crypto.createHmac('sha256',k).update(d).digest();
  const sk = hmac(hmac(hmac(hmac(`${MINIO_PASS}`,ds), 'us-east-1'), 's3'), 'aws4_request');
  const sig = crypto.createHmac('sha256',sk).update(sts).digest('hex');
  const auth = `AWS4-HMAC-SHA256 Credential=${MINIO_USER}/${cs}, SignedHeaders=host;x-amz-date, Signature=${sig}`;
  return new Promise((res,rej) => {
    const req = http.request({
      hostname:MINIO_HOST, port:MINIO_PORT,
      path:`/${MINIO_BUCKET}/${key}`, method:'GET',
      headers:{'x-amz-date':ts,'Authorization':auth}
    }, r => { let d=''; r.on('data',c=>d+=c); r.on('end',()=>r.statusCode===200?res(d):rej(new Error(`MinIO ${r.statusCode}`))}); });
    req.on('error',rej); req.end();
  });
}

// Niveau 1 – staging local (plus rapide, toujours prioritaire)
if (fs.existsSync(STAGING_FILE)) {
  const buf = fs.readFileSync(STAGING_FILE);
  const data = JSON.parse(buf.toString('utf8'));
  const rows = data.data || [];
  console.log(`[WF-BC-03] ✅ ${TABLE} : staging local – ${rows.length} lignes`);
  return [{ json:{ table:TABLE, rowCount:rows.length, key:MINIO_KEY, source:'staging' },
    binary:{ tableData:{ data:buf.toString('base64'), mimeType:'application/json', fileName:`$
{TABLE}.json` } } }];
}

// Niveau 2 – PostgreSQL externe (réseau école)
const pg = new Client({ host:'10.14.219.2', port:5433, database:'database_extern_local',
  user:'postgres', password:'postgres', connectionTimeoutMillis:5000, statement_timeout:30000 });
try {
  await pg.connect();
  const res = await pg.query(`SELECT row_to_json(t) AS r FROM "${TABLE}" t`);
  await pg.end();
  const rows = res.rows.map(r => r.r);
  const buf = Buffer.from(JSON.stringify({ data: rows }), 'utf8');
  console.log(`[WF-BC-03] ✅ ${TABLE} : PostgreSQL externe – ${rows.length} lignes`);
  return [{ json:{ table:TABLE, rowCount:rows.length, key:MINIO_KEY, source:'pg' },
    binary:{ tableData:{ data:buf.toString('base64'), mimeType:'application/json', fileName:`$
{TABLE}.json` } } }];
} catch(e) {
  try { await pg.end(); } catch(_) {}
  console.log(`[WF-BC-03] ⚠️ PostgreSQL inaccessible (${e.message}) – fallback MinIO`);
}

// Niveau 3 – MinIO Docker interne (toujours disponible dans le stack)
try {

```

```

const raw = await fetchMinIO(MINIO_KEY);
const data = JSON.parse(raw);
const rows = data.data || [];
const buf = Buffer.from(raw, 'utf8');
fs.mkdirSync('/staging/bc/database', { recursive:true });
fs.writeFileSync(STAGING_FILE, buf);
console.log(`[WF-BC-03] ✅ ${TABLE} : MinIO fallback – ${rows.length} lignes (staging restauré)`);
return [{ json:{ table:TABLE, rowCount:rows.length, key:MINIO_KEY, source:'minio' },
  binary:{ tableData:{ data:buf.toString('base64'), mimeType:'application/json', fileName:`$
{TABLE}.json` } } }];
} catch(e2) {
  throw new Error(`[WF-BC-03] Toutes sources indisponibles pour ${TABLE} : staging absent, PG inaccessible,
MinIO: ${e2.message}`);
}

```

Noeud : Stage — articles

```

const fs = require('fs');
const pgItem = $('Requête PG – articles').first();
const buf = Buffer.from(pgItem.binary.tableData.data, 'base64');
fs.mkdirSync('/staging/bc/database', { recursive: true });
fs.writeFileSync('/staging/bc/database/articles.json', buf);
const rows = $input.first().json.rowCount ?? pgItem.json.rowCount;
console.log(`[WF-BC-03] ✅ articles stagé : ' + rows + ' lignes`);
return [{ json: { status: 'success', table: 'articles', rows } }];

```

Noeud : Requête PG — salessshipmentheader

```

const { Client } = require('pg');
const fs = require('fs');
const http = require('http');
const crypto = require('crypto');

const TABLE = 'salessshipmentheader';
const STAGING_FILE = '/staging/bc/database/salessshipmentheader.json';
const MINIO_KEY = 'etl-projet-bc/raw/database/salessshipmentheader/latest/salessshipmentheader.json';
const MINIO_HOST = '172.18.0.3';
const MINIO_PORT = 9000;
const MINIO_BUCKET = 'bronze';
const MINIO_USER = 'nyanu';
const MINIO_PASS = 'nyanu1234';

// Lecture MinIO via AWS Signature V4 (http natif)
async function fetchMinIO(key) {
  const now = new Date();
  const ts = now.toISOString().replace(/[:-\]\.\{\}/g, '').slice(0,15)+'Z';
  const ds = ts.slice(0,8);
  const hdr = `host:${MINIO_HOST}:${MINIO_PORT}\n${x-amz-date:${ts}}\n`;
  const ph = crypto.createHash('sha256').update('').digest('hex');
  const cr = `GET\n/${MINIO_BUCKET}/${key}\n\n${hdr}\nhost;x-amz-date\n${ph}`;
  const cs = `${ds}/us-east-1/s3/aws4-request`;
  const sts = `AWS4-HMAC-SHA256\n${ts}\n${cs}\n${crypto.createHash('sha256').update(cr).digest('hex')}`;
  const hmac = (k,d) => crypto.createHmac('sha256',k).update(d).digest();
  const sk = hmac(hmac(hmac(hmac(`AWS4${MINIO_PASS}`,ds), 'us-east-1'), 's3'), 'aws4_request');
  const sig = crypto.createHmac('sha256',sk).update(sts).digest('hex');
  const auth = `AWS4-HMAC-SHA256 Credential=${MINIO_USER}/${cs}, SignedHeaders=host;x-amz-date, Signature=${sig}`;
  return new Promise((res,rej) => {
    const req = http.request({
      hostname:MINIO_HOST, port:MINIO_PORT,
      path: `/${MINIO_BUCKET}/${key}`, method:'GET',
      headers:{'x-amz-date':ts, 'Authorization':auth}
    }, r => { let d=''; r.on('data',c=>d+=c); r.on('end',()=>r.statusCode===200?res(d):rej(new Error(`MinIO
${r.statusCode}`))}); });
    req.on('error',rej); req.end();
  });
}

```

```

});
}

// Niveau 1 – staging local (plus rapide, toujours prioritaire)
if (fs.existsSync(STAGING_FILE)) {
  const buf = fs.readFileSync(STAGING_FILE);
  const data = JSON.parse(buf.toString('utf8'));
  const rows = data.data || [];
  console.log(`[WF-BC-03] ✅ ${TABLE} : staging local – ${rows.length} lignes`);
  return [{ json:{ table:TABLE, rowCount:rows.length, key:MINIO_KEY, source:'staging' },
    binary:{ tableData:{ data:buf.toString('base64'), mimeType:'application/json', fileName:`$
{TABLE}.json` } } }]];
}

// Niveau 2 – PostgreSQL externe (réseau école)
const pg = new Client({ host:'10.14.219.2', port:5433, database:'database_extern_local',
  user:'postgres', password:'postgres', connectionTimeoutMillis:5000, statement_timeout:30000 });
try {
  await pg.connect();
  const res = await pg.query(`SELECT row_to_json(t) AS r FROM "${TABLE}" t`);
  await pg.end();
  const rows = res.rows.map(r => r.r);
  const buf = Buffer.from(JSON.stringify({ data: rows }), 'utf8');
  console.log(`[WF-BC-03] ✅ ${TABLE} : PostgreSQL externe – ${rows.length} lignes`);
  return [{ json:{ table:TABLE, rowCount:rows.length, key:MINIO_KEY, source:'pg' },
    binary:{ tableData:{ data:buf.toString('base64'), mimeType:'application/json', fileName:`$
{TABLE}.json` } } }]];
} catch(e) {
  try { await pg.end(); } catch(_) {}
  console.log(`[WF-BC-03] ⚠️ PostgreSQL inaccessible (${e.message}) – fallback MinIO`);
}

// Niveau 3 – MinIO Docker interne (toujours disponible dans le stack)
try {
  const raw = await fetchMinIO(MINIO_KEY);
  const data = JSON.parse(raw);
  const rows = data.data || [];
  const buf = Buffer.from(raw, 'utf8');
  fs.mkdirSync('/staging/bc/database', { recursive:true });
  fs.writeFileSync(STAGING_FILE, buf);
  console.log(`[WF-BC-03] ✅ ${TABLE} : MinIO fallback – ${rows.length} lignes (staging restauré)`);
  return [{ json:{ table:TABLE, rowCount:rows.length, key:MINIO_KEY, source:'minio' },
    binary:{ tableData:{ data:buf.toString('base64'), mimeType:'application/json', fileName:`$
{TABLE}.json` } } }]];
} catch(e2) {
  throw new Error(`[WF-BC-03] Toutes sources indisponibles pour ${TABLE} : staging absent, PG inaccessible,
MinIO: ${e2.message}`);
}

```

Noeud : Stage — salesshipmentheader

```

const fs = require('fs');
const pgItem = $('Requête PG – salesshipmentheader').first();
const buf = Buffer.from(pgItem.binary.tableData.data, 'base64');
fs.mkdirSync('/staging/bc/database', { recursive: true });
fs.writeFileSync('/staging/bc/database/salesshipmentheader.json', buf);
const rows = $input.first().json.rowCount ?? pgItem.json.rowCount;
console.log(`[WF-BC-03] ✅ salesshipmentheader stagé : ' + rows + ' lignes`);
return [{ json: { status: 'success', table: 'salesshipmentheader', rows } }]];

```

Noeud : Requête PG — salesshipmentline

```

const { Client } = require('pg');
const fs = require('fs');
const http = require('http');

```

```

const crypto = require('crypto');

const TABLE      = 'salessshipmentline';
const STAGING_FILE = '/staging/bc/database/salessshipmentline.json';
const MINIO_KEY    = 'etl-projet-bc/raw/database/salessshipmentline/latest/salessshipmentline.json';
const MINIO_HOST   = '172.18.0.3';
const MINIO_PORT   = 9000;
const MINIO_BUCKET = 'bronze';
const MINIO_USER   = 'nyanu';
const MINIO_PASS   = 'nyanu1234';

// Lecture MinIO via AWS Signature V4 (http natif)
async function fetchMinIO(key) {
  const now = new Date();
  const ts = now.toISOString().replace(/[:]|\\.|{3}/g, '').slice(0,15)+'Z';
  const ds = ts.slice(0,8);
  const hdr = `host:${MINIO_HOST}:${MINIO_PORT}\\n-x-amz-date:${ts}\\n`;
  const ph = crypto.createHash('sha256').update('').digest('hex');
  const cr = `GET\\n/${MINIO_BUCKET}/${key}\\n\\n${hdr}\\nhost;x-amz-date\\n${ph}`;
  const cs = `${ds}/us-east-1/s3/aws4-request`;
  const sts = `AWS4-HMAC-SHA256\\n${ts}\\n${cs}\\n${crypto.createHash('sha256').update(cr).digest('hex')}`;
  const hmac = (k,d) => crypto.createHmac('sha256',k).update(d).digest();
  const sk = hmac(hmac(hmac('AWS4${MINIO_PASS}',ds), 'us-east-1'), 's3'), 'aws4-request');
  const sig = crypto.createHmac('sha256',sk).update(sts).digest('hex');
  const auth = `AWS4-HMAC-SHA256 Credential=${MINIO_USER}/${cs}, SignedHeaders=host;x-amz-date, Signature=${sig}`;
  return new Promise((res,rej) => {
    const req = http.request({
      hostname:MINIO_HOST, port:MINIO_PORT,
      path:`/${MINIO_BUCKET}/${key}`, method:'GET',
      headers:{'x-amz-date':ts, 'Authorization':auth}
    }, r => { let d=''; r.on('data',c=>d+=c); r.on('end',())=>r.statusCode===200?res(d):rej(new Error(`MinIO
    ${r.statusCode}`)); });
    req.on('error',rej); req.end();
  });
}

// Niveau 1 – staging local (plus rapide, toujours prioritaire)
if (fs.existsSync(STAGING_FILE)) {
  const buf = fs.readFileSync(STAGING_FILE);
  const data = JSON.parse(buf.toString('utf8'));
  const rows = data.data || [];
  console.log(`[WF-BC-03] ✅ ${TABLE} : staging local – ${rows.length} lignes`);
  return [{ json:{ table:TABLE, rowCount:rows.length, key:MINIO_KEY, source:'staging' },
    binary:{ tableData:{ data:buf.toString('base64'), mimeType:'application/json', fileName:`$
    {TABLE}.json` } } }];
}

// Niveau 2 – PostgreSQL externe (réseau école)
const pg = new Client({ host:'10.14.219.2', port:5433, database:'database_extern_local',
  user:'postgres', password:'postgres', connectionTimeoutMillis:5000, statement_timeout:30000 });
try {
  await pg.connect();
  const res = await pg.query(`SELECT row_to_json(t) AS r FROM "${TABLE}" t`);
  await pg.end();
  const rows = res.rows.map(r => r.r);
  const buf = Buffer.from(JSON.stringify({ data: rows }), 'utf8');
  console.log(`[WF-BC-03] ✅ ${TABLE} : PostgreSQL externe – ${rows.length} lignes`);
  return [{ json:{ table:TABLE, rowCount:rows.length, key:MINIO_KEY, source:'pg' },
    binary:{ tableData:{ data:buf.toString('base64'), mimeType:'application/json', fileName:`$
    {TABLE}.json` } } }];
} catch(e) {
  try { await pg.end(); } catch(_) {}
  console.log(`[WF-BC-03] ⚠️ PostgreSQL inaccessible (${e.message}) – fallback MinIO`);
}

// Niveau 3 – MinIO Docker interne (toujours disponible dans le stack)
try {
  const raw = await fetchMinIO(MINIO_KEY);
  const data = JSON.parse(raw);
  const rows = data.data || [];

```



```

const buf = Buffer.from(raw, 'utf8');
fs.mkdirSync('/staging/bc/database', { recursive:true });
fs.writeFileSync(STAGING_FILE, buf);
console.log(`[WF-BC-03] ✅ ${TABLE} : MinIO fallback – ${rows.length} lignes (staging restauré)`);
return [{ json:{ table:TABLE, rowCount:rows.length, key:MINIO_KEY, source:'minio' },
            binary:{ tableData:{ data:buf.toString('base64'), mimeType:'application/json', fileName:`$
{TABLE}.json` } } }]];
} catch(e2) {
  throw new Error(`[WF-BC-03] Toutes sources indisponibles pour ${TABLE} : staging absent, PG inaccessible,
MinIO: ${e2.message}`);
}

```

Noeud : Stage — salessshipmentline

```

const fs = require('fs');
const pgItem = $('Requête PG – salessshipmentline').first();
const buf = Buffer.from(pgItem.binary.tableData.data, 'base64');
fs.mkdirSync('/staging/bc/database', { recursive: true });
fs.writeFileSync('/staging/bc/database/salessshipmentline.json', buf);
const rows = $input.first().json.rowCount ?? pgItem.json.rowCount;
console.log(`[WF-BC-03] ✅ salessshipmentline stagé : ' + rows + ' lignes`);
return [{ json: { status: 'success', table: 'salessshipmentline', rows } }]];

```

Noeud : Chargement terminé

```

console.log(`[WF-BC-03] ✅ Toutes les tables PostgreSQL chargées dans /staging/bc/database/`);
return [{ json: { status:'success', tables: ['clients','articles','salessshipmentheader','salessshipmentline']
} }]];

```

C.8 n8n JavaScript — WF-04B : Distance App → MinIO (Webhook temps réel)

WF-04B reçoit les données GPS en temps réel via webhook HTTPS sécurisé (X-API-KEY). Il accumule les événements par session_id, calcule les métriques (distance, vitesse, durée active), et écrit le fichier de staging dans MinIO Bronze avec un checksum CDC pour éviter les doublons. Il déclenche ensuite WF-08D pour synchroniser vers BC.

Noeud : Auth — Valider X-API-KEY

```
const data    = $input.first().json;
const headers = data.headers || {};

// Accepter les deux casses courantes du header
const provided = (headers['x-api-key'] || headers['X-API-KEY'] || '').trim();
const expected = ($env.GPS_WEBHOOK_SECRET || '').trim();

if (!expected) {
  // Variable non configurée → refus par sécurité
  console.warn('[WF-04B] GPS_WEBHOOK_SECRET non défini dans les variables n8n');
  return [{ json: { _authorized: false, _reason: 'server_misconfiguration' } }];
}

const authorized = provided !== '' && provided === expected;
if (!authorized) {
  console.warn('[WF-04B] ❌ Clé API invalide – rejet (provided length: ${provided.length})`);
}

// Passer tout le payload original en conservant le flag d'autorisation
return [{ json: { ...data, _authorized: authorized } }];
```

Noeud : Accumuler Session

```
const fs      = require('fs');
const crypto  = require('crypto');

const raw     = $input.first().json;
const data    = raw.body || raw;
const event   = Object.fromEntries(
  Object.entries(data).filter(([k]) => !k.startsWith('_'))
);

if (!event || ![ 'distance_update', 'session_end' ].includes(event.event) || !event.session_id) {
  console.warn('[WF-04B] Payload invalide: ${JSON.stringify(event).substring(0,200)}`);
  return [{ json: { _payloadError: 'Payload invalide – champs requis: event dans {distance_update,session_end} et session_id obligatoire' } }];
}
```

Nœuds ajoutés : Payload valide ? + Répondre 400 Bad Request

Deux nœuds ajoutés pour corriger le bug de timeout sur payload invalide : le nœud IF « Payload valide ? » vérifie si _payloadError est présent (payload KO → Répondre 400 Bad Request) ou absent (payload OK → Données modifiées ?). Le nœud respondToWebhook retourne HTTP 400 avec le message d'erreur, en remplacement de l'exception non gérée qui laissait le client en timeout.

```

const STAGING_FILE = '/staging/distances/distances_realtime.json';
const CHECKSUM_FILE = '/staging/_metadata/checksums/distances_realtime.json';
const MINIO_PREFIX = 'etl-projet-bc/raw/distances';
const MINIO_FILENAME = 'distances_realtime.json';

let accumulated = { metadata: { generated_at: new Date().toISOString(), count: 0 }, data: [] };
try {
  const existing = fs.readFileSync(STAGING_FILE, 'utf8');
  accumulated = JSON.parse(existing);
  if (!Array.isArray(accumulated.data)) accumulated.data = [];
} catch(e) {}

const sessionId = String(event.session_id);
const vehicleId = event.vehicle_id || event.vehicleId
  || (raw.query && (raw.query.vehicle_id || raw.query.vehicleId))
  || 'VH-001';

let session = accumulated.data.find(s => s.session_id === sessionId);

if (event.event === 'session_end') {
  // Totaux définitifs envoyés par l'app en fin de trajet – remplace les valeurs accumulées
  const pts = Array.isArray(event.location_points) ? event.location_points : [];
  const lastPt = pts.length > 0 ? pts[pts.length - 1] : null;
  if (!session) { session = { session_id: sessionId }; accumulated.data.push(session); }
  session.session_date = event.start_time ? event.start_time.substring(0, 10) : new
Date().toISOString().substring(0, 10);
  session.vehicle_id = session.vehicle_id || vehicleId;
  session.total_distance_km = parseFloat((event.total_distance_km || 0).toFixed(6));
  session.total_distance_meters = parseFloat((event.total_distance_meters || 0).toFixed(2));
  session.max_speed_kmh = parseFloat((event.max_speed_kmh || 0).toFixed(3));
  session.avg_speed_kmh = parseFloat((event.avg_speed_kmh || 0).toFixed(3));
  session.total_active_seconds = parseInt(event.active_seconds || 0);
  session.event_count = (session.event_count || 0) + 1;
  session.first_event_at = event.start_time || session.first_event_at || null;
  session.last_event_at = event.end_time || session.last_event_at || null;
  session.last_latitude = lastPt ? (lastPt.latitude || lastPt.lat || null) :
(session.last_latitude || null);
  session.last_longitude = lastPt ? (lastPt.longitude || lastPt.lon || null) :
(session.last_longitude || null);
  console.log(`[WF-04B] session_end ${sessionId}: ${session.total_distance_km.toFixed(4)} km | avg $
{session.avg_speed_kmh} km/h | ${session.total_active_seconds}s`);
} else {
  // distance_update : accumulation incrémentale pendant le trajet
  const loc = event.location || {};
  if (!session) {
    session = {
      session_id : sessionId,
      session_date : event.timestamp ? event.timestamp.substring(0, 10) : new
Date().toISOString().substring(0, 10),
      vehicle_id : vehicleId,
      total_distance_km : 0,
      total_distance_meters : 0,
      max_speed_kmh : 0,
      avg_speed_kmh : 0,
      total_active_seconds : 0,
      event_count : 0,
      first_event_at : event.timestamp,
      last_event_at : event.timestamp,
      last_latitude : loc.latitude || null,
      last_longitude : loc.longitude || null
    };
    accumulated.data.push(session);
  }
  if (vehicleId && !session.vehicle_id) session.vehicle_id = vehicleId;
  session.total_distance_km = parseFloat((session.total_distance_km + (event.distance_km ||
0)).toFixed(6));
  session.total_distance_meters = parseFloat((session.total_distance_meters + (event.distance_meters ||
0)).toFixed(2));
  const spd = event.speed_kmh || 0;
  if (spd > session.max_speed_kmh) session.max_speed_kmh = parseFloat(spd.toFixed(3));

```

```

    session.total_active_seconds += (event.active_seconds || 0);
    session.event_count += 1;
    if (session.total_active_seconds > 0)
        session.avg_speed_kmh = parseFloat(((session.total_distance_km / session.total_active_seconds) *
3600).toFixed(3));
    if (!session.first_event_at || event.timestamp < session.first_event_at) session.first_event_at =
event.timestamp;
    if (!session.last_event_at || event.timestamp > session.last_event_at) session.last_event_at =
event.timestamp;
    if (loc.latitude) session.last_latitude = loc.latitude;
    if (loc.longitude) session.last_longitude = loc.longitude;
    console.log(`[WF-04B] distance_update ${sessionId}: evt#${session.event_count} | $
{session.total_distance_km.toFixed(4)} km | véhicule: ${session.vehicle_id || 'N/A'}`);
}

accumulated.metadata.generated_at = new Date().toISOString();
accumulated.metadata.count = accumulated.data.length;

const payload = JSON.stringify(accumulated);
const buf = Buffer.from(payload, 'utf8');
const currentHash = crypto.createHash('md5').update(buf).digest('hex');

let storedHash = null;
try {
    const chk = JSON.parse(fs.readFileSync(CHECKSUM_FILE, 'utf8'));
    storedHash = chk.hash;
} catch(e) {}

const now = new Date();
const y = now.getFullYear();
const m = String(now.getMonth()+1).padStart(2, '0');
const d = String(now.getDate()).padStart(2, '0');
const hh = String(now.getHours()).padStart(2, '0');
const mm = String(now.getMinutes()).padStart(2, '0');
const ts = `${y}${m}${d}_${hh}${mm}`;

const versionedKey = `${MINIO_PREFIX}/${y}/${m}/${d}/${ts}/${MINIO_FILENAME}`;
const latestKey = `${MINIO_PREFIX}/latest/${MINIO_FILENAME}`;

const binAttach = { data: buf.toString('base64'), mimeType: 'application/json', fileName: MINIO_FILENAME };

return [{
    json: {
        currentHash, storedHash,
        hashChanged : currentHash !== storedHash,
        versionedKey, latestKey,
        timestamp : now.toISOString(),
        count : accumulated.data.length,
        sessionId,
        vehicleId : session.vehicle_id,
        eventCount : session.event_count,
        lastLatitude : session.last_latitude,
        lastLongitude : session.last_longitude
    },
    binary: { distData: binAttach }
}];

```

Noeud : Restaurer distData

```

const cdcOut = $('Accumuler Session').first();
const s3Out = $input.first();
if (!cdcOut.binary || !cdcOut.binary.distData) {
    throw new Error('[WF-04B] Binaire distData introuvable dans Accumuler Session');
}
return [{ json: { ...cdcOut.json, ...s3Out.json }, binary: cdcOut.binary }];

```

Noeud : Stage+Checksum — distances

```
const fs = require('fs');

const cdcOut = $('Accumuler Session').first();
if (!cdcOut.binary || !cdcOut.binary.distData) {
  throw new Error('[WF-04B] Binaire distData introuvable');
}
const buf = Buffer.from(cdcOut.binary.distData.data, 'base64');

fs.mkdirSync('/staging/distances', { recursive: true });
fs.writeFileSync('/staging/distances/distances_realtime.json', buf);

const dir = '/staging/_metadata/checksums';
fs.mkdirSync(dir, { recursive: true });
fs.writeFileSync(`${dir}/distances_realtime.json`,
  JSON.stringify({
    hash      : cdcOut.json.currentHash,
    updatedAt : new Date().toISOString(),
    count     : cdcOut.json.count,
    versionedKey: cdcOut.json.versionedKey
  }, null, 2)
);

console.log(`[WF-04B] distances_realtime.json stagé (${cdcOut.json.count} sessions) → $
{cdcOut.json.versionedKey}`);
return [{ json: {
  status      : 'success',
  count       : cdcOut.json.count,
  versionedKey : cdcOut.json.versionedKey,
  sessionId   : cdcOut.json.sessionId,
  vehicleId   : cdcOut.json.vehicleId,
  lastLatitude : cdcOut.json.lastLatitude,
  lastLongitude : cdcOut.json.lastLongitude
} }];
```

Noeud : Skip — inchangé

```
const item = $input.first().json;
console.log(`[WF-04B] 🚫 Session ${item.sessionId} – données inchangées, pas d'upload`);
return [{ json: { status: 'skipped', sessionId: item.sessionId } }];
```

C.9 n8n JavaScript — WF-06 : gestion_logistique_vehicules.json → bc_gold

WF-06 lit le fichier gestion_logistique_vehicules.json depuis MinIO, applique un CDC par empreinte MD5 (stockée dans /staging/_metadata/checksums/), et met à jour la table bc_gold.vehicles uniquement si le fichier a été modifié depuis le dernier traitement.

Noeud : Lire JSON + CDC

```
const fs = require('fs');
const crypto = require('crypto');

const JSON_PATH = '/data_sources/gestion_logistique_vehicules.json';
const STAGING_PATH = '/staging/bc/vehicles/gestion_logistique_vehicules.json';
const CHECKSUM_PATH = '/staging/bc/_metadata/vehicles_checksum.md5';

if (!fs.existsSync(JSON_PATH)) throw new Error('[WF-06] Introuvable : ' + JSON_PATH);

const rawBuf = fs.readFileSync(JSON_PATH);
const rawContent = rawBuf.toString('utf8');
const newChecksum = crypto.createHash('md5').update(rawBuf).digest('hex');

let oldChecksum = '';
try { oldChecksum = fs.readFileSync(CHECKSUM_PATH, 'utf8').trim(); } catch(e) {}

const changed = newChecksum !== oldChecksum;
console.log(`[WF-06] Checksum: ${newChecksum} | Changed: ${changed}`);

const vehicles = JSON.parse(rawContent);
const today = new Date().toISOString().split('T')[0];

fs.mkdirSync('/staging/bc/vehicles', { recursive: true });
fs.writeFileSync(STAGING_PATH, rawBuf);

return [{
  json: { changed, newChecksum, rawContent, rowCount: vehicles.length, vehicles, today },
  binary: {
    data: {
      data: rawBuf.toString('base64'),
      mimeType: 'application/json',
      fileName: 'gestion_logistique_vehicules.json'
    }
  }
}];
```

Noeud : Upsert bc_gold.vehicles

```
const { Client } = require('pg');

const PG = {
  host: '172.18.0.4', port: 5432,
  database: 'data_warehouse_gold',
  user: 'nyanu', password: 'nyanu',
  connectionTimeoutMillis: 10000
};

const vehicles = $('Lire JSON + CDC').first().json.vehicles;
const pg = new Client(PG);
await pg.connect();

let upserted = 0;
try {
  for (const r of vehicles) {
```

```

    await pg.query(
      `INSERT INTO bc_gold.vehicles (vehicle_id, model, plate, cost_per_km, active, notes)
      VALUES ($1, $2, $3, $4, $5, $6)
      ON CONFLICT (vehicle_id) DO UPDATE SET
        model      = EXCLUDED.model,
        plate      = EXCLUDED.plate,
        cost_per_km = EXCLUDED.cost_per_km,
        active     = EXCLUDED.active,
        notes      = EXCLUDED.notes`,
      [
        r.vehicle_id || '|',
        r.model || '|',
        r.plate || '|',
        parseFloat(r.cost_per_km) || '|',
        r.active === true || r.active === 'true',
        r.notes || '|'
      ]
    );
    upserted++;
  }
  console.log(`[WF-06] ✅ ${upserted} véhicules upsertés dans bc_gold.vehicles`);
} finally {
  await pg.end();
}

return [{ json: { upserted, total: vehicles.length } }];

```

Noeud : Sauvegarder checksum

```

const fs = require('fs');
const CHECKSUM_PATH = '/staging/bc/_metadata/vehicles_checksum.md5';
const newChecksum = $('Lire JSON + CDC').first().json.newChecksum;
fs.mkdirSync('/staging/bc/_metadata', { recursive: true });
fs.writeFileSync(CHECKSUM_PATH, newChecksum, 'utf8');
console.log(`[WF-06] 🗒️ Checksum sauvegardé: ${newChecksum}`);
return $input.all();

```

Noeud : Résumé sync véhicules

```

const { rowCount, today, newChecksum } = $('Lire JSON + CDC').first().json;
const upserted = $('Upsert bc_gold.vehicles').first().json.upserted;
console.log(`[WF-06] ✅ Sync terminée: ${upserted}/${rowCount} véhicules | ${today}`);
return [{ json: {
  status: 'success',
  upserted,
  total: rowCount,
  date: today,
  checksum: newChecksum,
  message: `${upserted} véhicules → MinIO Bronze + bc_gold.vehicles`
} }];

```

C.10 n8n JavaScript — WF-08 : Orchestrateur Bronze → BC

WF-08 orchestre la synchronisation complète des 5 entités (clients, articles, en-têtes, lignes expédition, lignes e-commerce) depuis le staging Bronze vers BC. Il utilise un verrou fichier pour prévenir les exécutions simultanées, déclenche les sous-workflows 08A-08F en séquence et produit un résumé consolidé.

Noeud : Résumé sync BC

```
const nodes = [
  'Execute 08A – Customers',
  'Execute 08B – Articles',
  'Execute 08C – Shipment Headers',
  'Execute 08D – Distances',
  'Execute 08E – Shipment Lines',
  'Execute 08F – Ecommerce Lines',
];

const tables = [];
for (const nodeName of nodes) {
  try {
    const out = $(nodeName).first().json();
    if (out) tables.push(out);
  } catch(e) { /* nœud non exécuté */ }
}

const total_inserted = tables.reduce((s,t) => s + (t.inserted || 0), 0);
const total_updated = tables.reduce((s,t) => s + (t.updated || 0), 0);
const total_errors = tables.reduce((s,t) => s + (t.errors || 0), 0);
const status = total_errors === 0 ? 'success' : 'partial';

console.log(`[WF-08] ✅ ETL_Projet_BC → BC terminé: ${total_inserted} inserts | ${total_updated} updates | ${total_errors} errors`);
return [{ json: { status, tables, total_inserted, total_updated, total_errors } }];
```

Noeud : Sync déjà effectuée ?

```
const fs = require('fs');
const FLAG = '/staging/bc/_metadata/initial_sync_done.flag';
const alreadySynced = fs.existsSync(FLAG);
if (alreadySynced) {
  console.log(`[WF-08] ✅ Sync initiale déjà effectuée – données statiques inchangées`);
}
return [{ json: { alreadySynced } }];
```

Noeud : Marquer sync terminée

```
const fs = require('fs');
const FLAG = '/staging/bc/_metadata/initial_sync_done.flag';
fs.mkdirSync('/staging/bc/_metadata', { recursive: true });
fs.writeFileSync(FLAG, new Date().toISOString());
console.log(`[WF-08] 🚩 Flag sync initiale créé – prochains runs statiques ignorés`);
return $input.all();
```


C.11 n8n JavaScript — WF-08A à WF-08F : Sous-workflows Sync → BC

Chaque sous-workflow synchronise une entité métier vers BC via l'API OData en upsert (POST puis PATCH en cas de conflit 409/422). Ils sont appelés par WF-08 (orchestrateur) ou déclenchés de façon autonome.

WF-08A — Sync clients → BC — Noeud : Sync clients → BC

```
const fs = require('fs');
const http = require('http');

const STAGING_FILE = '/staging/bc/database/clients.json';
const PROXY = '172.18.0.8';
const PORT = 3128;
const BASE = '/BC260/api/etlpipeline/warehouse/v1.0';
const COMPANY = 'e6317465-9a3f-f011-be59-6045bde988e7';
const ENTITY = 'etlCustomers';
const NOW = new Date().toISOString();

function bcReq(method, path, body) {
  return new Promise((resolve, reject) => {
    const pl = body ? JSON.stringify(body) : null;
    const req = http.request({
      hostname: PROXY, port: PORT,
      path: `${BASE}/companies(${COMPANY})/${path}`,
      method,
      headers: {
        'Content-Type': 'application/json', 'Accept': 'application/json',
        ...(pl ? { 'Content-Length': Buffer.byteLength(pl) } : {}),
        ...(method === 'PATCH' ? { 'If-Match': '*' } : {})
      }
    }, res => { let d=''; res.on('data',c=>d+=c); res.on('end',()=>resolve({status:res.statusCode,body:d}));
  });
  req.on('error', reject);
  if (pl) req.write(pl);
  req.end();
});
}

async function upsert(keyVal, data) {
  const r = await bcReq('POST', ENTITY, data);
  if (r.status === 201 || r.status === 200) return { action:'inserted', key:keyVal, status:r.status };
  const isConflict = r.status===409 || r.status===422 || (r.status===400 &&
r.body.includes('EntityWithSameKey'));
  if (isConflict) {
    const r2 = await bcReq('PATCH', `${ENTITY}('${encodeURIComponent(keyVal)}')`, data);
    return { action:'updated', key:keyVal, status:r2.status };
  }
  return { action:'error', key:keyVal, status:r.status, detail:r.body.substring(0,200) };
}

let staging;
try {
  staging = JSON.parse(fs.readFileSync(STAGING_FILE, 'utf8'));
} catch(e) {
  console.log('[WF-08A] Fichier staging absent – aucun client a synchroniser');
  return [{ json: { table:'etlCustomers', total:0, inserted:0, updated:0, errors:0 } }];
}

const rows = staging.data || [];
if (rows.length === 0) {
  return [{ json: { table:'etlCustomers', total:0, inserted:0, updated:0, errors:0 } }];
}

const results = [];
for (const row of rows) {
  const r = await upsert(row.customer_id, {
    customerId: row.customer_id || '',
```

```

    name:          row.name          || '',
    country:       row.country        || '',
    postCode:      row.post_code      || '',
    locationCode:  row.location_code  || '',
    currencyCode:  row.currency_code  || '',
    salespersonCode: row.salesperson_code || '',
    etlLoadedAt:   NOW
  });
  results.push(r);
}
const inserted = results.filter(r=>r.action==='inserted').length;
const updated  = results.filter(r=>r.action==='updated').length;
const errors   = results.filter(r=>r.action==='error').length;
console.log(`[WF-08A] clients JSON -> BC: ${inserted} inserts | ${updated} updates | ${errors} errors`);
return [{ json: { table:'etlCustomers', total:results.length, inserted, updated, errors } }];

```

WF-08B — Sync articles → BC — Noeud : Sync articles → BC

```

const fs = require('fs');
const http = require('http');

const STAGING_FILE = '/staging/bc/database/articles.json';
const PROXY = '172.18.0.8';
const PORT = 3128;
const BASE = '/BC260/api/etlpipeline/warehouse/v1.0';
const COMPANYY = 'e6317465-9a3f-f011-be59-6045bde988e7';
const ENTITY = 'etlArticles';
const NOW = new Date().toISOString();

function bcReq(method, path, body) {
  return new Promise((resolve, reject) => {
    const pl = body ? JSON.stringify(body) : null;
    const req = http.request({
      hostname: PROXY, port: PORT,
      path: `${BASE}/companies(${COMPANYY})/${path}`,
      method,
      headers: {
        'Content-Type': 'application/json', 'Accept': 'application/json',
        ...(pl ? { 'Content-Length': Buffer.byteLength(pl) } : {}),
        ...(method === 'PATCH' ? { 'If-Match': '*' } : {})
      }
    }, res => { let d=''; res.on('data',c=>d+=c); res.on('end',()=>resolve({status:res.statusCode,body:d}));
  });
  req.on('error', reject);
  if (pl) req.write(pl);
  req.end()
});
}

async function upsert(keyVal, data) {
  const r = await bcReq('POST', ENTITY, data);
  if (r.status === 201 || r.status === 200) return { action:'inserted', key:keyVal, status:r.status };
  const isConflict = r.status===409 || r.status===422 || (r.status===400 &&
r.body.includes('EntityWithSameKey'));
  if (isConflict) {
    const r2 = await bcReq('PATCH', `${ENTITY}('${encodeURIComponent(keyVal)}')`, data);
    return { action:'updated', key:keyVal, status:r2.status };
  }
  return { action:'error', key:keyVal, status:r.status, detail:r.body.substring(0,200) };
}

let staging;
try {
  staging = JSON.parse(fs.readFileSync(STAGING_FILE, 'utf8'));
} catch(e) {
  console.log('[WF-08B] Fichier staging absent – aucun article a synchroniser');
  return [{ json: { table:'etlArticles', total:0, inserted:0, updated:0, errors:0 } }];
}

```

```

const rows = staging.data || [];
if (rows.length === 0) {
  return [{ json: { table:'etlArticles', total:0, inserted:0, updated:0, errors:0 } }];
}

const results = [];
for (const row of rows) {
  const r = await upsert(row.article_id, {
    articleId: row.article_id || '',
    description: row.description || '',
    unitOfMeasure: row.unit_of_measure || '',
    type: row.type || '',
    itemCategory: row.item_category || '',
    unitPrice: parseFloat(row.unit_price) || 0,
    unitCost: parseFloat(row.unit_cost) || 0,
    costingMethod: row.costing_method || '',
    etlLoadedAt: NOW
  });
  results.push(r);
}
const inserted = results.filter(r=>r.action==='inserted').length;
const updated = results.filter(r=>r.action==='updated').length;
const errors = results.filter(r=>r.action==='error').length;
console.log(`[WF-08B] articles JSON -> BC: ${inserted} inserts | ${updated} updates | ${errors} errors`);
return [{ json: { table:'etlArticles', total:results.length, inserted, updated, errors } }];

```

WF-08C — Sync en-têtes expédition → BC — Noeud : Sync salessshipmentheader → BC

```

const fs = require('fs');
const http = require('http');

const STAGING_FILE = '/staging/bc/database/salessshipmentheader.json';
const PROXY = '172.18.0.8';
const PORT = 3128;
const BASE = '/BC260/api/etlpipeline/warehouse/v1.0';
const COMPANY = 'e6317465-9a3f-f011-be59-6045bde988e7';
const ENTITY = 'etlShipmentHeaders';
const NOW = new Date().toISOString();

function bcReq(method, path, body) {
  return new Promise((resolve, reject) => {
    const pl = body ? JSON.stringify(body) : null;
    const req = http.request({
      hostname: PROXY, port: PORT,
      path: `${BASE}/companies(${COMPANY})/${path}`,
      method,
      headers: {
        'Content-Type': 'application/json', 'Accept': 'application/json',
        ...(pl ? { 'Content-Length': Buffer.byteLength(pl) } : {}),
        ...(method === 'PATCH' ? { 'If-Match': '*' } : {})
      }
    }, res => { let d=''; res.on('data',c=>d+=c); res.on('end',()=>resolve({status:res.statusCode,body:d}));
  });
  req.on('error', reject);
  if (pl) req.write(pl);
  req.end();
});
}

async function upsert(keyVal, data) {
  const r = await bcReq('POST', ENTITY, data);
  if (r.status === 201 || r.status === 200) return { action:'inserted', key:keyVal, status:r.status };
  const isConflict = r.status===409 || r.status===422 || (r.status===400 &&
r.body.includes('EntityWithSameKey'));
  if (isConflict) {
    const r2 = await bcReq('PATCH', `${ENTITY}(${encodeURIComponent(keyVal)})`, data);
    return { action:'updated', key:keyVal, status:r2.status };
  }
}

```

```

    }
    return { action:'error', key:keyVal, status:r.status, detail:r.body.substring(0,200) };
}

let staging;
try {
    staging = JSON.parse(fs.readFileSync(STAGING_FILE, 'utf8'));
} catch(e) {
    console.log(['WF-08C'] Fichier staging absent – aucun en-tete a synchroniser');
    return [{ json: { table:'etlShipmentHeaders', total:0, inserted:0, updated:0, errors:0 } }];
}

const rows = staging.data || [];
if (rows.length === 0) {
    return [{ json: { table:'etlShipmentHeaders', total:0, inserted:0, updated:0, errors:0 } }];
}

const results = [];
for (const row of rows) {
    const r = await upsert(row.header_id, {
        headerId:    row.header_id    || '',
        customerNo:  row.customer_id  || '',
        orderNo:     row.order_no     || '',
        shipmentDate: row.shipment_date || '',
        shipToCity:   row.ship_to_city || '',
        shipToCountry: row.ship_to_country || '',
        etlLoadedAt:  NOW
    });
    results.push(r);
}
const inserted = results.filter(r=>r.action==='inserted').length;
const updated   = results.filter(r=>r.action==='updated').length;
const errors    = results.filter(r=>r.action==='error').length;
console.log(['WF-08C'] salesshipmentheader JSON -> BC: ${inserted} inserts | ${updated} updates | ${errors} errors`);
return [{ json: { table:'etlShipmentHeaders', total:results.length, inserted, updated, errors } }];

```

WF-08D — Sync distances GPS → BC — Noeud : Sync distances → BC

```

const fs = require('fs');
const http = require('http');

const STAGING_FILE = '/staging/distances/distances_realtime.json';
const PROXY = '172.18.0.8';
const PORT = 3128;
const BASE = '/BC260/api/etlpipeline/warehouse/v1.0';
const COMPANY = 'e6317465-9a3f-f011-be59-6045bde988e7';
const ENTITY = 'etlDistances';
const NOW = new Date().toISOString();

// BC valide les Date fields contre les périodes comptables ouvertes (mois 11,12,01,02 seulement).
// toAllowedDate remplace les mois hors période par '01' pour éviter l'erreur Internal_DataNotFoundFilter.
// La vraie date reste dans bc_gold.distances (session_date) et dans le session_id (timestamp ms).
function toAllowedDate(d) {
    if (!d || d === '0001-01-01') return d;
    const m = parseInt((d.split('-')[1] || '01'), 10);
    if ([11,12,1,2].includes(m)) return d;
    return d.replace(/(-\d{2})-/, '-01-');
}

// If datetime is missing, fall back to sessionDate at midnight UTC
function toDateTime(dt, fallbackDate) {
    if (dt && dt.trim()) return dt;
    return (fallbackDate || '2026-01-01') + 'T00:00:00Z';
}

function bcReq(method, path, body) {
    return new Promise((resolve, reject) => {

```

```

const pl = body ? JSON.stringify(body) : null;
const req = http.request({
  hostname: PROXY, port: PORT,
  path: `${BASE}/companies(${COMPANY})/${path}`,
  method,
  headers: {
    'Content-Type': 'application/json', 'Accept': 'application/json',
    ...(pl ? { 'Content-Length': Buffer.byteLength(pl) } : {}),
    ...(method === 'PATCH' ? { 'If-Match': '*' } : {})
  }
}, res => { let d=''; res.on('data',c=>d+=c); res.on('end',()=>resolve({status:res.statusCode,body:d}));
});
req.on('error', reject);
if (pl) req.write(pl);
req.end();
});
}

async function upsert(keyVal, data) {
  const r = await bcReq('POST', ENTITY, data);
  if (r.status === 201 || r.status === 200) return { action:'inserted', key:keyVal, status:r.status };
  const isConflict = r.status===409 || r.status===422 || (r.status===400 &&
  r.body.includes('EntityWithSameKey'));
  if (isConflict) {
    const r2 = await bcReq('PATCH', `${ENTITY}('${encodeURIComponent(keyVal)}')`, data);
    return { action:'updated', key:keyVal, status:r2.status };
  }
  return { action:'error', key:keyVal, status:r.status, detail:r.body.substring(0,200) };
}

let accumulated;
try {
  accumulated = JSON.parse(fs.readFileSync(STAGING_FILE, 'utf8'));
} catch(e) {
  console.log('[WF-08D] Fichier staging absent – aucune distance a synchroniser');
  return [{ json: { table:'etlDistances', total:0, inserted:0, updated:0, errors:0 } }];
}

const sessions = accumulated.data || [];
if (sessions.length === 0) {
  console.log('[WF-08D] Staging vide – rien a synchroniser');
  return [{ json: { table:'etlDistances', total:0, inserted:0, updated:0, errors:0 } }];
}

const results = [];
for (const s of sessions) {
  const sessionDate = toAllowedDate(s.session_date || '');
  const r = await upsert(s.session_id, {
    sessionId:      s.session_id                || '',
    sessionDate,
    totalDistanceKm: parseFloat(s.total_distance_km) || 0,
    totalDistanceMeters: parseFloat(s.total_distance_meters) || 0,
    maxSpeedKmh:      parseFloat(s.max_speed_kmh)      || 0,
    avgSpeedKmh:       parseFloat(s.avg_speed_kmh)      || 0,
    totalActiveSeconds: parseInt(s.total_active_seconds) || 0,
    eventCount:        parseInt(s.event_count)          || 0,
    firstEventAt:      toDateTime(s.first_event_at, sessionDate),
    lastEventAt:       toDateTime(s.last_event_at, sessionDate),
    etlLoadedAt:       NOW,
    realSessionDate:   (s.session_date || '').substring(0, 10)
  });
  results.push(r);
}
const inserted = results.filter(r=>r.action==='inserted').length;
const updated = results.filter(r=>r.action==='updated').length;
const errors = results.filter(r=>r.action==='error').length;
console.log('[WF-08D] JSON staging -> BC: ${inserted} inserts | ${updated} updates | ${errors} errors');
return [{ json: { table:'etlDistances', total:results.length, inserted, updated, errors } }];

```

WF-08E — Sync lignes expédition → BC — Noeud : Sync salesshipmentline → BC

```
const fs = require('fs');
const http = require('http');

const STAGING_FILE = '/staging/bc/database/salesshipmentline.json';
const PROXY = '172.18.0.8';
const PORT = 3128;
const BASE = '/BC260/api/etlpipeline/warehouse/v1.0';
const COMPANY = 'e6317465-9a3f-f011-be59-6045bde988e7';
const ENTITY = 'etlShipmentLines';
const NOW = new Date().toISOString();

function bcReq(method, path, body) {
  return new Promise((resolve, reject) => {
    const pl = body ? JSON.stringify(body) : null;
    const req = http.request({
      hostname: PROXY, port: PORT,
      path: `${BASE}/companies(${COMPANY})/${path}`,
      method,
      headers: {
        'Content-Type': 'application/json', 'Accept': 'application/json',
        ...(pl ? { 'Content-Length': Buffer.byteLength(pl) } : {}),
        ...(method === 'PATCH' ? { 'If-Match': '*' } : {})
      }
    });
    req.on('error', reject);
    if (pl) req.write(pl);
    req.end();
  });
}

async function upsert(documentNo, lineNo, data) {
  const r = await bcReq('POST', ENTITY, data);
  if (r.status === 201 || r.status === 200) return { action: 'inserted', key: `${documentNo}|${lineNo}`, status: r.status };
  const isConflict = r.status === 409 || r.status === 422 || (r.status === 400 && r.body.includes('EntityWithSameKey'));
  if (isConflict) {
    const r2 = await bcReq('PATCH', `${ENTITY}(documentNo='${encodeURIComponent(documentNo)}',lineNo=${lineNo})`, data);
    return { action: 'updated', key: `${documentNo}|${lineNo}`, status: r2.status };
  }
  return { action: 'error', key: `${documentNo}|${lineNo}`, status: r.status, detail: r.body.substring(0, 200) };
}

let staging;
try {
  staging = JSON.parse(fs.readFileSync(STAGING_FILE, 'utf8'));
} catch (e) {
  console.log('[WF-08E] Fichier staging absent – aucune ligne expédition à synchroniser');
  return [{ json: { table: 'etlShipmentLines', total: 0, inserted: 0, updated: 0, errors: 0 } }];
}

const rows = staging.data || [];
if (rows.length === 0) {
  return [{ json: { table: 'etlShipmentLines', total: 0, inserted: 0, updated: 0, errors: 0 } }];
}

const results = [];
for (const row of rows) {
  const r = await upsert(row.document_no, row.line_no, {
    documentNo: row.document_no || '',
    lineNo: parseInt(row.line_no) || 0,
    articleId: row.article_id || '',
    quantity: parseFloat(row.quantity) || 0,
    locationCode: row.location_code || '',
    etlLoadedAt: NOW
  });
}
```

```

    results.push(r);
  }
  const inserted = results.filter(r=>r.action==='inserted').length;
  const updated = results.filter(r=>r.action==='updated').length;
  const errors = results.filter(r=>r.action==='error').length;
  console.log(`[WF-08E] salesshipmentline JSON -> BC: ${inserted} inserts | ${updated} updates | ${errors} errors`);
  return [{ json: { table:'etlShipmentLines', total:results.length, inserted, updated, errors } }];

```

WF-08F — Sync lignes e-commerce → BC — Noeud : Sync data_merged → BC

```

const fs = require('fs');
const crypto = require('crypto');
const http = require('http');

const STAGING_FILE = '/staging/bc/flat_files/data_merged.json';
const CHECKSUM_FILE = '/staging/bc/_metadata/checksums/data_merged_bc_sync.json';
const PROXY = '172.18.0.8';
const PORT = 3128;
const BASE = '/BC260/api/etlpipeline/warehouse/v1.0';
const COMPANY = 'e6317465-9a3f-f011-be59-6045bde988e7';
const ENTITY = 'etlEcommerceLines';
const NOW = new Date().toISOString();

// - Skip si données inchangées -
let rawBuf;
try {
  rawBuf = fs.readFileSync(STAGING_FILE);
} catch(e) {
  console.log('[WF-08F] Fichier staging absent - rien à synchroniser');
  return [{ json: { table:'etlEcommerceLines', total:0, inserted:0, updated:0, errors:0, skipped:true } }];
}

const currentHash = crypto.createHash('md5').update(rawBuf).digest('hex');
let storedHash = null;
try { storedHash = JSON.parse(fs.readFileSync(CHECKSUM_FILE, 'utf8')).hash; } catch(_) {}
if (currentHash === storedHash) {
  console.log('[WF-08F] 🚫 data_merged.json inchangé - sync BC ignorée (données statiques)');
  return [{ json: { table:'etlEcommerceLines', total:0, inserted:0, updated:0, errors:0, skipped:true } }];
}

const COUNTRY_MAP = {
  'United Kingdom':'GB', 'France':'FR', 'Germany':'DE', 'Spain':'ES', 'Italy':'IT',
  'Netherlands':'NL', 'Belgium':'BE', 'Switzerland':'CH', 'Austria':'AT', 'Portugal':'PT',
  'Sweden':'SE', 'Norway':'NO', 'Denmark':'DK', 'Finland':'FI', 'Poland':'PL',
  'Australia':'AU', 'USA':'US', 'United States':'US', 'Canada':'CA', 'Japan':'JP',
  'Singapore':'SG', 'Cyprus':'CY', 'Channel Islands':'GB', 'EIRE':'IE',
  'European Community':'EU', 'Unspecified':'', 'Nigeria':'NG', 'Bahrain':'BH',
  'Malta':'MT', 'Lebanon':'LB', 'Lithuania':'LT', 'Iceland':'IS', 'Brazil':'BR',
  'Greece':'GR', 'Czech Republic':'CZ', 'RSA':'ZA', 'Israel':'IL', 'Hong Kong':'HK',
  'Saudi Arabia':'SA', 'United Arab Emirates':'AE', 'Kuwait':'KW'
};

function toISO2(country) {
  if (!country) return '';
  if (country.length <= 2) return country.toUpperCase();
  return COUNTRY_MAP[country] || country.substring(0,2).toUpperCase();
}

function bcReq(method, path, body) {
  return new Promise((resolve, reject) => {
    const pl = body ? JSON.stringify(body) : null;
    const req = http.request({
      hostname: PROXY, port: PORT,
      path: `${BASE}/companies(${COMPANY})/${path}`,
      method,
      headers: {
        'Content-Type': 'application/json', 'Accept': 'application/json',
        ...(pl ? { 'Content-Length': Buffer.byteLength(pl) } : {}),

```

```

        ...(method === 'PATCH' ? { 'If-Match': '*' } : {}))
    }
    }, res => { let d=''; res.on('data',c=>d+=c); res.on('end',()=>resolve({status:res.statusCode,body:d}));
});
req.on('error', reject);
if (pl) req.write(pl);
req.end();
});
}

async function upsert(invoiceNo, lineNo, data) {
    const r = await bcReq('POST', ENTITY, data);
    if (r.status === 201 || r.status === 200) return { action:'inserted', status:r.status };
    const isConflict = r.status===409 || r.status===422 || (r.status===400 &&
r.body.includes('EntityWithSameKey'));
    if (isConflict) {
        const key = `${ENTITY}{invoiceNo='${encodeURIComponent(invoiceNo)}',lineNo=${lineNo}}`;
        const r2 = await bcReq('PATCH', key, data);
        return { action:'updated', status:r2.status };
    }
    return { action:'error', status:r.status, detail:r.body.substring(0,200) };
}

const MAX_ROWS = 10000;
const allRows = JSON.parse(rawBuf.toString('utf8'));
const rows = Array.isArray(allRows) ? allRows.slice(0, MAX_ROWS)
: (Array.isArray(allRows.data) ? allRows.data.slice(0, MAX_ROWS) : []);
console.log(`[WF-08F] ${rows.length} lignes à synchroniser (sur ${Array.isArray(allRows) ? allRows.length :
'?' } total)`);
const results = [];
for (const row of rows) {
    const invoiceNo = row.invoice_no || row.InvoiceNo || '';
    const lineNo = parseInt(row.line_no || row.LineNo || 0);
    const r = await upsert(invoiceNo, lineNo, {
        invoiceNo,
        lineNo,
        customerId: row.customer_id || row.CustomerId || '',
        articleId: row.article_id || row.ArticleId || '',
        quantity: parseFloat(row.quantity || row.Quantity || 0),
        unitPrice: parseFloat(row.unit_price || row.UnitPrice || 0),
        totalAmount: parseFloat(row.total_amount|| row.TotalAmount|| 0),
        country: toISO2(row.country || row.Country || ''),
        orderDate: row.order_date || row.OrderDate || '',
        etlLoadedAt: NOW
    });
    results.push(r);
}

const inserted = results.filter(r=>r.action==='inserted').length;
const updated = results.filter(r=>r.action==='updated').length;
const errors = results.filter(r=>r.action==='error').length;
console.log(`[WF-08F] data_merged → BC: ${inserted} inserts | ${updated} updates | ${errors} errors`);

// Sauvegarder le checksum après sync réussie
fs.mkdirSync('/staging/bc/_metadata/checksums', { recursive: true });
fs.writeFileSync(CHECKSUM_FILE, JSON.stringify({ hash: currentHash, syncedAt: NOW }));

return [{ json: { table:'etlEcommerceLines', total:results.length, inserted, updated, errors,
skipped:false } }];

```


C.12 n8n JavaScript — WF-09BC : BC → Gold PostgreSQL

WF-09BC est le coeur de la réplication BC → Data Warehouse. Il extrait toutes les entités depuis l'API OData BC (via le proxy NTLM 172.18.0.8:3128) et les insère ou met à jour dans le schéma `bc_gold` de PostgreSQL par upsert sur clé naturelle. Déclenché en temps réel par WF-08D après chaque synchronisation GPS.

Noeud : Extraire BC → Gold

```
const http = require('http');
const { Client } = require('pg');

const PROXY = '172.18.0.8';
const PORT = 3128;
const BASE = '/BC260/api/etlpipeline/warehouse/v1.0';
const COMPANY = 'e6317465-9a3f-f011-be59-6045bde988e7';
const PG = { host: '172.18.0.4', port: 5432, database: 'data_warehouse_gold', user: 'nyanu', password: 'nyanu', connectionTimeoutMillis: 10000 };

function s(v, max) { return (v||'').toString().substring(0, max); }

function bcGet(entity, top = 5000) {
  return new Promise((resolve, reject) => {
    const path = `${BASE}/companies/${COMPANY}/${entity}?$top=${top}`;
    const req = http.request({ hostname: PROXY, port: PORT, path, method: 'GET', headers: { 'Accept': 'application/json' } }, res => {
      let d = '';
      res.on('data', c => d += c);
      res.on('end', () => {
        if (res.statusCode >= 400) reject(new Error(`BC ${entity}: HTTP ${res.statusCode}`));
        else { try { resolve(JSON.parse(d).value || []); } catch(e) { reject(new Error('Parse error: ' + d.substring(0,100))); } }
      });
    });
    req.setTimeout(120000, () => { req.destroy(); reject(new Error('Timeout 120s')); });
    req.on('error', reject);
    req.end();
  });
}

const pg = new Client(PG);
await pg.connect();
const results = {};
const errors = {};

try {
  // — 1. Customers —————
  const customers = await bcGet('etlCustomers');
  let cErr = 0;
  for (const r of customers) {
    try {
      await pg.query(
        `INSERT INTO bc_gold.customers
        (customer_id,name,country,post_code,location_code,currency_code,salesperson_code,extracted_at)
        VALUES ($1,$2,$3,$4,$5,$6,$7,NOW())
        ON CONFLICT (customer_id) DO UPDATE SET
        name=$2,country=$3,post_code=$4,location_code=$5,currency_code=$6,salesperson_code=$7,extracted_at=NOW()`,
        [s(r.customerId,50), s(r.name,200), s(r.country,100), s(r.postCode,20),
        s(r.locationCode,50), s(r.currencyCode,20), s(r.salespersonCode,50)]
      );
    } catch(e) { cErr++; console.error('[WF-09BC] ERR customers', r.customerId, e.message.substring(0,80)); }
  }
  results.customers = customers.length - cErr; errors.customers = cErr;

  // — 2. Articles —————
  const articles = await bcGet('etlArticles');
```

```

let aErr = 0;
for (const r of articles) {
  try {
    await pg.query(
      `INSERT INTO bc_gold.articles
(article_id,description,unit_price,uom_code,item_category,extracted_at)
VALUES ($1,$2,$3,$4,$5,NOW())
ON CONFLICT (article_id) DO UPDATE SET
description=$2,unit_price=$3,uom_code=$4,item_category=$5,extracted_at=NOW()`,
      [s(r.articleId,50), s(r.description,300), parseFloat(r.unitPrice||0), s(r.unitOfMeasure,50),
s(r.itemCategory,200)]
    );
  } catch(e) { aErr++; console.error('[WF-09BC] ERR articles', r.articleId, e.message.substring(0,80)); }
}
results.articles = articles.length - aErr; errors.articles = aErr;

// — 3. Shipment Headers —————
const headers = await bcGet('etlShipmentHeaders', 500);
let hErr = 0;
for (const r of headers) {
  try {
    await pg.query(
      `INSERT INTO bc_gold.shipment_headers
(document_no,customer_id,shipment_date,location_code,status,extracted_at)
VALUES ($1,$2,$3,$4,$5,NOW())
ON CONFLICT (document_no) DO UPDATE SET
customer_id=$2,shipment_date=$3,location_code=$4,status=$5,extracted_at=NOW()`,
      [s(r.headerId,50), s(r.customerNo,50),
r.shipmentDate && r.shipmentDate !== '0001-01-01' ? r.shipmentDate : null,
s(r.shipToCity,200), s(r.shipToCountry,10)]
    );
  } catch(e) { hErr++; console.error('[WF-09BC] ERR headers', r.headerId, e.message.substring(0,80)); }
}
results.shipment_headers = headers.length - hErr; errors.shipment_headers = hErr;

// — 4. Shipment Lines —————
const lines = await bcGet('etlShipmentLines', 2000);
let lErr = 0;
for (const r of lines) {
  try {
    await pg.query(
      `INSERT INTO bc_gold.shipment_lines
(document_no,line_no,article_id,quantity,uom_code,gross_weight,net_weight,unit_volume,location_code,shipment_date,extracted_at)
VALUES ($1,$2,$3,$4,$5,$6,$7,$8,$9,$10,NOW())
ON CONFLICT (document_no,line_no) DO UPDATE SET
article_id=$3,quantity=$4,uom_code=$5,gross_weight=$6,net_weight=$7,unit_volume=$8,location_code=$9,shipment_date=$10,extracted_at=NOW()`,
      [s(r.documentNo,50), parseInt(r.lineNo||0), s(r.articleId,50),
parseFloat(r.quantity||0), s(r.uomCode,50),
parseFloat(r.grossWeight||0), parseFloat(r.netWeight||0), parseFloat(r.unitVolume||0),
s(r.locationCode,50), r.shipmentDate||null]
    );
  } catch(e) { lErr++; console.error('[WF-09BC] ERR shiplines', r.documentNo,
e.message.substring(0,80)); }
}
results.shipment_lines = lines.length - lErr; errors.shipment_lines = lErr;

// — 5. Distances —————
const distances = await bcGet('etlDistances');
let dErr = 0;
for (const r of distances) {
  try {
    // realSessionDate = champ Text[10] BC (vraie date GPS, hors contrainte licence)
    const realDate = r.realSessionDate && r.realSessionDate.length === 10 ? r.realSessionDate : null;
    await pg.query(
      `INSERT INTO bc_gold.distances
(session_id,session_date,real_session_date,total_distance_km,total_distance_meters,max_speed_kmh,avg_speed_kmh,total_active_seconds,event_count,first_event_at,last_event_at,extracted_at)
VALUES ($1,$2,$3,$4,$5,$6,$7,$8,$9,$10,$11,NOW())`
    );
  }
}

```

```

        ON CONFLICT (session_id) DO UPDATE SET
session_date=$2,real_session_date=$3,total_distance_km=$4,total_distance_meters=$5,max_speed_kmh=$6,avg_spee
d_kmh=$7,total_active_seconds=$8,event_count=$9,first_event_at=$10,last_event_at=$11,extracted_at=NOW(),
        [s(r.sessionId,100),
        r.sessionDate && r.sessionDate !== '0001-01-01' ? r.sessionDate : null,
        realDate,
        parseFloat(r.totalDistanceKm||0), parseFloat(r.totalDistanceMeters||0),
        parseFloat(r.maxSpeedKmh||0), parseFloat(r.avgSpeedKmh||0),
        parseInt(r.totalActiveSeconds||0), parseInt(r.eventCount||0),
        r.firstEventAt||null, r.lastEventAt||null]
    );
    } catch(e) { dErr++; console.error('[WF-09BC] ERR distances', r.sessionId, e.message.substring(0,80)); }
}
results.distances = distances.length - dErr; errors.distances = dErr;

// — 6. Ecommerce Lines —————
const ecom = await bcGet('etlEcommerceLines', 25000);
let eErr = 0;
for (const r of ecom) {
    try {
        await pg.query(
            `INSERT INTO bc_gold.ecommerce_lines
(invoice_no,line_no,customer_id,article_id,quantity,unit_price,total_amount,country,order_date,extracted_at)
VALUES ($1,$2,$3,$4,$5,$6,$7,$8,$9,NOW())
ON CONFLICT (invoice_no,line_no) DO UPDATE SET
customer_id=$3,article_id=$4,quantity=$5,unit_price=$6,total_amount=$7,country=$8,order_date=$9,extracted_at
=NOW()`,
            [s(r.invoiceNo,30), parseInt(r.lineNo||0), s(r.customerId,30), s(r.stockCode,50),
            parseFloat(r.quantity||0), parseFloat(r.unitPrice||0), parseFloat(r.totalAmount||0),
            s(r.country,2), r.invoiceDate||null]
        );
    } catch(e) { eErr++; console.error('[WF-09BC] ERR ecom', r.invoiceNo, r.lineNo,
e.message.substring(0,80)); }
}
results.ecommerce_lines = ecom.length - eErr; errors.ecommerce_lines = eErr;
} finally {
    await pg.end();
}

const total = Object.values(results).reduce((s,v)=>s+v,0);
const totalErrors = Object.values(errors).reduce((s,v)=>s+v,0);
console.log(`[WF-09BC] ✅ bc_gold mis à jour: ${JSON.stringify(results)} – total ${total} | erreurs $
{totalErrors}`);
return [{ json: { status: totalErrors===0?'success':'partial', ...results, errors, total, extractedAt: new
Date().toISOString() } }];

```

C.13 n8n JavaScript — WF-10BC : Health Monitor BC + Alertes Gmail

WF-10BC vérifie la santé du pipeline ETL : disponibilité de BC (OData ping), cohérence du nombre de sessions Gold vs BC, délai de fraîcheur des données. En cas d'anomalie, une alerte Gmail est envoyée à l'administrateur et l'incident est journalisé dans `bc_gold.incident_log` pour traçabilité.

Noeud : Monitorer BC + Gold

```
const { Client } = require('pg');
const http = require('http');

const PG = { host: '172.18.0.4', port: 5432, database: 'data_warehouse_gold', user: 'nyanu', password: 'nyanu', connectionTimeoutMillis: 10000 };
const PROXY = '172.18.0.8';
const PROXY_PORT = 3128;
const BC_PATH = '/BC260/api/etlpipeline/warehouse/v1.0/companies(e6317465-9a3f-f011-be59-6045bde988e7)/etlCustomers?$top=1';

// 1. Ping BC API
const bc_start = Date.now();
let bc_ok = false, bc_error = null, bc_latency_ms = 0;
try {
  const status = await new Promise((resolve, reject) => {
    const req = http.request(
      { hostname: PROXY, port: PROXY_PORT, path: BC_PATH, method: 'GET',
        headers: { Accept: 'application/json' }, timeout: 10000 },
      res => { res.resume(); res.on('end', () => resolve(res.statusCode)); }
    );
    req.on('error', reject);
    req.on('timeout', () => { req.destroy(); reject(new Error('Timeout 10s')); });
    req.end();
  });
  bc_ok = status < 500;
  if (!bc_ok) bc_error = `HTTP ${status}`;
} catch(e) {
  bc_error = e.message;
}
bc_latency_ms = Date.now() - bc_start;

// 2. Vérifier dernière sync Gold
const pg = new Client(PG);
await pg.connect();
let sync_ok = false, hours_ago = 99, last_sync = null, last_alert_at = null;
try {
  await pg.query(`
    CREATE TABLE IF NOT EXISTS bc_gold.incident_log (
      id SERIAL PRIMARY KEY,
      checked_at TIMESTAMPTZ DEFAULT NOW(),
      bc_ok BOOLEAN, bc_latency_ms INTEGER, bc_error TEXT,
      sync_ok BOOLEAN, hours_since_sync NUMERIC(6,2),
      alert_sent BOOLEAN DEFAULT false, resolved BOOLEAN DEFAULT false
    );`);

  const syncRes = await pg.query(`
    SELECT MAX(extracted_at) AS last_sync,
      EXTRACT(EPOCH FROM (NOW() - MAX(extracted_at)))/3600 AS hours_ago
    FROM bc_gold.customers`);
  last_sync = syncRes.rows[0].last_sync;
  hours_ago = parseFloat(syncRes.rows[0].hours_ago || 99);
  sync_ok = hours_ago < 26;

  // Anti-spam: ne pas alerter si déjà alerté dans les 2h
  const lastAlertRes = await pg.query(`
    SELECT checked_at FROM bc_gold.incident_log
    WHERE alert_sent = true
    AND (bc_ok = false OR sync_ok = false)`);
```

```

        AND checked_at > NOW() - INTERVAL '2 hours'
        ORDER BY checked_at DESC LIMIT 1`);
    last_alert_at = lastAlertRes.rows[0]?.checked_at || null;
  } finally {
    await pg.end();
  }

  const needs_alert = (!bc_ok || !sync_ok) && !last_alert_at;
  const now = new Date().toISOString();

  console.log(`[WF-10BC] BC=${bc_ok?'OK':'KO'} ${bc_latency_ms}ms | Sync=${sync_ok?'OK':'KO'} ($
{hours_ago.toFixed(1)}h) | Alert=${needs_alert}`);

  return [{ json: { bc_ok, bc_error, bc_latency_ms, sync_ok, hours_ago, last_sync, needs_alert, now,
    last_alert_at } }];

```

Noeud : Gmail — Alerte BC KO

```

const d = $input.first().json;
const { Client } = require('pg');
const PG = { host: '172.18.0.4', port: 5432, database: 'data_warehouse_gold', user: 'nyanu', password:
'nyanu' };
const msg = [
  '[WF-10BC] 🚨 ALERTE — Intervention requise',
  'BC=' + (d.bc_ok ? 'OK' : 'KO') + (d.bc_error ? ' (' + d.bc_error + ') ' : '') + ' | Latence=' +
d.bc_latency_ms + 'ms',
  'Sync=' + (d.sync_ok ? 'OK' : 'RETARD ' + d.hours_ago?.toFixed(1) + 'h'),
  'Heure: ' + d.now,
  'Actions: curl -X POST http://172.18.0.5:5678/webhook/run-gold-to-bc'
].join(' | ');
console.warn(msg);
// Écrire dans le log PostgreSQL
const pg = new Client(PG);
try {
  await pg.connect();
  await pg.query(
    'INSERT INTO bc_gold.incident_log (bc_ok,bc_latency_ms,bc_error,sync_ok,hours_since_sync,alert_sent)
VALUES ($1,$2,$3,$4,$5,true)',
    [d.bc_ok, d.bc_latency_ms, d.bc_error||null, d.sync_ok, parseFloat((d.hours_ago||0).toFixed(2))]
  );
} finally { try { await pg.end(); } catch(_) {} }
return [{ json: { ...d, alert_logged: true, alert_channel: 'console+postgres' } }];

```

Noeud : Logger incident (alerte)

```

const input = $input.first().json;
return [{ json: { ...input, alert_sent: true } }];

```

Noeud : Logger statut OK

```

return [{ json: { ...$input.first().json, alert_sent: false } }];

```

Noeud : Écrire bc_gold.incident_log

```

const { Client } = require('pg');
const PG = { host: '172.18.0.4', port: 5432, database: 'data_warehouse_gold', user: 'nyanu', password:
'nyanu' };

const d = $('Monitorer BC + Gold').first().json;

```

```

const alertSent = $input.first().json?.alert_sent ?? false;

const pg = new Client(PG);
await pg.connect();
try {
  await pg.query(
    `INSERT INTO bc_gold.incident_log (bc_ok, bc_latency_ms, bc_error, sync_ok, hours_since_sync,
alert_sent)
  VALUES ($1,$2,$3,$4,$5,$6)\`,
    [d.bc_ok, d.bc_latency_ms, d.bc_error, d.sync_ok,
      parseFloat(d.hours_ago.toFixed(2)), alertSent]
  );
  console.log(`[WF-10BC] Incident logué | alert_sent=${alertSent}`);
} finally {
  await pg.end();
}
return [{ json: { logged: true, ...d } }];

```

Noeud : Résumé monitoring

```

const d = $('Monitorer BC + Gold').first().json;
const status = (d.bc_ok && d.sync_ok) ? '✅ SAIN' : '🚨 PROBLÈME';
console.log(`[WF-10BC] Check terminé: ${status} | BC=${d.bc_ok} | Sync=${d.sync_ok} ($
{d.hours_ago?.toFixed(1)}h)`);
return [{ json: { status, ...d } }];

```

Annexe D — Business Case Complet

1. Méthodologie et Hypothèses

L'étude compare deux trajectoires technologiques pour une PME de 50 collaborateurs (5 utilisateurs BI actifs) sur 3 ans.

- Solution A : Metabase (Open Source) hébergé sur les serveurs internes existants.
- Solution B : Power BI Pro avec stockage des données sur le cloud Azure.
- Hypothèse de gain : Économie de 5h/semaine/manager (5 managers) à 80 CHF/h.
- Facteur de prudence : 50% (Bénéfice retenu : 156 000 CHF sur 3 ans).

2. Comparaison détaillée du TCO sur 3 ans

Poste de coût	Solution A (Meta-base)	Solution B (Power BI)	Justification détaillée des opérations
Licences BI	0 CHF	1 800 CHF	A : Logiciel libre. B : 10 CHF/mois/user (5 users sur 36 mois).
Infrastruc-ture	0 CHF	1 620 CHF	A : Serveurs & Bastion mis à disposition gratuitement. B : Coût estimé Azure SQL (Data Warehouse).
Formation	800 CHF	1 500 CHF	A : Interface simplifiée. B : Complexité accrue (DAX/Power Query).
Mainte-nance	900 CHF	3 195 CHF	A : Support léger du pipeline Python. B : Suivi des mises à jour Azure/Microsoft.
TOTAL TCO	1 700 CHF	8 115 CHF	La Solution A est 4,7x moins chère en exploitation.

--	--	--	--

3. Analyse de la Rentabilité (ROI & Payback)

Indicateur	Solution A	Solution B	Calcul / Formule
Investissement (TCO)	1 700 CHF	8 115 CHF	Somme des coûts de fonctionnement.
Bénéfice Net (3 ans)	156 000 CHF	156 000 CHF	Gain de productivité avec prudence 50%.
ROI (Retour sur Inv.)	9 076 %	1 822 %	

Payback Period	< 1 mois	~2 mois	Délai pour amortir les coûts de fonctionnement.
-----------------------	--------------------	----------------	---

4. Analyse Qualitative et Risques

Critère	Solution A (Metabase)	Solution B (Power BI)
Atout majeur	Coût de fonctionnement quasi nul.	Capacité d'analyse poussée (IA, calculs complexes).
Infrastructure	Maximise l'existant (Serveurs/Bastion).	Dépendance totale au Cloud Microsoft Azure.
Flexibilité	Totale (Open Source, pas de contrat).	Limitée par les tarifs et formats propriétaires.
Risque principal	Maintenance interne du script Python.	Augmentation imprévisible des coûts Azure.

5. Recommandation Stratégique

La **Solution A (Metabase)** est la recommandation prioritaire.

1. **Valorisation des actifs existants** : En utilisant les serveurs et le bastion déjà disponibles gratuitement, l'entreprise transforme une charge potentielle en avantage compétitif.

2. **Rentabilité immédiate** : Un ROI de plus de 9 000% est un argument financier imparable. Le coût mensuel de fonctionnement revient à moins de **48 CHF** pour l'ensemble de la structure.
3. **Indépendance technique** : La solution reste interne, sécurisée via le bastion, et ne subit pas les fluctuations de prix des abonnements Cloud tiers.

Conclusion : La Solution A permet un déploiement "zéro risque" avec une efficacité maximale. La Solution B ne devra être envisagée que si des besoins de calculs statistiques extrêmement complexes (non couverts par SQL/Metabase) apparaissent à l'avenir.